System Hub

System Hub

役割

このVaultにおける「思考の入口」と「制御塔」。
すべての作業はここから開始する。

基本原則

- 必ず「問い」から入る
- いきなり書かない（まず分類する）
- ノードを辿ってから出力する
- 出力後は必ず検証する

人力運用フロー（必須）

1. Intent Interpretationで問いの型を判定
2. Mode Selectionで処理モードを決定
3. Task Routingで行き先を決める
4. Goal Framingで目的を固定
5. Thinking Protocolに従って作業
6. Constraint Monitorで検証
7. 必要に応じてTool Orchestration

分岐（モード）

- 即答 → Direct Answer Mode
- 情報不足 → Search-Augmented Mode
- Vault参照 → File-Grounded Mode
- 複雑 → Stepwise Reasoning Mode
- ノート生成 → Artifact Generation Mode

NG行動

- いきなり結論を書く
- 既存ノートを見ない
- 仮説なしで探索する

Zettelkasten OS Overview

Zettelkasten OS Overview

概要

このVaultは「思考の再現機械」である。
人間が同じ手順で辿れば同じ結論に近づく構造を持つ。

レイヤー構造（必ず意識する）

- System（制御）→ 今ここ
- Ontology（言葉の定義）
- World Model（世界の構造）
- Thinking Engine（推論）
- Knowledge Graph（接続）
- Case（事例）
- Decision（意思決定）

最重要ルール

1. 抽象と具体を混ぜない

- 抽象 → 02_world_model/Concept
- 具体 → 05_case_intelligence/cases

---

2. 昇格ルール

- Case → Pattern → Mechanism
  参照:
- Pattern Extraction Method
- Case to Pattern Promotion

---

3. 必ず接続する

孤立ノート禁止

参照:

- 99_oldzettelkasten/04_knowledge_graph/Concept Links
- 99_oldzettelkasten/04_knowledge_graph/Knowledge Graph

人力での使い方

1. まずCaseを見る
2. Patternを探す
3. Mechanismに当てる
4. Structureで整理する

Thinking Protocol

Thinking Protocol

目的

思考を再現可能にする

基本フロー

① 分解

問いを要素に分ける

---

② 探索

以下を探す

- Concept
- Pattern
- Mechanism

参照:

- 99_oldzettelkasten/Concept Hub
- old_zettelkasten/pattern/Pattern Hub
- 02_zettelkasten/02_world_model/mechanism/Mechanism Hub

---

③ 接続

関係を作る
→ 99_oldzettelkasten/04_knowledge_graph/Concept Links

---

④ 推論

- 因果
- 構造

- 比較

参照:

- 99_oldzettelkasten/04_knowledge_graph/Reasoning Strategy

---

⑤ 検証

→ Constraint Monitor

NG

- いきなり結論
- ノードを見ない

Flow Selection Engine

Flow Selection Engine

1. 目的

入力された問いに対して、適切な推論フローを選択する。

---

2. フロー一覧

- Case Flow（事例・観察）
- Solution Flow（問題解決・設計）
- Hypothesis Flow（仮説検証）
- Search Flow（探索・理解）
- Research Flow（厳密検証）
- Cross Domain Flow（応用・転用）

---

3. 判定アルゴリズム

Step 1：問いの型判定

条件	フロー
「なぜ」「どうして」＋具体事例	Case Flow
「どうすれば」「設計」「改善」	Solution Flow
「～では？」「仮説」	Hypothesis Flow
「とは何か」「整理」「一覧」	Search Flow
「検証」「データ」「分析」	Research Flow
「別領域」「応用」「転用」	Cross Domain Flow

Step 2：優先順位

複数該当する場合：

1. Solution（最優先）
2. Hypothesis

3. Case
4. Search
5. Cross Domain
6. Research（明示的な場合のみ）

---

4. 出力形式

```
Flow Type: ○○
Reason: ○○
Next Step:
  - 呼び出すHub
  - 使用ノート
```

---

5. フロー別呼び出し先

Case Flow

- Case Hub
- Pattern Extraction
- Pattern Hub
- Structure Hub
- Mechanism Hub

Solution Flow

- Problem Type
- Structure Hub
- Mechanism Hub
- Method Hub
- Decision Hub

Hypothesis Flow

- Hypothesis
- Structure Hub

- Mechanism Hub
- Mismatch Detection
- Research（必要時）

Search Flow

- Graph Traversal
- Concept Links
- Pattern / Structure / Mechanism Hub

Cross Domain Flow

- Cross Domain Mapping
- Bridge Concept

Research Flow

- Research Loop
- dataset / test / result

---

6. 注意点

- Flowは1つに固定しない（必要に応じて切り替え）
- 初期選択が最も重要（誤ると探索コスト増大）
- 「わからない場合はSearch Flowから開始」

---

7. 典型パターン

- Case → Structure → Mechanism（観察・分析）
- Problem → Structure → Mechanism → Solution（設計）
- Hypothesis → Mechanism → Test（検証）
- Search → Pattern → Structure（理解）

Artifact Generation Mode

Artifact Generation Mode

Artifact Generation Mode は、回答文ではなく、**ユーザーがそのまま使える成果物そのものを完成させる運転モード**である。
ここでいう成果物とは、ノート、文書、コード、表、設定ファイル、スライド構成、テンプレートなど、**再利用・保存・貼り付け・実行の対象となる具体物**を指す。

要点

- このモードの目的は説明ではなく完成物の生成である
- 形式適合性と再利用性が特に重要になる
- 中身の正しさだけでなく、貼り付け可能性・実行可能性・編集可能性が求められる
- 説明文は補助であり、本体は成果物である
- 完成条件は「読める」ではなく「使える」で判断する

なぜ必要か

ユーザーはしばしば、知識説明ではなく具体物を求める。

例:

- ノート本文を作って
- そのまま貼れる形で出して
- Mermaid付きで構造化して
- JSONで返して
- コードを書いて
- テンプレートを作って
- スプレッドシートやスライドの元案を作って

このような依頼では、分析や説明だけでは足りない。
必要なのは、**仕様に合った完成物を出すこと**である。

そのため Artifact Generation Mode は、**推論結果を成果物へ変換する専用モード**として必要になる。

適用場面

1. ノート生成

例:

- Obsidianノート
- Hubノート
- Structureノート
- テンプレートノート

2. コード生成

例:

- Python
- SQL
- JSON
- YAML
- Mermaid
- HTML

3. 文書生成

例:

- 提案書
- メール草案
- 契約テンプレート
- 説明資料本文

4. 構造化成果物生成

例:

- 表
- チェックリスト
- 手順書
- スキーマ
- データ定義

適用してはいけない場面

- 単なる質問応答
- 軽い意見交換
- 根拠確認が先の案件
- まず比較や分析が必要で、成果物形が未定の場面

この場合は Direct Answer Mode や Comparative Reasoning Mode を先行させる方がよい。

中核機能

1. Artifact Type Identification

何を作るのかを特定する。

対象:

- ノート
- コード
- 文章
- 表
- 設計書
- スライド骨子
- データ形式
- 設定ファイル

成果物タイプによって、完成条件が大きく変わる。

---

2. Format Constraint Binding

形式要件を明確に取り込む。

例:

- コードブロックで出す
- YAML frontmatter を含める
- Mermaidを閉じる
- JSON valid にする
- Markdownで整形する
- 表の列構成を守る

Artifact Generation では、形式要件は本質的要件である。

3. Content Planning

成果物の中身を組み立てるための構成案を立てる。

内容:

- セクション構成
- 項目順
- 必須要素
- 再利用単位
- 見出し体系
- 依存部品

---

4. Template Realization

必要に応じてテンプレートや定型構造へ落とし込む。

例:

- frontmatter
- 見出しテンプレート
- 関連ノート欄
- コード雛形
- 変数配置
- 表形式の列

---

5. Artifact Drafting

実際の成果物本文を生成する。

この段階では、説明の美しさよりも、

- そのまま使えるか
- 仕様を守っているか
- 欠けがないか

が重要になる。

---

6. Integrity Checking

成果物として成立しているかを確認する。

確認項目:

- 形式が壊れていないか
- 必須要素が揃っているか
- コードブロックが閉じているか
- Markdown構造が破綻していないか
- セクション順が妥当か

---

7. Delivery Packaging

成果物本体と、必要最小限の補足を分けて提示する。

原則:

- 本体優先
- 補足は短く
- 余計な説明で成果物を埋めない

成果物生成の基本形

1. 何を作るか決める
2. 形式要件を確認する
3. 構成を設計する
4. 本文を生成する
5. 壊れていないか検査する
6. そのまま使える形で渡す

下位構造

A. Artifact Spec Reader

成果物仕様を読む部分。

B. Structure Planner

成果物構成を設計する部分。

C. Template Binder

定型部品を接続する部分。

D. Draft Generator

本文を作る部分。

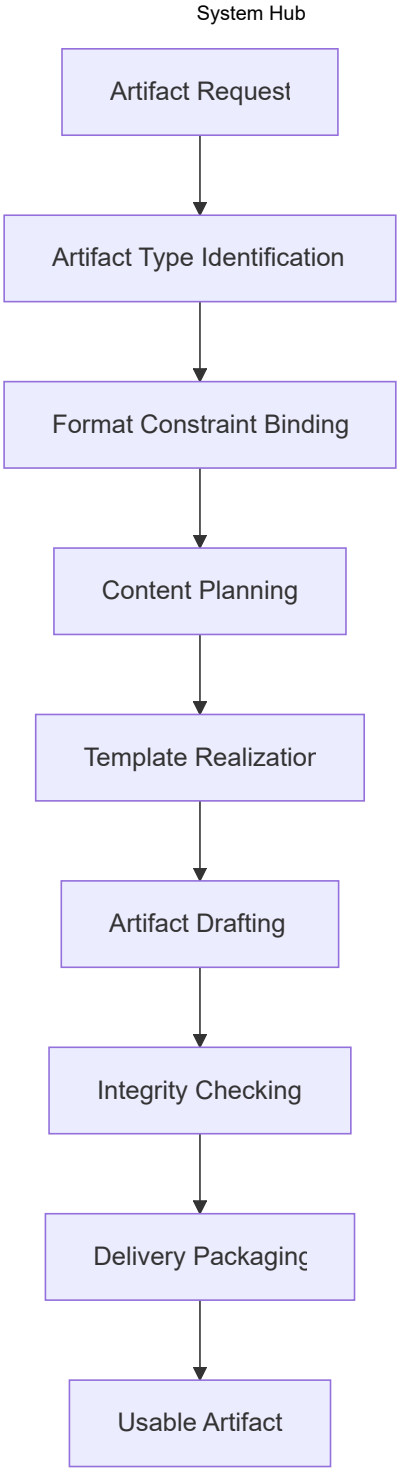
E. Integrity Checker

形式破綻や欠落を検査する部分。

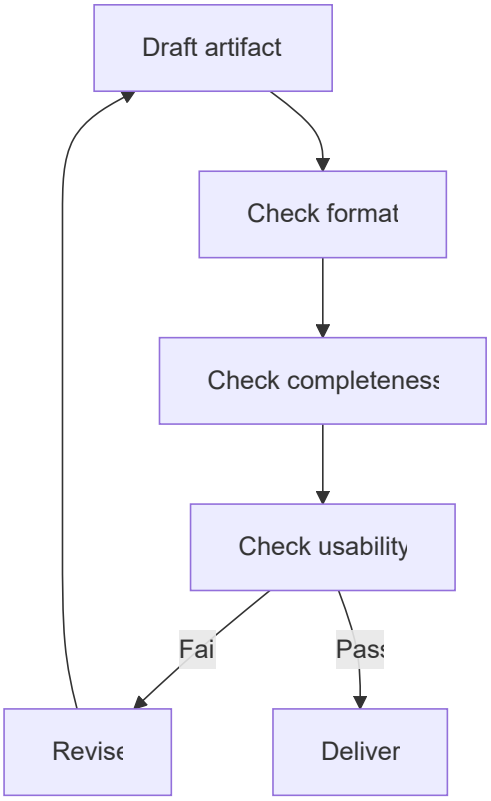
F. Delivery Formatter

本体を使いやすい形で提示する部分。

全体構造


System Hub
Artifact Request
Artifact Type Identification
Format Constraint Binding
Content Planning
Template Realizatior
Artifact Drafting
Integrity Checking
Delivery Packaging
Usable Artifact

生成検査ループ



典型例

入力	Artifact Generation Mode の動き
ノートを作って	frontmatter付き本文を完成させる
コードを書いて	実行可能性を意識して生成する
Mermaidで図を書いて	構文破綻なく図を生成する
表にして	列構成を保って出す
貼り付ける部分だけ	本体のみを整形して返す

よくある失敗

1. 説明が本体を食う

成果物より前置きが長くなる。

2. 形式が壊れる

コードブロックやJSONやMermaidが閉じない。

3. 仕様漏れ

frontmatterや必須欄が欠ける。

4. 使えない完成物になる

読めるが、そのまま貼れない・実行できない。

5. 本体と補足が混ざる

ユーザーがどこを使えばよいか分かりにくくなる。

設計原則

- 成果物本体を最優先する
- 形式要件を最初に固定する
- 構成を先に設計する
- テンプレートを活用する
- 生成後に必ず整合性を確認する
- 補足より再利用性を重視する
- そのまま使える状態で渡す

位置づけ

Artifact Generation Mode は、
推論や設計の結果を、実際に使える完成物へ変換する成果物生成モードである。

これが強いと、

- 回答が作業成果へ直結し
- ユーザーの手直しが減り
- LLMが実務制作パートナーとして機能する

したがってこのモードは、単なる文章生成ではなく、
仕様拘束下で使える成果物を完成させる制作実行モードである。

関連ノート

- Mode Selection
- Comparative Reasoning Mode
- Stepwise Reasoning Mode
- Direct Answer Mode
- LLM Output Layer
- Tool Orchestration

Constraint Monitor

Constraint Monitor

Constraint Monitor は、LLM の処理全体に対して、**守るべき条件・越えてはいけない境界・満たすべき形式要件**を監視し続ける構造である。
これは単なる安全フィルタではなく、**実行前・実行中・出力前の各段階で制約違反を防ぎ、必要なら進路を修正する監督機構**である。

要点

- 制約は最後にまとめて確認するのではなく、全工程にかける
- 制約には安全だけでなく、形式・鮮度・根拠・権限・ユーザー指定も含まれる
- Constraint Monitor は「禁止」を判定するだけでなく、進路変更も行う
- 制約違反を完全拒否にせず、可能なら安全な代替経路へ誘導する
- 良い応答は、内容の正しさだけでなく制約遵守でも評価される

制約とは何か

ここでいう制約とは、LLM が自由に出力・行動してよい範囲を限定する条件群である。
主なものは以下である。

- 安全制約
- 法的・倫理的制約
- ユーザー指示制約
- フォーマット制約
- 最新性確認制約
- 引用制約
- ツール使用規則
- 接続ソース制約
- 長さ・粒度・言語制約
- 権限制約

Constraint Monitor は、これらを一つの監視系として扱う。

中核機能

1. Constraint Detection

入力・計画・ツール利用・出力案の各段階から、関係する制約を抽出する。

例:

- 「最新情報」→ web確認制約
- 「JSONで」→ 形式制約
- 「送って」→ 外部書き込み制約
- 医療・法律・金融 → 高精度確認制約
- 添付ファイルあり → 根拠限定制約
- 明確な危険意図 → 安全制約

---

2. Applicability Check

その制約が今回のケースに本当に適用されるかを判定する。

同じ制約でも、すべてのタスクに一律適用されるわけではない。
たとえば、

- 一般知識の説明なら web不要
- 最新株価なら finance必須
- PDF分析なら PDF読解手順が必要
- メール送信は、下書きか即送信かを区別する

Constraint Monitor は、**何が適用対象か**を見極める。

---

3. Ongoing Monitoring

処理途中で制約違反の兆候が出ていないかを継続監視する。

監視対象:

- 記憶だけで最新情報に答えようとしていないか
- 許可なく外部変更しようとしていないか
- フォーマット逸脱が起きていないか
- 根拠の薄い断定が混じっていないか
- 禁止領域へ深掘りしていないか

---

4. Intervention

違反または違反リスクを見つけたとき、進行を修正する。

修正方法:

- web検索へ切り替える
- 出力形式を再構成する
- 拒否モードへ切り替える
- 代替案へ誘導する
- ツール使用順を変更する
- 断定表現を緩和する

つまり Monitor は、受動的警報機ではなく、**能動的介入機構**である。

---

5. Compliance Verification

最終出力直前に、主要制約を満たしているかを検査する。

検査項目:

- 指定言語か
- 指定形式か
- 必要な引用があるか
- 必要な最新確認を行ったか
- 禁止表現がないか
- 書き込み行為がユーザー意図に合うか
- 不確実性の扱いが適切か

---

6. Constraint Logging

何の制約をどう適用したかを内部的に保持できるようにする。

利点:

- 同じ誤りの再発防止
- 処理判断の一貫性向上
- 後続層への引き継ぎ
- 部分達成の説明容易化

制約の主要分類

A. Safety Constraint

危険行為、有害支援、プライバシー侵害、不適切内容を制御する。

B. Freshness Constraint

最新情報が必要な場合、外部確認を必須にする。

C. Format Constraint

JSON、コードブロック、表、文書形式などの指定を守らせる。

D. Grounding Constraint

ファイル・引用・根拠に基づく回答を要求する。

E. Permission Constraint

メール送信、予定作成、削除など外部変更の権限条件を監視する。

F. Instruction Constraint

ユーザーの明示指示や会話内ルールを守らせる。

G. Scope Constraint

聞かれた範囲を超えすぎないようにする。

下位構造

A. Constraint Extractor

入力やタスクから制約候補を抽出する部分。

B. Constraint Classifier

制約の種類と優先度を分類する部分。

C. Runtime Watcher

処理途中の制約逸脱を監視する部分。

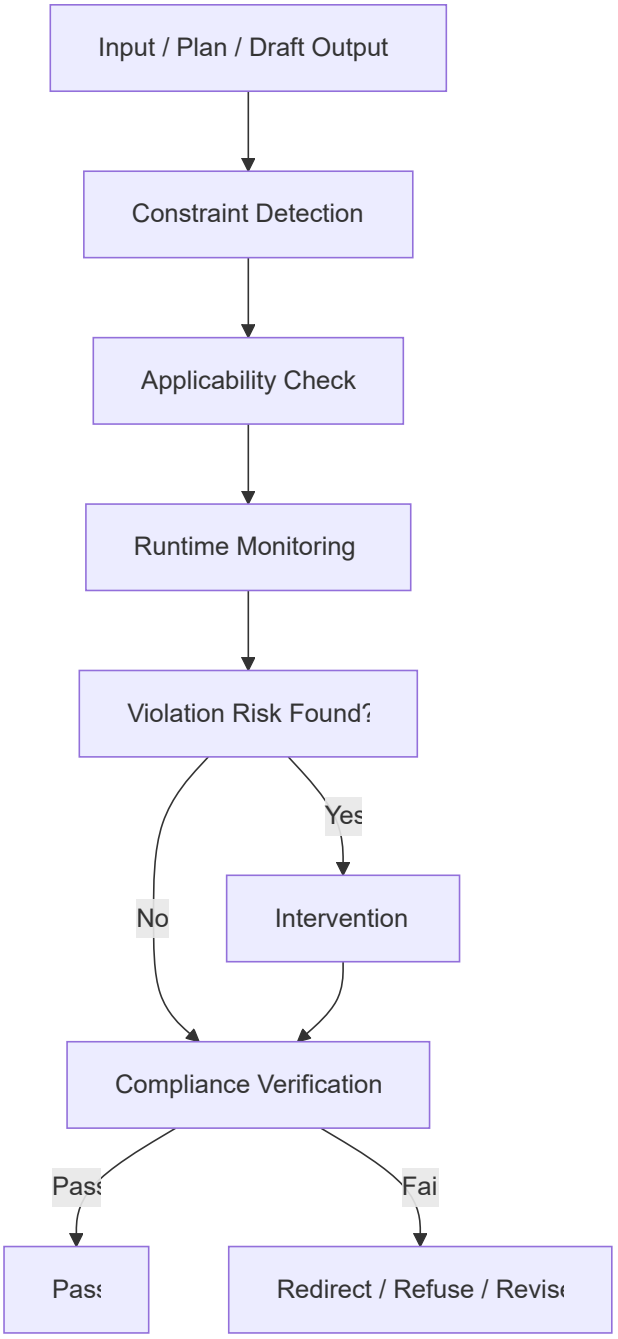
D. Compliance Gate

最終出力前の通過判定を行う部分。

E. Safe Redirector

違反時に拒否・縮退・代替案提示へ導く部分。

全体構造


Input / Plan / Draft Output
Constraint Detection
Applicability Check
Runtime Monitoring
Violation Risk Found?
Yes
No
Intervention
Compliance Verification
Pass
Fai
Pass
Redirect / Refuse / Revise

制約監視の時間軸

Before Executior
During Execution
Before Output

典型介入例

状況	介入内容
最新情報を記憶で答えそう	web検索を挿入する
JSON指定なのに解説文が混ざる	出力構造を再整形する
危険手順へ踏み込みそう	拒否または抽象化へ切り替える
根拠不十分な断定がある	表現を弱める
ユーザー指定言語から逸脱	言語を戻す
外部変更が曖昧	実行せず草案化する

よくある失敗

1. 制約を最後にしか見ない

途中で間違った方向に進み、最後に大きく修正が必要になる。

2. 安全制約しか見ていない

形式・鮮度・引用・権限などの実務制約を見落とす。

3. 一律拒否する

安全に答えられる範囲まで縮退できるのに、全面拒否してしまう。

4. 制約適用が過剰

必要ない web検索や過剰な但し書きで、有用性を落とす。

5. 表面だけ整える

中身は要件未達なのに、文面だけきれいにして通してしまう。

設計原則

- 制約は前・中・後で監視する
- 制約の適用範囲を見極める
- 違反時は停止だけでなく修正も考える

- 安全性と有用性の両立を目指す
- 形式制約も安全制約と同等に重視する
- 制約を守った上で最大限の部分達成を行う

位置づけ

Constraint Monitor は、
LLM の自由な生成を、実用可能な範囲に保つ監査・補正構造である。

これがないと、

- 最新性を誤り
- 形式を破り
- 安全境界を越え
- ユーザー意図から逸脱しやすくなる

したがってこの構造は、抑圧装置ではなく、
信頼可能な応答を成立させる品質保証機構である。

関連ノート

- LLM Control Layer
- Tool Orchestration
- Task Routing
- Termination Control
- Safety Filter
- LLM Output Layer
- LLM Output Layer]]
- LLM Output Layer

Direct Answer Mode

Direct Answer Mode

Direct Answer Mode は、ユーザーの依頼に対して、**外部検索や重い多段処理を挟まず、内部知識と軽量な推論によって直接答える運転モード**である。
このモードの本質は、単なる即答ではなく、**不要な探索を省きつつ、十分な正確性と明快さで最短距離の応答を返すこと**にある。

要点

- 安定知識や低変動知識に対して使う
- 目的は速度と簡潔性の両立である
- 外部情報取得を行わないぶん、適用範囲の見極めが重要になる
- 深い調査より、問いに対する直接的な解答を優先する
- ただし、単純化しすぎて根拠や注意点を落としてはいけない

なぜ必要か

すべての質問に検索や長い推論を挟むと、応答は重く遅くなる。
一方で、多くの問いは、

- 定義説明
- 概念整理
- 基礎的比較
- 一般原理の解説
- 既知内容の構造化

のように、内部知識と短い推論で十分答えられる。

このような場面で Direct Answer Mode を使うことで、

- 速度が出る
- 応答が簡潔になる
- ユーザーの認知負荷が下がる
- 不要なツール依存を避けられる

したがってこれは、**LLM の基本応答能力を最も効率よく使う標準モード**である。

適用場面

1. 安定知識の説明

例:

- 概念の意味
- 基本原理
- 一般的な違い
- 古典的知識

2. 軽い論理整理

例:

- この考え方は妥当か
- どう整理すべきか
- 構造上どう位置づけるか

3. ユーザー文脈に基づく継続作業

例:

- 続きを作る
- 直前のフォーマットを踏襲する
- ノート構造の延長を書く

4. 形式変換・再表現

例:

- 言い換え
- 要約
- 箇条書き化
- コードブロック化

適用してはいけない場面

Direct Answer Mode が不適切なのは、主に次のような場合である。

- 最新性が必要
- 現在の制度や価格が関わる
- ニッチで不確かな固有情報が必要

- ファイル本文が根拠となる
- 外部状態の読み取りが必要
- 厳密な引用が必要
- 実行操作が必要

この場合は、Search-Augmented Mode や File-Grounded Mode などへ切り替えるべきである。

中核機能

1. Immediate Interpretation

入力を素早く解釈し、主質問を短く明確に捉える。

ここでは、

- 何を聞かれているか
- どの範囲で答えるか
- どの深さで足りるか

を即時に見極める。

---

2. Internal Knowledge Retrieval

内部知識から、関連概念・原理・既知パターンを取り出す。

重要なのは、網羅性よりも**質問に直接関係する知識を優先すること**である。

---

3. Light Reasoning

必要最小限の推論で、知識を問いに適合させる。

内容:

- 定義の適用
- 分類
- 軽い比較
- 論理接続
- 簡潔な因果整理

このモードでは、複雑な探索木ではなく、**短い推論経路**を使う。

---

4. Answer Prioritization

まず何を答えるべきかを決める。

原則:

- 結論先出し
- 余計な前置きを減らす
- 主質問に直接答える
- 補足は必要分だけ添える

---

5. Minimal Sufficiency Check

簡潔であっても、最低限の十分性を満たしているか確認する。

確認項目:

- 問いに答えているか
- 誤解を招かないか
- 必要な限定や但し書きがあるか
- 構造がわかりやすいか

処理の特徴

速度優先

深掘り前にまず答える。

構造優先

長い説明より、明快な整理を重視する。

低コスト

ツールや検索を使わずに済む。

限定的確信

不確かな場合は断定を避ける必要がある。

下位構造

A. Fast Intent Reader

依頼の中心を素早く読む部分。

B. Internal Retriever

内部知識から必要項目を取り出す部分。

C. Light Reasoner

短い推論で答えを組み立てる部分。

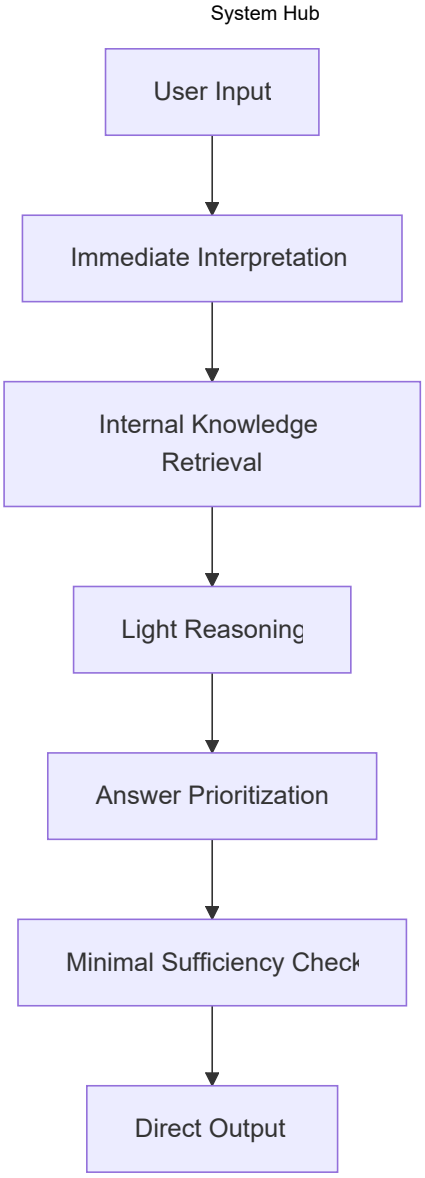
D. Direct Composer

結論先出しで応答文を組み立てる部分。

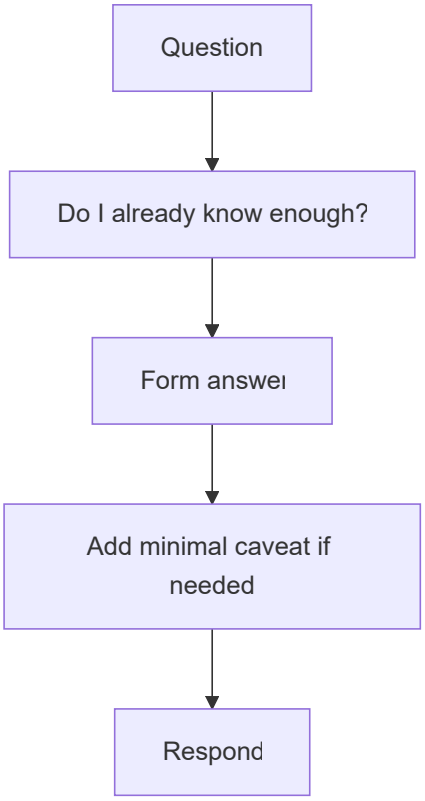
E. Sufficiency Checker

短くても必要条件を満たしているか確認する部分。

全体構造


System Hub
User Input
Immediate Interpretation
Internal Knowledge Retrieval
Light Reasoning
Answer Prioritization
Minimal Sufficiency Check
Direct Output

典型フロー



典型例

入力	Direct Answer Mode の応答傾向
概念の違いを教えて	定義と差分を簡潔に出す
この構造は何ですか	位置づけと役割を説明する
どう整理するのがよいですか	基本方針を即答する
続けてください	直前形式を踏襲して続行する
この文章を短くして	直接圧縮して返す

よくある失敗

1. 検索すべき内容まで直答する

最新性や外部根拠が必要なのに内部知識で済ませてしまう。

2. 速さを優先しすぎて粗くなる

問いには答えるが、重要な条件や限定を落とす。

3. 前置きが長い

Direct なのに直接答えず、背景説明から入ってしまう。

4. 深掘り不足

比較や構造整理が必要なのに、定義だけで終わる。

5. 自信過剰

不確かな内容を断定口調で出してしまう。

設計原則

- まず問いに答える
- 安定知識に限定して使う
- 必要最小限の推論で組み立てる
- 結論を先に置く
- 補足は必要分だけ付ける
- 不確実なら軽く限定する
- 検索が必要ならすぐ別モードへ切り替える

位置づけ

Direct Answer Mode は、
LLM の内部知識と軽量推論を最短距離で出力へ変える基本応答モードである。

これが強いと、

- 速く
- 明快で
- 過不足の少ない

応答ができる。
一方でこれを過信すると、最新性や根拠性を誤る。
したがってこのモードは、**最も基本だが、適用範囲の見極めが重要な標準運転形態**である。

関連ノート

- Mode Selection
- Search-Augmented Mode
- File-Grounded Mode
- Task Routing
- Termination Control
- LLM Output Layer
- LLM Output Layer]]
- LLM Output Layer]]
- LLM Output Layer]]
- LLM Output Layer]]

File-Grounded Mode

File-Grounded Mode

File-Grounded Mode は、ユーザーが与えたファイルや接続ソース上の文書を**主要な根拠源として読み取り、その内容に基づいて回答・要約・分析・生成を行う運転モード**である。
このモードの本質は、一般知識で埋めることではなく、**特定文書に根拠を固定し、その範囲内で忠実かつ有用に処理すること**にある。

要点

- 主根拠は外部公開知識ではなく、与えられた文書や資料である
- 回答は「その文書に何が書いてあるか」に強く拘束される
- 要約・比較・抽出・変換・引用対応が中心になる
- 必要に応じて一般知識は補助的に使うが、文書根拠を上書きしてはいけない
- 文書未確認部分を推測で埋めすぎないことが重要である

なぜ必要か

多くの実務依頼では、一般知識よりも特定資料の内容が重要になる。
たとえば、

- アップロードされたPDFの要約
- Google Drive上のドキュメントの確認
- 仕様書の比較
- 議事録の抽出
- 提案書や契約書のレビュー
- スライドやシートの読み取り

こうした場面では、一般的に正しいことを述べるだけでは足りない。
必要なのは、**その文書に何があり、何がなく、どこに根拠があるか**を把握することである。

そのため File-Grounded Mode は、**文書基盤の読解と応答を安定させる専用モード**として必要になる。

適用場面

1. 文書内容の要約

例:

- このPDFを要約して
- この資料の要点をまとめて

2. 文書内容の抽出

例:

- この契約書の期限条項を抜いて
- 重要な数値を拾って

3. 文書間比較

例:

- この2つの提案書の違いを比較して
- バージョン差分を見て

4. 文書に基づく生成

例:

- この資料をもとにメールを書いて
- この議事録からアクション項目を作って

5. 根拠限定の質問応答

例:

- このファイルにそう書いてありますか
- この資料の範囲で答えて

適用してはいけない場面

- 添付資料がない一般質問
- 最新ニュースの確認
- 公開Web情報が主対象の調査
- 純粋な創作や相談
- 文書と無関係な一般概念説明

この場合は Direct Answer Mode や Search-Augmented Mode の方が適切である。

中核機能

1. Document Targeting

どのファイル・どの資料・どの範囲を根拠にするかを特定する。

対象:

- アップロードファイル
- 接続ドキュメント
- スライド
- スプレッドシート
- PDF
- スレッドや記録文書

ここで根拠対象を誤ると、全体がずれる。

---

2. Document Reading

対象文書を読み取り、必要部分へアクセスする。

読む対象:

- 本文
- 見出し
- 表
- 数値
- 箇条書き
- ページ構造
- 画像や図表の周辺文脈

このモードでは、単なる全文走査ではなく、**問いに必要な読解単位**が重要である。

---

3. Relevance Mapping

文書内のどの部分が質問に関係するかを特定する。

見る観点:

- キーワード一致

- 意味的近接
- セクション構造
- 役割上の重要箇所
- 数値や固有名詞の出現
- 依頼された範囲指定

---

4. Grounded Extraction

関係箇所から、必要な主張・数値・論点・アクションを抽出する。

重要なのは、

- 文書にあること
- 文書にないこと
- 文書から推定できること

を分けることである。

---

5. Grounded Reasoning

抽出した文書内容に基づいて、必要な整理・比較・要約・変換を行う。

ここでの推論は、**文書拘束下の推論**である。
つまり自由な一般知識推論ではなく、資料に書かれた内容を壊さない範囲で行う。

---

6. Citation or Trace Binding

回答のどの部分が文書のどこに基づいているかを対応づける。

用途:

- 検証可能性の確保
- ユーザーによる追跡
- 解釈過剰の防止
- 比較結果の根拠明示

---

7. Gap Handling

文書に書かれていない場合、その不在を適切に扱う。

対応:

- 書かれていないと明示する
- 資料からは確認できないと述べる
- 必要に応じて補助検索へ切り替える
- 推定で埋めすぎない

文書根拠モードの基本姿勢

文書優先

まず資料に忠実であること。

推測抑制

書かれていないことを安易に補わないこと。

範囲明示

どの資料・どの範囲に基づくかを意識すること。

構造読解

単語一致だけでなく、見出しや文脈も読むこと。

下位構造

A. File Locator

対象ファイルを特定する部分。

B. Section Reader

必要箇所を読む部分。

C. Relevance Mapper

質問と文書箇所を対応づける部分。

D. Grounded Extractor

文書中の事実や論点を抽出する部分。

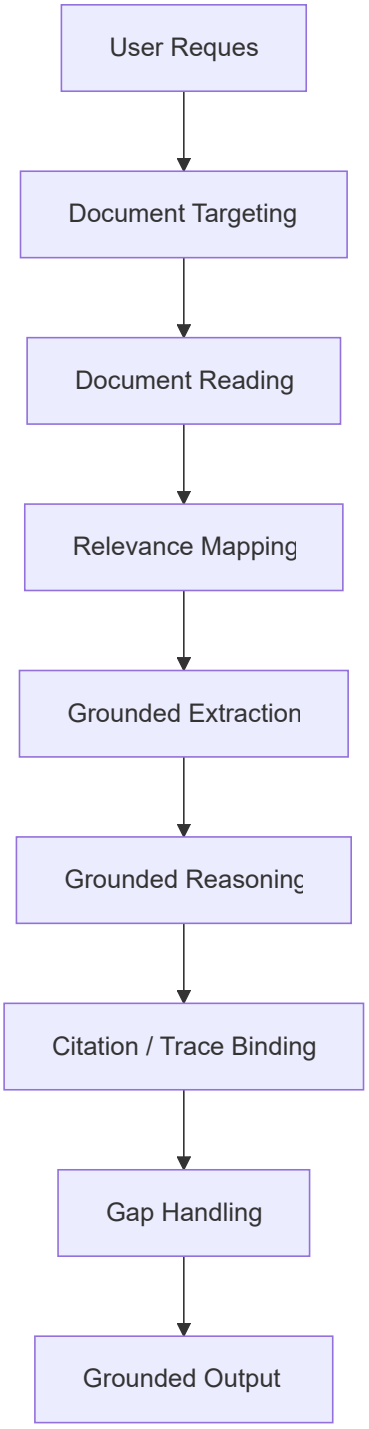
E. Trace Binder

回答と根拠位置を結びつける部分。

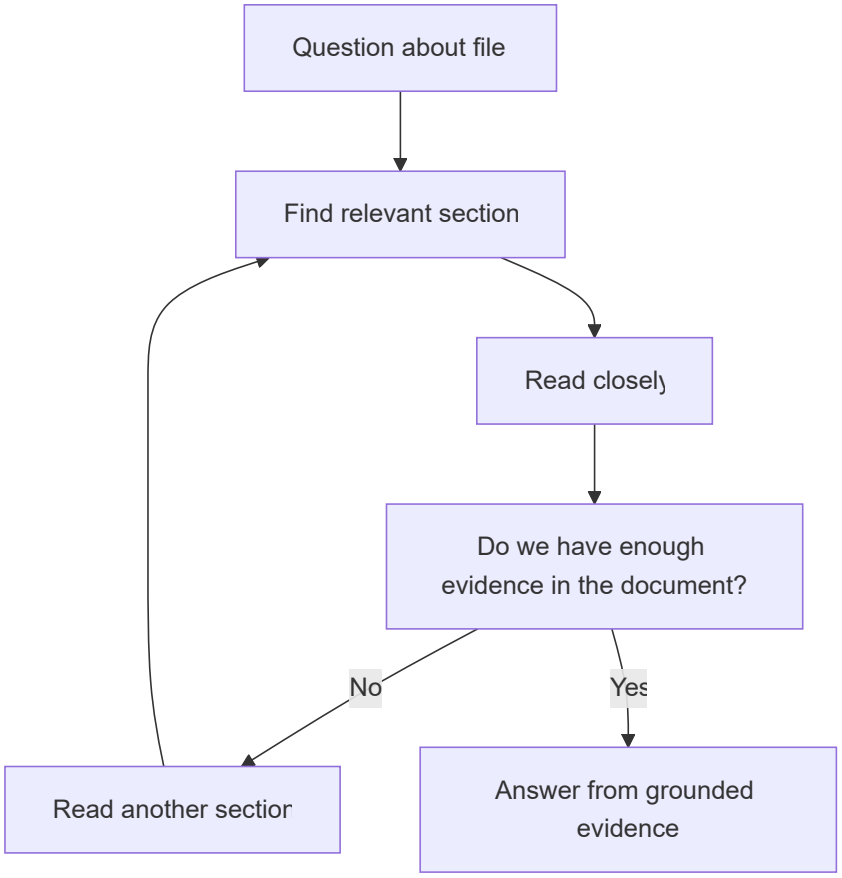
F. Gap Manager

文書不足や未記載を扱う部分。

全体構造


User Reques
Document Targeting
Document Reading
Relevance Mapping
Grounded Extraction
Grounded Reasoning
Citation / Trace Binding
Gap Handling
Grounded Output

文書読解ループ



典型例

入力	File-Grounded Mode の動き
このPDFを要約して	PDFを読み主要論点を抽出する
この契約書の更新条項は	該当条項を特定して抜き出す
この2つの資料を比べて	両方の関係箇所を比較する
この議事録からToDoを出して	アクション記述を抽出して整理する
この資料にその主張はあるか	根拠箇所の有無を確認する

よくある失敗

1. 一般知識で埋める

文書にない内容を、もっともらしく補ってしまう。

2. 根拠対象を誤る

別ファイルや別セクションの内容を混ぜてしまう。

3. 文書全体を平板に扱う

見出し構造や重要箇所の重みづけがない。

4. 不在情報を誤読する

書いていないことを、あるいはその逆に解釈する。

5. 根拠対応が弱い

どの結論がどの箇所から来たのか追えない。

設計原則

- 文書を主根拠として扱う
- 対象資料を最初に明確にする
- 質問に関係する箇所を重点読解する
- 事実・推定・不在を分ける
- 資料に忠実な推論を行う
- 根拠位置を追跡可能にする
- 不足時は不足として明示する

位置づけ

File-Grounded Mode は、
特定文書に根拠を固定し、その範囲内で読解・整理・応答を行う文書拘束型モードである。

これが強いと、

- 資料ベースの精度が上がり
- 引用可能性が増し
- 実務文書への対応力が高まる

一方で、文書拘束を忘れると一般論に逃げやすい。
したがってこのモードは、**文書の中に答えを探し、文書の外へ出す時は明示的に出るための読解統制モード**である。

関連ノート

- 02_zettelkasten/00_system/Mode Selection
- 02_zettelkasten/00_system/Direct Answer Mode
- Search-Augmented Mode
- Tool Orchestration
- 02_zettelkasten/00_system/Constraint Monitor
- LLM Output Layer
- LLM Output Layer]]
- LLM Output Layer

Comparative Reasoning Mode

Comparative Reasoning Mode

Comparative Reasoning Mode は、複数の対象・案・解釈・選択肢を、**同一の比較軸の上に並べ、差分・優劣・適合条件を明らかにする運転モード**である。
このモードの本質は、単に「違いを並べる」ことではなく、**何を基準に比べるべきかを定め、その基準に照らして判断可能な形へ整理すること**にある。

要点

- 比較では、対象そのもの以上に比較軸の設計が重要である
- 差分列挙だけでなく、どの条件でどちらが有利かを示す
- 比較対象が複数でも、基準はできるだけ共通化する
- 結果は一覧化だけでなく、判断支援へつなげる
- 比較モードは、分析・意思決定・提案設計の中間に位置する

なぜ必要か

ユーザーの依頼には、次のような比較要求が多い。

- AとBの違いを知りたい
- どちらがよいか判断したい
- 選択肢を並べた上で推奨が欲しい
- 類似概念の境界を知りたい
- 複数案の長所短所を整理したい

このとき、単なる特徴列挙では足りない。
必要なのは、

- どの観点で比べるのか
- どこが本質的差分なのか
- 何を重視すると結論が変わるのか

を明らかにすることである。

そのため Comparative Reasoning Mode は、**比較可能性を設計し、判断可能性へ変換する専用モード**として必要になる。

適用場面

1. 選択肢比較

例:

- この2案のどちらがよいか
- A案とB案を比較して
- 候補を並べて推奨して

2. 概念比較

例:

- structureとmechanismの違い
- hubとindexの違い
- policyとruleの違い

3. 文書比較

例:

- 2つの提案書の差分
- バージョン比較
- 条項差分の整理

4. 解釈比較

例:

- この読み方とあの読み方の違い
- どの見解がより妥当か
- 論点ごとの評価をして

適用してはいけない場面

- 単純な定義説明だけで足りる場合
- 比較対象が一つしかない場合
- 比較より因果分析が本体の場合
- ユーザーが明確な完成物だけを求めている場合

この場合は Direct Answer Mode や Stepwise Reasoning Mode の方が適切である。

中核機能

1. Comparison Target Identification

何と何を比較するのかを特定する。

対象は、

- 概念
- 案
- 製品
- 文書
- 主張
- 時点
- 戦略
- 手法

など多様である。
この段階では、比較単位を揃えることが重要である。

---

2. Axis Construction

どの観点で比べるかを設計する。

代表軸:

- 目的適合性
- コスト
- 速度
- 精度
- 柔軟性
- 安全性
- 拡張性
- 理解しやすさ
- 実装容易性

比較軸が悪いと、比較は浅くなる。
このモードでは、**比較軸設計が中心作業**である。

---

3. Normalization

異なる対象を、比較可能な形へ揃える。

例:

- 用語レベルを揃える
- 比較単位を揃える
- 時点を揃える
- 評価基準を揃える
- 不均等な情報量を補正する

---

4. Difference Extraction

比較軸ごとに差分を抽出する。

ここで重要なのは、

- 表面的差分
- 本質的差分
- 条件付き差分

を分けることである。

---

5. Trade-off Analysis

各対象の強みと弱みの交換関係を整理する。

例:

- 精度は高いが遅い
- 柔軟だが管理コストが高い
- 簡単だが拡張性が低い

単なる優劣ではなく、**何を取って何を捨てるか**を明示する。

---

6. Recommendation Framing

比較結果から、どの条件でどれを推奨するかを組み立てる。

形式:

- 一般推奨

- 条件付き推奨
- 用途別推奨
- 優先順位別推奨

---

7. Comparative Output Structuring

比較結果を、表・箇条書き・整理文として出力する。

このモードでは、

- 比較軸
- 差分
- 結論
- 推奨条件

が一目で見える構造が望ましい。

比較の基本構造

比較はしばしば次の形を取る。

1. 比較対象を確定する
2. 比較軸を決める
3. 軸ごとの差分を出す
4. トレードオフを整理する
5. 条件付きで結論を出す

下位構造

A. Target Pairer

比較対象を定める部分。

B. Axis Builder

比較軸を設計する部分。

C. Difference Mapper

軸ごとの差分を整理する部分。

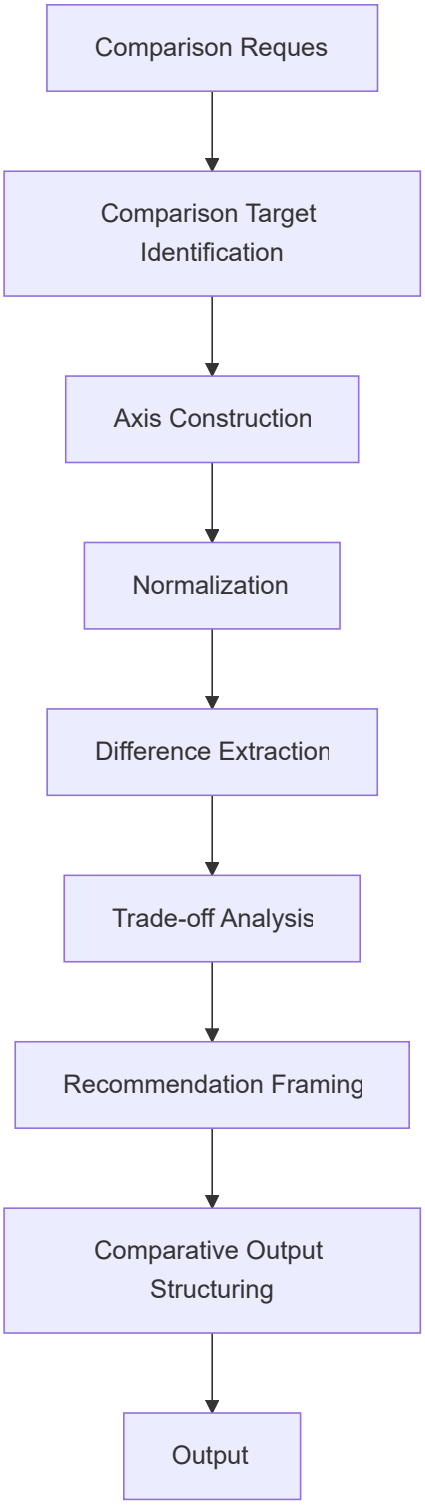
D. Trade-off Analyzer

得失の交換関係を分析する部分。

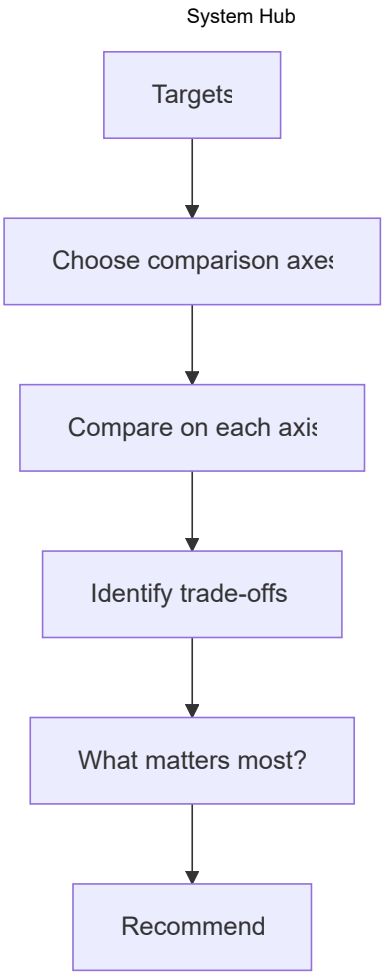
E. Recommendation Binder

比較結果を推奨へ結びつける部分。

全体構造


Comparison Reques
Comparison Target Identification
Axis Construction
Normalization
Difference Extraction
Trade-off Analysis
Recommendation Framing
Comparative Output Structuring
Output

比較判断ループ



典型例

入力	Comparative Reasoning Mode の動き
AとBの違いは	比較軸を立てて差分を整理する
どちらがよいですか	条件付き推奨まで出す
structureとmechanismを比較して	定義・役割・使い方で比べる
2案を評価して	評価軸を設定して採点・整理する
この2資料の差分は	同項目比較を行う

よくある失敗

1. 比較軸なしで並べる

特徴列挙になってしまい、比較にならない。

2. 表面的差分だけ見る

本質的な違いが見えない。

3. 優劣を一律に断定する

条件によって結論が変わるのに単純化しすぎる。

4. 情報量の偏りを放置する

片方だけ詳しく、比較が不公平になる。

5. 結論がない

比較表だけ出して、判断支援に至らない。

設計原則

- まず比較軸を立てる
- 比較単位を揃える
- 差分と共通点の両方を見る
- トレードオフを明示する
- 条件付き推奨を許容する
- 比較結果を判断へつなげる

位置づけ

Comparative Reasoning Mode は、
複数対象を比較可能な形へ整え、差分を判断材料へ変換する比較駆動モードである。

これが強いと、

- 違いが明確になり
- 選択理由が見え
- 推奨の根拠も整理される

したがってこのモードは、単なる一覧化ではなく、
比較を通じて意思決定可能性を高める評価整理モードである。

関連ノート

- 02_zettelkasten/00_system/Mode Selection
- Stepwise Reasoning Mode
- Artifact Generation Mode
- Direct Answer Mode
- Termination Control
- LLM Output Layer

Goal Framing

Goal Framing

Goal Framing は、解釈された意図を、**実行可能で評価可能な処理目標へ定式化する構造**である。
これは「何を言われたか」を「何を達成すべきか」へ変換する工程であり、LLM の処理全体に方向性・終了条件・評価基準を与える。

要点

- ユーザーの依頼は、そのままでは処理目標として曖昧なことが多い
- Goal Framing は、発話を達成目標・成功条件・制約条件へ変換する
- タスク分類は「何の仕事か」を決めるが、Goal Framing は「その仕事で何を成功とみなすか」を決める
- 良いフレーミングは、深掘りしすぎや浅すぎを防ぐ
- 目標が明確になると、ツール選択・停止判断・出力形式も安定する

なぜ必要か

たとえば「これについて教えてください」という依頼は、そのままでは曖昧である。
本当に必要なのは、

- 概要説明なのか
- 比較付き説明なのか
- 初学者向け整理なのか
- 実務で使うための整理なのか
- 判断材料の提示なのか

を明確にすることである。

また「作ってください」という依頼も、

- 叩き台が欲しいのか
- 完成版が欲しいのか
- 貼り付け可能な形が欲しいのか
- 理由説明込みが欲しいのか

で達成条件が違う。

Goal Framing は、このような曖昧さを、**処理可能な目標形式へ落とし込む中枢**である。

中核機能

1. Objective Extraction

発話や文脈から、主たる達成目標を抽出する。

例:

- 情報理解
- 妥当性評価
- 比較判断
- 成果物生成
- 実行支援
- 誤り修正
- 構造整理

この段階では、依頼の中心目的を一つまたは少数に絞る。

---

2. Success Criteria Definition

何をもって成功とみなすかを定義する。

例:

- 説明タスク
  → 概念が正確でわかりやすいこと
- 比較タスク
  → 比較軸・差分・推奨が明示されていること
- ノート生成
  → 指定形式でそのまま貼れること
- 調査タスク
  → 十分な根拠と引用があること

これにより、Termination Control が働きやすくなる。

---

3. Scope Bounding

目標の対象範囲を定める。

境界対象:

- 扱う論点の幅
- 時間範囲
- 深さ
- 対象読者
- 出力長
- 根拠範囲
- 作業単位

これを怠ると、関係のない補足が肥大化しやすい。

---

4. Constraint Incorporation

達成目標の中に制約を組み込む。

例:

- コードブロックで出す
- Mermaid を閉じ忘れない
- Web確認が必要
- 引用必須
- 余計な説明を省く
- 日本語で出す
- 直前の形式を踏襲する

Goal Framing は、目標と制約を別物としてではなく、**制約込みの目標**として扱う。

---

5. Output Target Specification

最終的にどのような成果物を作るのかを定める。

例:

- 回答文
- 比較表
- ノート本文
- JSON
- コード

- スケジュール案
- メール草案
- 分析レポート

これは出力形式の指定であると同時に、処理中の判断基準でもある。

---

6. Priority Framing

主目標と副目標の優先順位を決める。

例:

- 主目標: ノート本文完成
- 副目標: 理由説明
- 副目標: 関連ノート列挙

優先順位が明確であれば、限られた長さや時間でも主目的を守れる。

---

7. Evaluability Design

目標が、後で達成判定できる形になっているかを確認する。

良い目標は、

- 完了したか判定できる
- 不足部分がわかる
- 停止条件が置ける
- 出力品質を評価できる

という性質を持つ。

目標フレームの基本構成

Goal Framing は、しばしば次の要素で構成される。

- 何を達成するか
- 何を含めるか
- 何を含めないか
- 何形式で出すか
- どの深さで出すか

- 何を成功とみなすか
- どの制約を守るか

下位構造

A. Objective Builder

主目的を抽出・定式化する部分。

B. Success Criterion Setter

成功条件を置く部分。

C. Scope Definer

対象範囲と深さを決める部分。

D. Constraint Integrator

制約を目標定義へ組み込む部分。

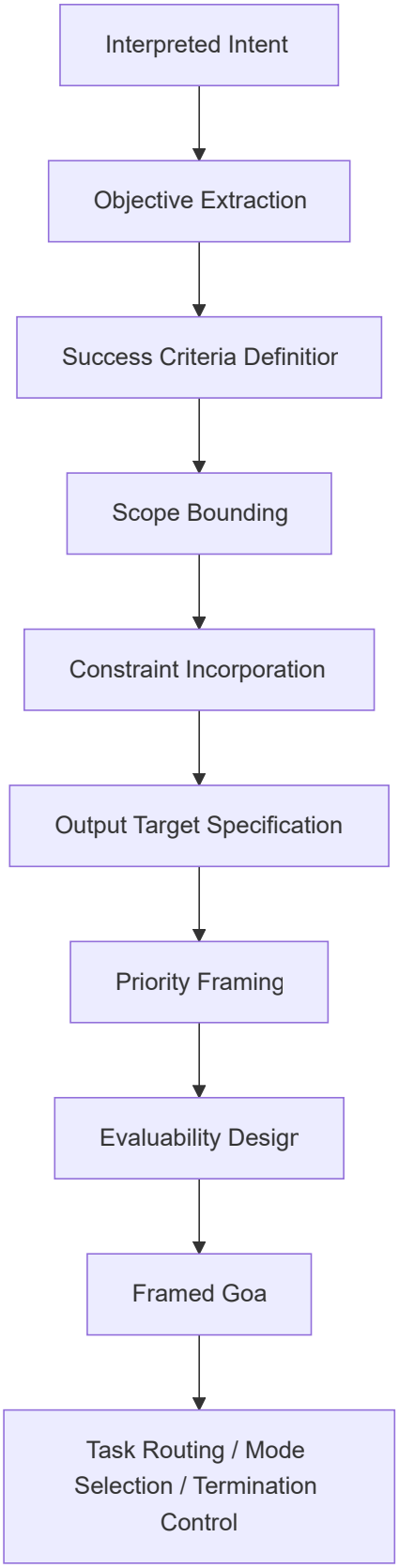
E. Output Targeter

最終成果物像を定義する部分。

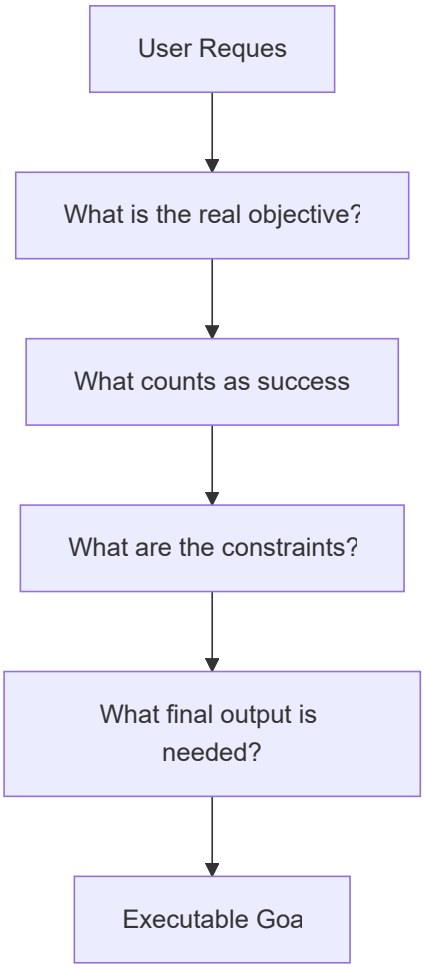
F. Priority Setter

主目標と副目標の優先順位を決める部分。

全体構造


Interpreted Intent
Objective Extraction
Success Criteria Definitior
Scope Bounding
Constraint Incorporation
Output Target Specification
Priority Framing
Evaluability Desigr
Framed Goa
Task Routing / Mode Selection / Termination Control

目標定式化の流れ



典型例

入力	Goal Framing の結果
これを説明してください	正確で理解しやすい説明を作る
比較してください	比較軸・差分・結論を示す
ノートを作ってください	貼り付け可能なノート本文を完成させる
続けてください	直前フォーマットのまま未完部分を継続完成させる
最新情報を教えてください	外部確認済みの最新情報を根拠付きで要約する
この形式で出してください	指定形式準拠を成功条件に含める

よくある失敗

1. 目標が発話の言い換えで終わる

実行可能な成功条件まで落ちていない。

2. 範囲が広すぎる

何でも含めようとして焦点がぼける。

3. 制約を後付けにする

最後に形式だけ合わせようとして中身がずれる。

4. 副目標が主目標を圧迫する

補足説明が本体より長くなる。

5. 達成判定できない

どこで終えてよいかわからなくなる。

設計原則

- 発話を達成目標に翻訳する
- 成功条件を必ず置く
- 範囲を先に切る
- 制約を目標定義に組み込む
- 成果物像を明確にする
- 主目標と副目標を分ける
- 停止判定できる形にする

位置づけ

Goal Framing は、
解釈された意図に、実行可能性と評価可能性を与える目標定式化構造である。

これが弱いと、

- 何となく進み

- 過不足のある応答になり
- どこで止めるべきかも曖昧になり
- 成果物の完成条件もぶれる

したがってこの構造は、単なる目的理解ではなく、
LLM の処理全体に進行方向と完了条件を与える目標設計機構である。

関連ノート

- Intent Interpretation
- Task Routing
- 02_zettelkasten/00_system/Mode Selection
- Termination Control
- Constraint Monitor
- LLM Control Layer
- LLM Control Layer
- LLM Control Layer]]
- LLM Control Layer]]

Intent Interpretation

Intent Interpretation

Intent Interpretation は、ユーザー入力の表層的な文言の背後にある **実際の依頼意図・達成したい目的・期待される応答形** を読み取る構造である。
ここで重要なのは、言葉を字面どおりに受け取ることではなく、**この発話によって何をしてほしいのか、どの形式で、どの粒度で、どこまで進めることを期待しているのか** を実行可能な形に変換することである。

この構造は system 層の入口に位置し、後続の Goal Framing、Task Routing、02_zettelkasten/00_system/Mode Selection に処理可能な意図表現を渡す。

要点

- 入力文の意味は、語句だけでなく文脈・省略・前提・会話履歴で決まる
- 同じ表現でも、実際の要求は複数ありうる
- 意図解釈は、タスク分類よりも前に行うべきである
- 出力の長さ、形式、完成度への期待も意図の一部である
- 継続依頼、修正依頼、比較依頼、成果物依頼を取り違えないことが重要である
- この段階がずれると、その後の routing・mode・output がすべてずれる

この構造が必要な理由

自然言語の依頼は、しばしば不完全である。

たとえば次のような入力は、そのままでは曖昧である。

- 作ってください
- 続けてください
- 見てください
- どう思いますか
- これでいいですか
- 貼れる形でください
- 前の形式でお願いします

これらは短いが、実際には次のような含意を持ちうる。

- 直前のフォーマットを維持する

- 既出部分を繰り返さない
- 完成物を出す
- 説明より本文を優先する
- 評価や改善案も含める
- 余計な前置きを省く

LLM が実務的に有用になるためには、入力をただ読むだけでなく、
会話行為として何を依頼されているか を把握する必要がある。
それを担うのが Intent Interpretation である。

役割

Intent Interpretation の役割は、次の5つにまとめられる。

1. 発話の字面を読む

何が明示的に書かれているかを把握する。

2. 暗黙要求を読む

書かれていないが高確率で含意されている要求を補う。

3. 会話文脈につなぐ

単独では曖昧な入力を、現在の作業状態に接続する。

4. 出力期待を読む

何を、どの形式で、どこまで完成させるべきかを推定する。

5. 構造化意図へ変換する

後続層が扱えるように、意図を整理して受け渡す。

中核機能

1. Surface Meaning Capture

まず、入力文の表面意味を把握する。

対象:

- 依頼動詞

- 対象物
- 条件句
- 否定
- 形式指定
- 範囲指定
- 継続指示
- 修正指示

例:

- 「コードブロックで出してください」
- 「貼り付ける部分だけ」
- 「続けてください」
- 「今の粒度で」

この段階は出発点だが、これだけでは不十分である。

---

2. Pragmatic Intent Inference

表面には書かれていないが、会話上ほぼ当然に期待されている要求を推定する。

例:

- 「見てください」
  - 読むだけではなく、要点把握・評価・反応を期待している可能性が高い
- 「作ってください」
  - 解説ではなく、使える完成物を期待している可能性が高い
- 「どう思いますか」
  - 単なる感想ではなく、論理評価や妥当性判定を求めていることがある
- 「続けてください」
  - 直前の形式・粒度・トーン・範囲を踏襲することを含意する場合が多い

この機能は、**会話の行為目的** を読む機能である。

---

3. Context Binding

現在の発話を、会話の流れ・既出成果物・進行中タスクに接続する。

参照対象:

- 直前に生成したノート

- 直前に確定したフォーマット
- 「まだ未作成のもの」の一覧
- 今どのHubやLayerを作っているか
- ユーザーがすでに否定した方向性
- 会話内で定着したルール

これにより、単独では曖昧な発話も具体化できる。

例:

- 「続けてください」
  - 新規作成ではなく、今の系列の続き
- 「それも追加してください」
  - 直前の一覧に対する追加
- 「こういうのではないです」
  - 方向修正の指示

---

4. Goal Signal Extraction

発話の背後にある最終目的を抽出する。

例:

- 「一覧にしてください」
  - 俯瞰可能性の向上
- 「コードブロックにしてください」
  - そのまま貼れる状態にする
- 「内容を濃くしてください」
  - 密度と具体性を上げる
- 「一つずつ作ってください」
  - 分割提示と逐次確定を優先する

つまり Intent Interpretation は、
表面上の要求を、処理目的の信号へ変換する。

---

5. Output Expectation Modeling

ユーザーがどのような出力を期待しているかを推定する。

対象:

- 長さ
- 粒度
- 形式
- 完成度
- 丁寧さ
- 実用性
- コピペ可能性
- 説明量と本体量の比率

たとえば「貼り付ける部分だけ」とあれば、

- 前置き最小
- 本体優先
- 余計な説明を削る

という期待を意味する。

---

6. Ambiguity Resolution

複数の解釈がありうる場合、最も整合的なものを選ぶ。

判断基準:

- 会話文脈との整合
- 直前の出力形式との一致
- ユーザーの進行中タスクとの一致
- 既出指示との整合
- 最小驚愕性
- 成果物としての有用性

必要なら曖昧さを一部保持しつつ、最も自然な解釈で進む。

---

7. Intent Packaging

後続層に渡せるように、解釈結果を構造化する。

典型項目:

- 主要求
- 副要求
- 明示条件

- 暗黙期待
- 出力形式
- 継続条件
- 修正点
- 望ましい完成度
- 会話文脈上の依存関係

主要な意図類型

A. Explicit Request

明示的に書かれた要求。

例:

- 「Tool Orchestration を作ってください」
- 「コードブロックで出してください」

B. Implicit Request

書かれていないが、会話上強く含意される要求。

例:

- 直前形式の踏襲
- 既出内容の再出力回避
- 貼り付け可能性の確保

C. Continuation Intent

進行中作業の続きとしての意図。

例:

- 続けてください
- 順に作ってください
- 次を作ってください

D. Correction Intent

誤り修正・方向修正の意図。

例:

- こうではないです
- 閉じ忘れがあります
- 以前のものはもういいです

E. Evaluation Intent

感想よりも判定・検証・評価を求める意図。

例:

- これでいいですか
- 妥当ですか
- どう思いますか

F. Artifact Intent

説明ではなく、完成物を求める意図。

例:

- ノートを作ってください
- 完成版をください
- 貼り付ける部分だけ

下位構造

A. Surface Parser

文面上の依頼内容を抽出する部分。

B. Pragmatic Interpreter

発話行為の実際の目的を推定する部分。

C. Context Linker

発話を会話履歴・作業状態へ接続する部分。

D. Expectation Estimator

出力粒度・形式・完成度への期待を読む部分。

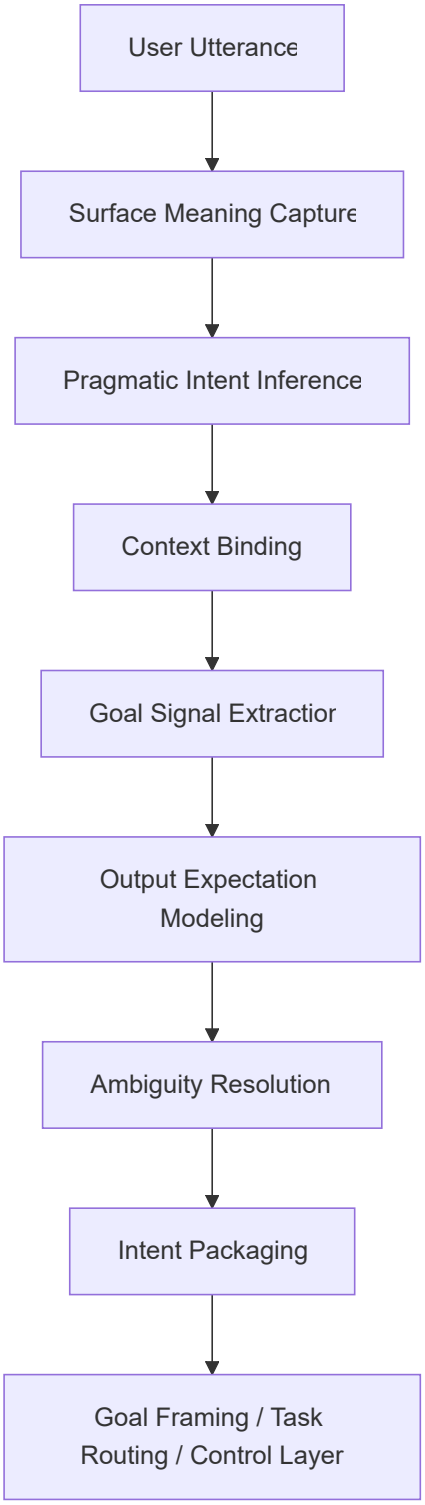
E. Ambiguity Resolver

複数解釈の中から最も妥当なものを選ぶ部分。

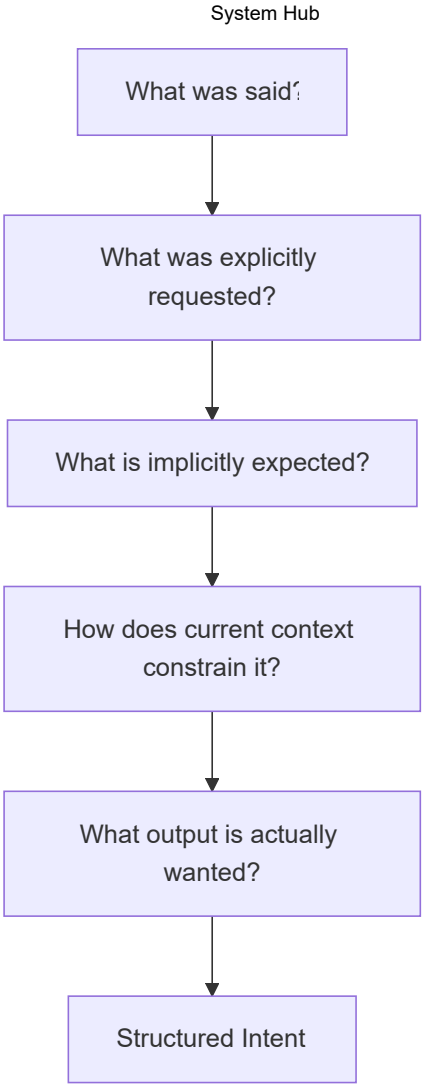
F. Intent Packager

後続層へ渡せる構造化意図に変換する部分。

全体構造


User Utterance
Surface Meaning Capture
Pragmatic Intent Inference
Context Binding
Goal Signal Extractior
Output Expectation Modeling
Ambiguity Resolution
Intent Packaging
Goal Framing / Task Routing / Control Layer

意図解釈の処理フロー



入出力イメージ

入力文	解釈される意図
続けてください	直前の系列を同じ形式で継続
作ってください	説明ではなく完成物を生成
見てください	読解 + 必要に応じて評価や整理
これでいいですか	妥当性判定 + 改善点提示
貼り付ける部分だけ	余計な説明を省き、本体のみ提示
順に作ってください	一括ではなく逐次生成
以前のものはもういいです	再出力を避け、未作成分のみ対象にする

他構造との関係

Goal Framing との関係

Intent Interpretation は「何を求めているか」を読む。
Goal Framing は「それをどのような達成目標として定義するか」を決める。

Task Routing との関係

Intent Interpretation は意図を構造化する。
Task Routing はそれをどの処理ラインへ送るか決める。

02_zettelkasten/00_system/Mode Selection との関係

Intent Interpretation は要求の性質を読む。
Mode Selection はそれに適した運転形態を選ぶ。

Constraint Monitor との関係

Intent Interpretation は依頼内容を読む。
Constraint Monitor はそれが許容範囲内か監視する。

LLM Output Layer との関係

Intent Interpretation は期待される出力像を先読みする。
Output Layer は実際の最終形式へ整える。

よくある失敗

1. 字面どおりにしか読まない

暗黙要求を拾えず、期待外れになる。

2. 文脈を切って読む

「続けてください」を新規依頼として扱ってしまう。

3. 出力期待を見落とす

内容は合っていても、形式や密度がずれる。

4. 補助要求を主要求と取り違える

説明補足ばかり出して、本体が弱くなる。

5. 修正意図を見逃す

同じ失敗形式を繰り返す。

6. 暗黙要求を盛りすぎる

ユーザーが求めていない方向へ広げすぎる。

設計原則

- 字面と含意を分けて読む
- 文脈を必ず接続する
- 依頼を目的レベルへ引き上げる
- 出力期待も意図の一部として扱う
- 継続・修正・完成物依頼を特に重視する
- 曖昧さは最も整合的な形で処理する
- 後続層が扱える構造化意図へ変換する

位置づけ

Intent Interpretation は、
自然言語入力を、実行可能な依頼意図へ翻訳する system 層の入口構造 である。

これが弱いと、

- 言われたことをやっているのに満足されない
- 形式や粒度がずれる
- 継続作業で文脈が切れる
- 修正要求に追従できない
- routing や mode も誤る

したがってこの構造は、単なる意味解析ではなく、
会話を運転可能な処理要求へ変換する最初の統制機構として扱うべきである。

関連ノート

- LLM System Hub
- LLM Control Layer
- Goal Framing
- Task Routing
- 02_zettelkasten/00_system/Mode Selection
- Constraint Monitor
- LLM Output Layer 02_zettelkasten/00_system/Task Routing
- 02_zettelkasten/00_system/Mode Selection
- Constraint Monitor
- LLM Output Layerayer]]

Mode Selection

Mode Selection

Mode Selection は、解釈された意図とタスクに基づいて、**LLM をどのような認知・実行モードで動かすかを決める構造**である。
ここでいうモードとは、単なる出力スタイルではなく、**情報取得の要否、推論の深さ、外部ツール依存度、比較や生成の優先度、停止条件の傾向**まで含む処理運転形態である。

要点

- 同じ入力でも、選ぶモードによって処理の進み方は大きく変わる
- Mode Selection は、Task Routing で決まった処理ラインをさらに具体的な運転状態へ落とし込む
- モードは「何をするか」ではなく「どう動くか」を決める
- 良いモード選択は、過不足ない深さ・速度・正確性・形式適合性を生む
- モードを誤ると、正しいルートでも応答品質が崩れる

なぜ必要か

たとえば「この件を教えてください」という依頼でも、

- 安定知識なら直答でよい
- 最新情報なら検索前提で動くべき
- ファイルが主根拠なら file-grounded にすべき
- 比較なら比較軸生成を先に走らせるべき
- 成果物生成なら構造化テンプレートを優先すべき

この差は、単なるタスク分類では足りない。
同じ「説明」でも、

- 短く答える説明
- 出典付き説明
- 比較を含む説明
- 生成物の前置きとしての説明

では内部運転が違う。

そのため、処理ラインの中での**実際の走り方**を決める Mode Selection が必要となる。

中核機能

1. Mode Requirement Detection

今回のタスクが、どのような処理モードを必要とするかを判定する。

判断材料:

- 最新性の要否
- 根拠の強さ要求
- ファイル依存の有無
- 成果物要求
- 比較や分解の必要性
- 実行操作の有無
- 安全リスク
- 簡潔さの要求

---

2. Mode Choice

候補モードの中から、最も適合するものを選ぶ。

代表モード:

- Direct Answer Mode
- Search-Augmented Mode
- File-Grounded Mode
- Comparative Reasoning Mode
- Stepwise Reasoning Mode
- Artifact Generation Mode
- Tool Execution Mode
- Safety Refusal Mode
- Compression Mode
- Validation Mode

モードは単一とは限らず、主モードと補助モードの組み合わせもありうる。

---

3. Depth Setting

選んだモードに応じて、どの程度まで深掘りするかを設定する。

たとえば、

- 直答モードなら浅く速く
- 比較モードなら比較軸を十分立てる
- 検証モードなら根拠の対応づけを厚くする
- 生成モードなら形式完成を優先する

---

4. Resource Profile Binding

各モードで、どの資源を優先利用するかを定める。

例:

- Search-Augmented → Web優先
- File-Grounded → ファイル優先
- Tool Execution → 実行系ツール優先
- Direct Answer → 内部知識優先
- Validation → 根拠抽出優先

---

5. Reasoning Pattern Activation

モードごとに、適した推論パターンを起動する。

例:

- Comparative → 比較軸・差分・推奨
- Stepwise → 分解・段階処理
- Validation → 主張と根拠の対応づけ
- Compression → 主論点抽出と重複除去
- Artifact Generation → テンプレート充填と整形

---

6. Output Bias Setting

出力側で何を重視するかを先に定める。

例:

- Direct Answer → 結論先出し
- Validation → 根拠明示

- Artifact Generation → そのまま使える完成形
- Compression → 最小限で要点維持
- Safety Refusal → 境界説明と代替案

---

7. Mode Switching

処理途中で必要に応じてモードを切り替える。

例:

- 直答で始めたが最新性が必要と判明 → Search-Augmented へ移行
- 調査中に比較依頼を含むと判明 → Comparative を追加
- 外部実行が必要 → Tool Execution へ移行
- 危険領域へ接近 → Safety Refusal に切替

主要モード

A. Direct Answer Mode

内部知識と軽い推論で即答するモード。
高速性と簡潔性を重視する。

B. Search-Augmented Mode

外部検索を前提とするモード。
最新性・確認性を重視する。

C. File-Grounded Mode

添付ファイルや接続ドキュメントを主根拠とするモード。
根拠限定性が高い。

D. Comparative Reasoning Mode

比較軸を立てて差分と推奨を導くモード。

E. Stepwise Reasoning Mode

複雑課題を段階的に分解して処理するモード。

F. Artifact Generation Mode

ノート、文書、コード、表など完成物を作るモード。

G. Tool Execution Mode

メール送信、予定作成、分析実行など外部行動を伴うモード。

H. Safety Refusal Mode

安全境界上の案件で、拒否または安全代替を行うモード。

I. Validation Mode

真偽・妥当性・根拠対応を検証するモード。

J. Compression Mode

長い入力や複雑結果を短く圧縮するモード。

下位構造

A. Mode Detector

必要モードを検出する部分。

B. Mode Chooser

候補から最適モードを選ぶ部分。

C. Profile Binder

深さ・資源・出力傾向を結びつける部分。

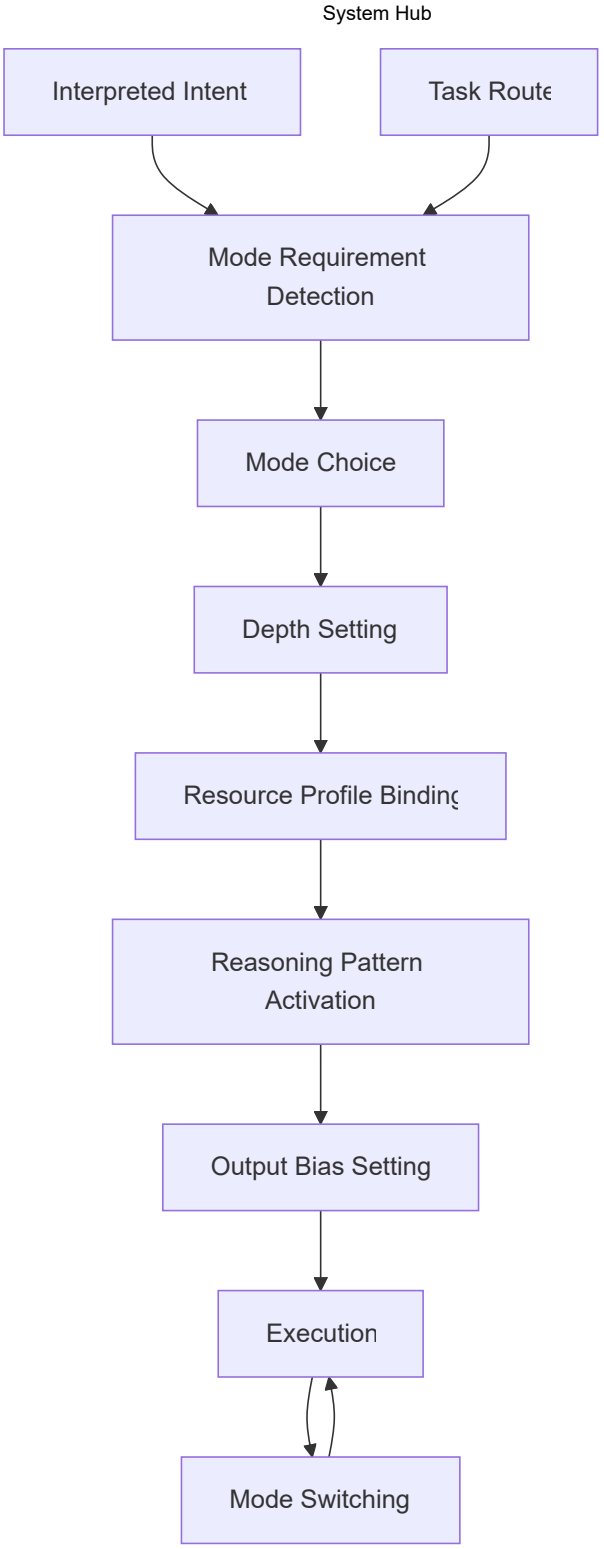
D. Switch Controller

途中でのモード切替を管理する部分。

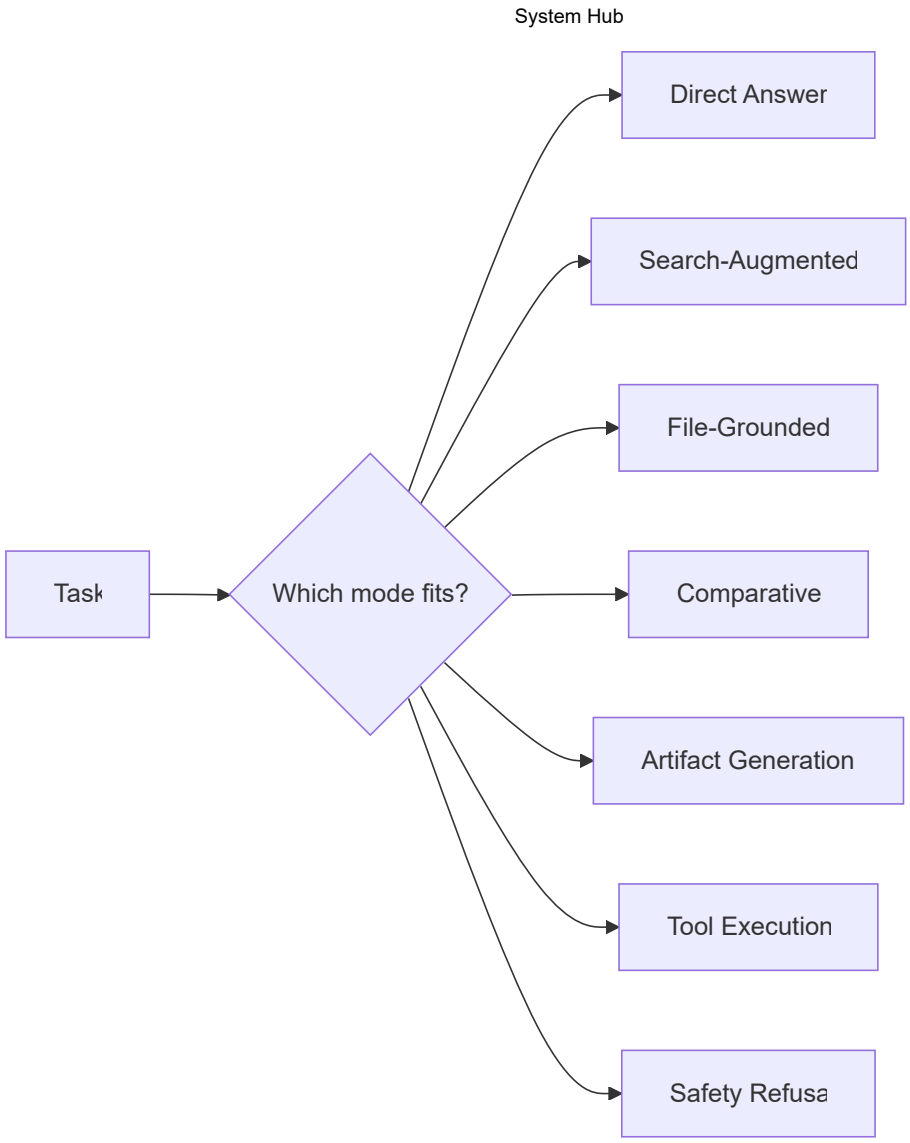
E. Mode State Holder

現在の処理モード状態を保持する部分。

全体構造


System Hub
Interpreted Intent
Task Route
Mode Requirement Detection
Mode Choice
Depth Setting
Resource Profile Binding
Reasoning Pattern Activation
Output Bias Setting
Execution
Mode Switching

モード分岐図



典型例

状況	選ばれやすいモード
一般知識の説明	Direct Answer Mode
最新ニュースの確認	Search-Augmented Mode
PDF要約	File-Grounded Mode + Compression Mode
選択肢比較	Comparative Reasoning Mode
複雑問題の切り分け	Stepwise Reasoning Mode
ノート作成	Artifact Generation Mode

状況	選ばれやすいモード
メール送信	Tool Execution Mode
危険依頼	Safety Refusal Mode

よくある失敗

1. モードを固定しすぎる

何でも同じ調子で処理してしまう。

2. 直答モードのまま進む

検索やファイル根拠が必要なのに切り替わらない。

3. 生成タスクなのに説明モードで動く

完成物ではなく解説ばかり出る。

4. 比較なのに比較モードに入らない

差分や比較軸が立たない。

5. 途中切替がない

新情報で条件が変わっても同じ走り方を続ける。

設計原則

- タスクだけでなく運転形態を選ぶ
- 主モードと補助モードを区別する
- 深さ設定をモードに結びつける
- 資源利用方針もモードに埋め込む
- 処理中の切替を許容する
- 出力傾向まで先に決めておく

位置づけ

Mode Selection は、
処理ラインに対して具体的な運転状態を与える制御構造である。

これが弱いと、

- 直答すべきところで過剰探索し
- 検索すべきところで記憶に頼り
- 生成すべきところで説明に終始し
- 比較すべきところで論点が散る

したがってこの構造は、単なる設定変更ではなく、
LLM の処理品質を場面ごとに最適化する運転モード制御装置である。

関連ノート

- Task Routing
- Goal Framing
- Intent Interpretation
- Tool Orchestration
- Termination Control
- LLM Control Layer Termination Control
- LLM Control Layer
- LLM Control Layer]]
- LLM Control Layer

Search-Augmented Mode

Search-Augmented Mode

Search-Augmented Mode は、ユーザーの依頼に答えるために、**外部情報源から必要な情報を取得し、その結果を解釈・統合した上で応答する運転モード**である。
このモードの本質は、検索結果を貼ることではなく、**外部世界に接続して得た情報を、問いに適合する知識へ変換すること**にある。

要点

- 最新性・変動性・確認性が必要な場面で使う
- 外部検索は手段であり、最終目標は信頼できる回答生成である
- 検索結果の量より、関連性と解釈品質が重要である
- 検索後には必ず、比較・統合・評価の工程が必要になる
- 検索しない誤りより、検索しただけで終わる誤りも危険である

なぜ必要か

LLM の内部知識だけでは不十分な問いがある。たとえば、

- 最新ニュース
- 現在の価格
- 現行制度
- 直近の人事
- 最近のソフトウェア仕様
- 変化しうる統計
- 今起きている出来事

これらは、記憶に頼ると誤りやすい。
そのため外部情報への接続が必要となる。

しかし、検索してもそのままでは答えにならない。
複数情報源から、

- どれが関係あるか
- どれが信頼できるか
- 何が一致しているか
- 何が未確定か

を整理しなければならない。
それを担うのが Search-Augmented Mode である。

適用場面

1. 最新性が必要な質問

例:

- 今日のニュース
- 現在の役職者
- 直近の出来事
- 現在の仕様や制度

2. 高変動情報

例:

- 価格
- 在庫
- スケジュール
- スポーツ結果
- 天気
- 相場

3. 検証要求

例:

- 本当にそうか調べて
- 出典を示して
- 最新情報で確認して

4. ニッチ・曖昧・不確かな語

例:

- 用語が不明
- 表記揺れがある
- 最近出てきた概念
- 誤記の可能性がある語

適用してはいけない場面

Search-Augmented Mode を使わなくてよい、あるいは使いすぎると逆効果な場面もある。

- 一般的で安定した概念説明
- ユーザーが与えたテキストの要約
- 純粋な文章生成や書き換え
- 会話上の軽い相談
- 内部知識だけで十分な原理解説

この場合は Direct Answer Mode などの方が適する。

中核機能

1. Search Need Recognition

検索が必要かどうかを判定する。

判定基準:

- 情報が変わりうるか
- 精度要求が高いか
- 出典が必要か
- 内部知識で足りるか
- ユーザーが確認を明示しているか

---

2. Query Formulation

問いを、検索可能なクエリへ変換する。

よいクエリは、

- 対象が明確
- 無駄が少ない
- 比較や検証に必要な語を含む
- 必要なら時間条件や領域条件を持つ

この段階で質が低いと、後続結果も崩れる。

---

3. Source Acquisition

外部情報源から候補情報を取得する。

対象:

- 検索結果
- 公式情報
- 一次情報
- 補助的報道
- 構造化データ

ここでは量より、信頼度と関連度が重要である。

---

4. Source Evaluation

取得した情報源を評価する。

評価観点:

- 信頼性
- 更新日時
- 一次性
- 関連性
- 一貫性
- 詳細度

---

5. Information Extraction

必要な事実・数値・主張・日付・差分を抽出する。

この時点で、

- 何が答えに必要か
- どの情報が荷重の高い根拠か
- 何が補助情報か

を分ける。

---

6. Cross-Source Synthesis

複数情報源を付き合わせ、答えに必要な形へ統合する。

内容:

- 一致点の抽出
- 差異の把握
- 時系列整理
- 強い根拠の優先
- 不一致の扱い
- 推定と事実の区別

---

7. Citation-Aware Response Building

出典を対応づけた形で回答を構築する。

ここでは、

- どの主張に何が根拠か
- どこが未確定か
- 何が最新時点の情報か

を明確にする必要がある。

検索拡張の処理段階

A. Need to Search?

検索要否の判断。

B. What to Search?

検索対象と検索語の設計。

C. What Did We Find?

取得結果の整理。

D. What Matters?

答えに必要な部分の選別。

E. What Can We Safely Conclude?

結論可能な内容の確定。

下位構造

A. Search Trigger

検索必要性を検出する部分。

B. Query Builder

検索語を組み立てる部分。

C. Source Filter

取得結果を信頼性・関連性で絞る部分。

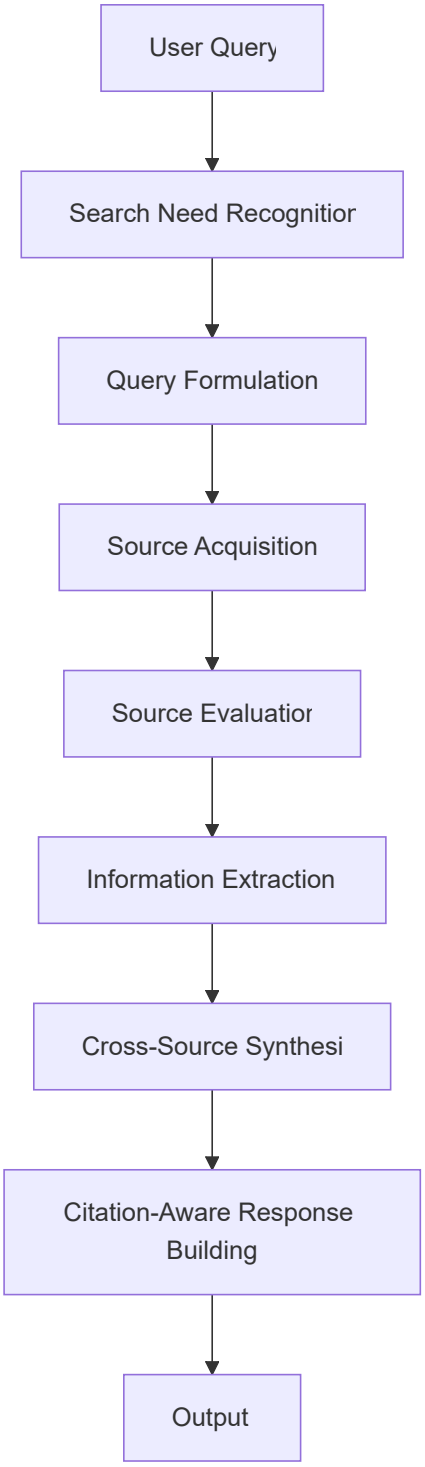
D. Evidence Synthesizer

複数情報源を統合する部分。

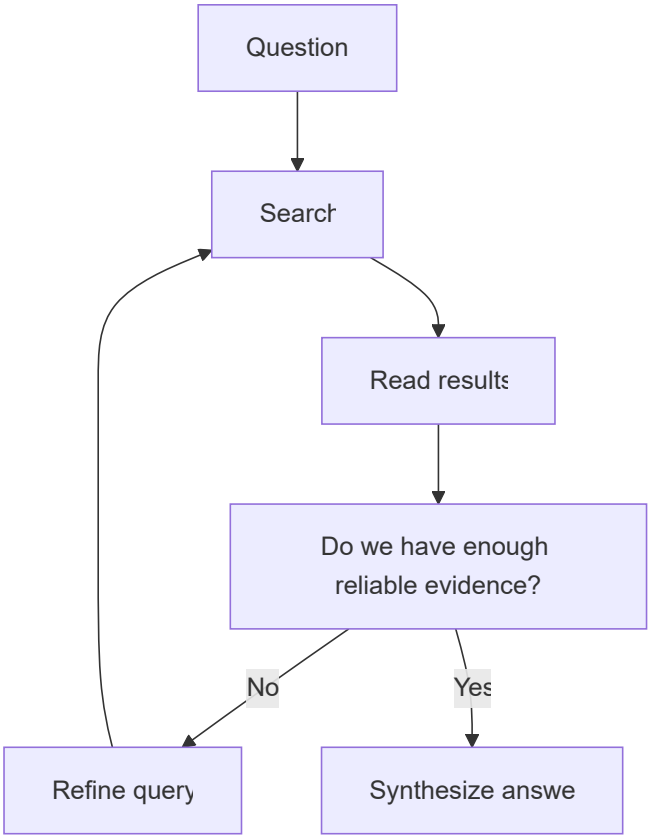
E. Citation Binder

主張と出典を対応づける部分。

全体構造


User Query
Search Need Recognition
Query Formulation
Source Acquisition
Source Evaluation
Information Extraction
Cross-Source Synthesis
Citation-Aware Response Building
Output

検索統合ループ



典型例

入力	Search-Augmented Mode の動き
最新のCEOは誰ですか	現在情報を検索して確認する
そのニュースは本当ですか	複数ソースで検証する
今日の天気を教えて	最新天気情報を取得する
この制度は今どうなっていますか	現行制度を確認する
その用語は何ですか	用語自体を検索して確認する

よくある失敗

1. 検索せずに答える

最新情報や不安定情報を記憶で済ませてしまう。

2. 検索しただけで終わる

結果を貼るだけで、統合や判断がない。

3. 情報源の質を見ない

低品質情報を重く扱ってしまう。

4. 古い情報を最新として扱う

時点の確認が甘い。

5. 引用対応が弱い

どの主張がどの情報源に支えられているか不明瞭になる。

設計原則

- 変動情報にはまず検索を当てる
- クエリ設計を丁寧に行う
- 一次情報や信頼源を優先する
- 検索結果は比較・統合して使う
- 日付と時点を明示する
- 出典対応を明確にする
- 十分な証拠が集まったら止める

位置づけ

Search-Augmented Mode は、
LLM の内部知識を外部世界の最新情報で補強し、検証可能な回答へ変えるモードである。

これが強いと、

- 変動情報に対応でき
- 確認性が上がり
- 引用付きの信頼可能な応答ができる

一方で、検索に依存しすぎると冗長になる。
したがってこのモードは、**必要な時にだけ起動し、外部情報を意味のある答えへ変換する確認駆動モード**である。

関連ノート

- Mode Selection
- Direct Answer Mode
- File-Grounded Mode
- Tool Orchestration
- Constraint Monitor
- LLM Output Layer02_zettelkasten/00_system/Tool Orchestration
- 02_zettelkasten/00_system/Constraint Monitor
- LLM Output Layer

Stepwise Reasoning Mode

Stepwise Reasoning Mode

Stepwise Reasoning Mode は、複雑な問いや作業を**一度に処理せず、段階・部品・論点に分解して順番に解く運転モード**である。
このモードの本質は、考える量を増やすことではなく、**問題を扱える単位へ切り分け、各段階で中間成果を確定しながら進むこと**にある。

要点

- 複雑問題は、分解しないと処理精度が落ちる
- 各段階には固有の目的と完了条件が必要である
- 前段の出力を後段の入力として再利用する
- 途中で方針修正や不足補完がしやすい
- 最終的には、段階の列を一つの結論へ束ねる

なぜ必要か

LLM への依頼には、次のような複雑課題がある。

- 論点が多い
- 手順が複数ある
- 前提整理が必要
- 比較と提案が混在している
- 調査・要約・生成が連なっている
- 原因分析や設計が絡む

このような依頼を一括で処理すると、

- 論点漏れ
- 順序混乱
- 根拠不足
- 結論の飛躍

が起きやすい。

そのため Stepwise Reasoning Mode は、**複雑依頼を工程化して安定的に処理するための多段推論モード**として必要になる。

適用場面

1. 複雑問題の分解

例:

- どう整理すればよいか
- 何から考えるべきか
- 多面的に分析して

2. 原因分析

例:

- なぜこうなったか
- 要因を分解して
- 真因を特定して

3. 設計・構成作業

例:

- システム構造を設計して
- 段階的にノートを組み立てて
- 完成版まで順に作って

4. 複合依頼

例:

- 調べて比較して提案して
- 要約して、その後表にして
- 読んで、評価して、改善案を出して

適用してはいけない場面

- 単純な概念説明
- 短い言い換え
- 即答で十分な質問
- 軽い雑談や一言助言

この場合は Direct Answer Mode の方が適切である。

中核機能

1. Problem Decomposition

問題を、処理可能な小単位へ分解する。

分解観点:

- 時系列
- 手順
- 論点
- 層
- 因果段階
- 成果物部品
- 意思決定段階

---

2. Step Definition

各段階で何を行うかを定義する。

よい段階定義は、

- 役割が明確
- 前後関係が明確
- 中間成果が明確
- 完了判定が可能

である。

---

3. Sequential Execution

順番に段階を実行する。

ここで重要なのは、

- 前段の結果を踏まえる
- 飛ばさない
- 必要なら戻る
- 各段階ごとに小さく確定する

ということである。

---

4. Intermediate State Preservation

各段階の結果を保持し、後続工程で再利用する。

内容:

- 論点一覧
- 仮説
- 比較軸
- 要件整理
- 中間結論
- 制約条件

---

5. Local Verification

各段階ごとに、小さな妥当性確認を行う。

確認内容:

- その段階の目的を満たしたか
- 次へ進む材料がそろったか
- 見落としがないか
- 前提が崩れていないか

---

6. Integration

各段階の成果を統合して、全体結論へまとめる。

ここでは、単なる足し合わせではなく、

- 整合性確認
- 重複除去
- 優先順位づけ
- 結論化

が必要となる。

---

7. Adaptive Re-entry

途中で不足や矛盾が出た場合、前段へ戻って補正する。

これにより、段階処理が機械的な一方向列に固定されず、柔軟性を持つ。

段階推論の基本形

1. 問題を分ける
2. 段階を定義する
3. 順番に処理する
4. 各段階を検証する
5. 最後に束ねる

下位構造

A. Decomposer

問題を分解する部分。

B. Step Planner

処理段階を設計する部分。

C. Intermediate Memory

中間成果を保持する部分。

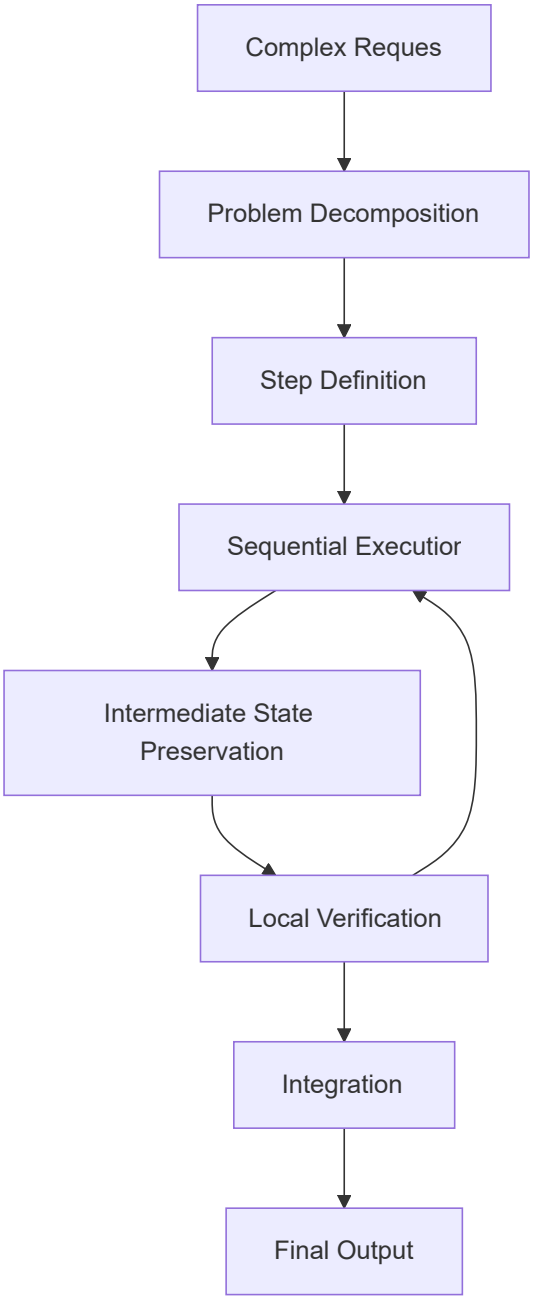
D. Local Checker

各段階の妥当性を確認する部分。

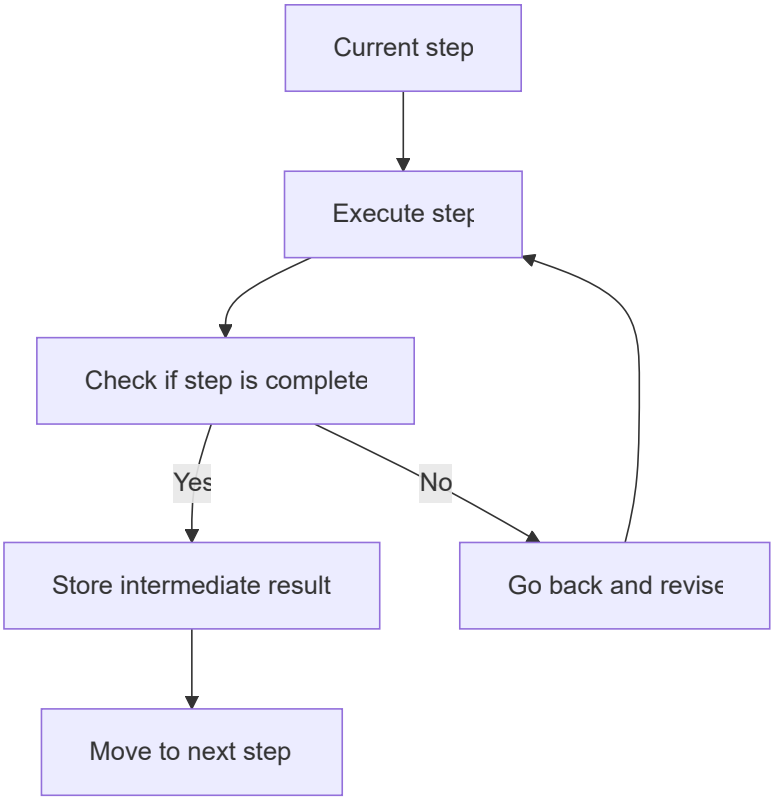
E. Integrator

段階成果を全体結論へ束ねる部分。

全体構造


Complex Reques
Problem Decomposition
Step Definition
Sequential Executior
Intermediate State Preservation
Local Verification
Integration
Final Output

段階処理ループ



典型例

入力	Stepwise Reasoning Mode の動き
この問題を整理してください	論点を分解して順に整理する
原因分析してください	要因を段階分解して真因に近づく
完成版を順に作って	部品ごとに作成して統合する
比較して提案して	比較段階と提案段階に分ける
調べてまとめて生成して	検索・整理・生成を分離する

よくある失敗

1. 分解しすぎる

細かすぎて前進が遅くなる。

2. 分解が粗すぎる

結局一括処理と変わらなくなる。

3. 中間成果を使わない

段階化したのに、毎回考え直してしまう。

4. 統合が弱い

段階ごとの結果はあるが、全体結論が弱い。

5. 戻りがない

途中矛盾に気づいても前段修正が行われない。

設計原則

- 複雑問題はまず分解する
- 各段階に役割と完了条件を置く
- 中間成果を残す
- 段階ごとに小さく検証する
- 必要に応じて戻る
- 最後に必ず統合する

位置づけ

Stepwise Reasoning Mode は、
複雑課題を段階列へ変換し、中間成果を積み上げて安定的に結論へ到達する多段処理モードである。

これが強いと、

- 論点漏れが減り
- 工程が明確になり
- 複合依頼にも対応しやすくなる

したがってこのモードは、単に丁寧に考えるモードではなく、
複雑性を工程へ変換する構造化推論モードである。

関連ノート

- Mode Selection
- Comparative Reasoning Mode
- Artifact Generation Mode
- Termination Control
- Goal Framing
- LLM Output Layer/00_system/Goal Framing]]
- LLM Output Layer

Task Routing

Task Routing

Task Routing は、入力された依頼や質問を、**適切な処理系・推論モード・ツールチェーン・出力モードへ振り分ける構造**である。
その核心は、表面上の文を読むことではなく、**この依頼は実際にはどの仕事なのか**を見抜き、最も合った処理ラインへ送ることにある。

要点

- 同じ自然言語でも、実際のタスク型は異なる
- Task Routing は「何を答えるか」より前に、「何として扱うか」を決める
- ルーティングを誤ると、以後の推論・検索・出力がすべてずれる
- 一つの入力に複数タスクが含まれる場合、分解して別ルートへ流す必要がある
- 良いルーティングは、過不足のない処理深度と適切な成果物形式を生む

なぜ必要か

自然言語の依頼は曖昧であり、同じ表現でも意図は大きく異なる。

例:

- 「これを見てください」
  - 要約なのか
  - 評価なのか
  - 誤り検出なのか
  - 書き直しなのか
- 「作ってください」
  - 説明文作成なのか
  - ノート生成なのか
  - コード作成なのか
  - スプレッドシート作成なのか
- 「どう思いますか」
  - 感想要求なのか
  - 論理評価なのか
  - 妥当性判定なのか
  - 改善提案なのか

この曖昧性を解消しないまま進むと、回答の方向がずれる。
Task Routing は、このずれを防ぐための最初の振り分け機構である。

中核機能

1. Task Type Identification

入力がどの種類のタスクかを特定する。

代表類型:

- 質問応答
- 説明
- 要約
- 分析
- 比較
- 評価
- 生成
- 編集
- 検索
- 実行
- 計画
- 変換
- 拒否判定付き依頼

この識別は、表面語だけでなく、要求成果物・必要根拠・操作対象を見て行う。

---

2. Intent Disambiguation

曖昧な表現の背後にある実意図を絞る。

見るべきもの:

- 依頼動詞
- 添付物の有無
- 指定形式
- ドメイン
- 直前会話文脈
- ユーザーの作業段階
- 出力の期待粒度

たとえば「続きを作ってください」は、

- 前ノートの続き
- 前回フォーマット踏襲
- 同一粒度維持
- 貼り付け可能形式

を含意する場合がある。

---

3. Task Decomposition

1つの依頼に複数タスクが混在している場合、それを分ける。

例:

- 「調べて比較して提案してください」
  - 検索
  - 比較
  - 推奨
- 「このPDFを要約して、その後表にしてください」
  - 読解
  - 要約
  - 構造化変換

Task Routing は、必要に応じて複合依頼をサブタスク列へ分割する。

---

4. Route Selection

識別したタスクを、どの処理ラインへ送るか決める。

代表ルート:

- Direct Answer Route
- Search Route
- File Grounded Route
- Tool Execution Route
- Artifact Generation Route
- Comparative Reasoning Route
- Safety Refusal Route

ここでの誤りは、その後の全工程に伝播する。

5. Mode Binding

ルートだけでなく、必要な処理モードを結びつける。

例:

- 比較 → 比較軸生成モード
- 要約 → 圧縮重視モード
- 設計 → 構造化生成モード
- 検証 → 根拠重視モード
- メール草案 → 文体調整モード

---

6. Priority Ordering

複数タスクがある時、何を先に処理するかを決める。

優先基準:

- 依頼の主目的
- 外部取得の必要性
- 制約確認の必要性
- 後続依存関係
- ユーザーがすぐ欲しい部分

---

7. Handover Specification

選んだルートに対し、何を持って引き渡すかを決める。

引継ぎ内容:

- タスク種別
- 成果物形式
- 使用言語
- 根拠要求
- 深さ
- 制約条件
- 優先順位
- 分解済みサブタスク列

これにより後段処理が安定する。

タスク類型の代表マップ

A. Informational Tasks

何かを知りたい、理解したいという依頼。

例:

- 説明
- 定義
- 背景整理
- 要約

B. Analytical Tasks

構造・因果・比較・評価を行う依頼。

例:

- 分析
- 検証
- 妥当性判定
- 原因分析

C. Generative Tasks

新しい成果物を作る依頼。

例:

- ノート
- 文書
- コード
- 表
- 設計案

D. Action Tasks

外部ツールやシステムに働きかける依頼。

例:

- 送信
- 作成
- 更新
- 削除
- 予約

E. Transform Tasks

形式や表現を変える依頼。

例:

- 書き換え
- 翻訳
- 表化
- Markdown化
- JSON化

下位構造

A. Input Pattern Recognizer

依頼文のパターンを認識する部分。

B. Intent Resolver

曖昧な要求を解釈し、主たる意図を決める部分。

C. Task Splitter

複合依頼をサブタスクへ分解する部分。

D. Route Binder

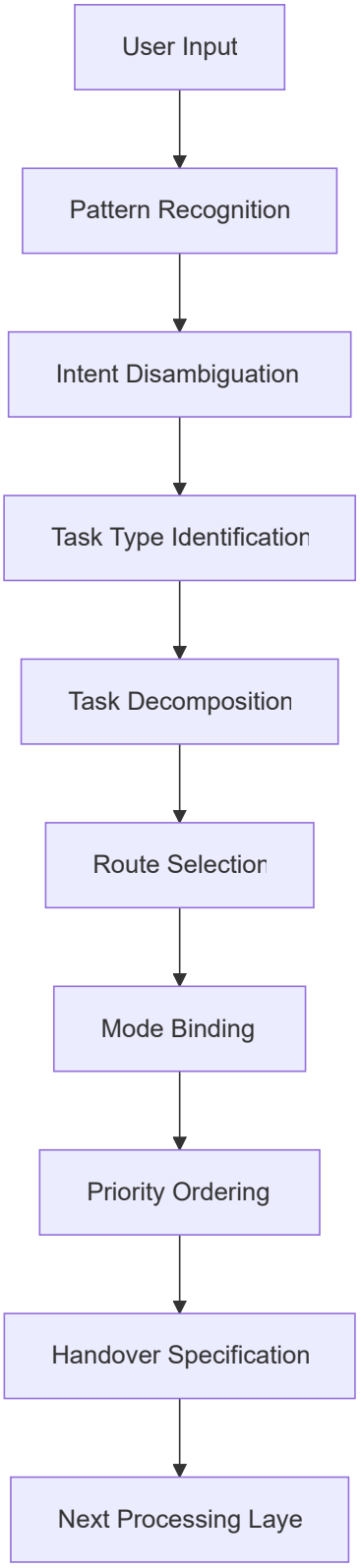
タスクと処理ラインを結びつける部分。

E. Handover Builder

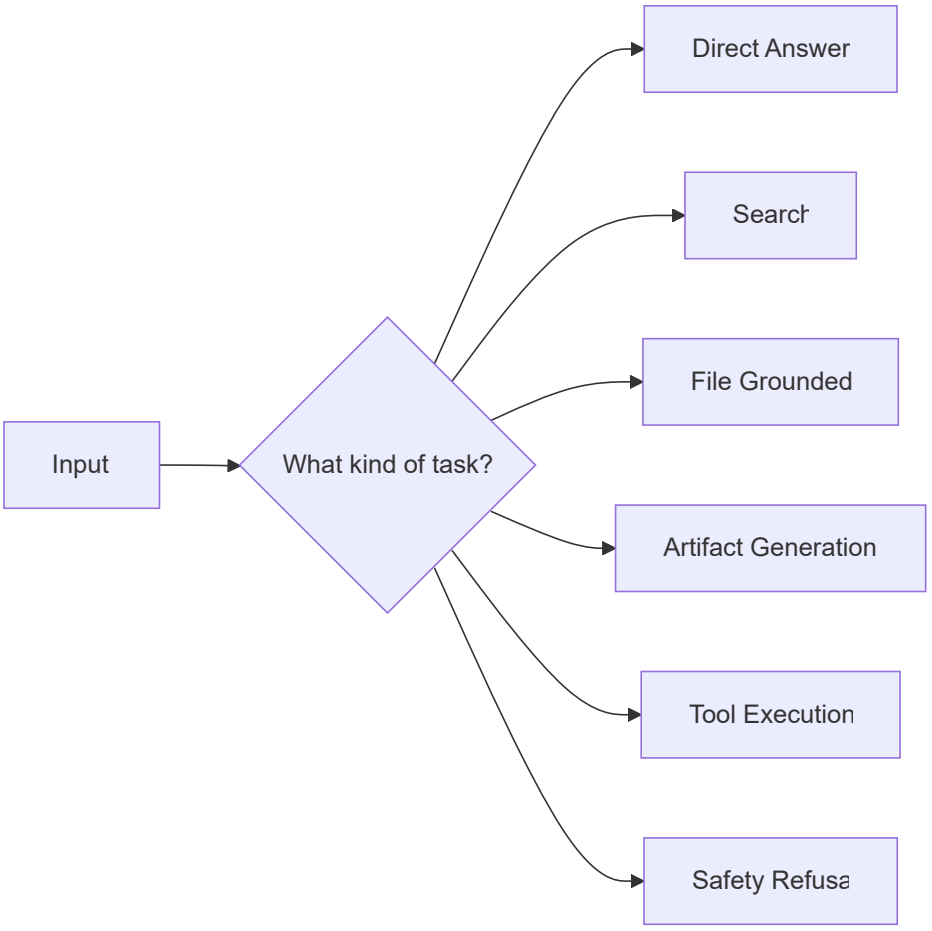
後段へ渡す処理仕様をまとめる部分。

全体構造

System Hub


User Input
Pattern Recognition
Intent Disambiguation
Task Type Identification
Task Decomposition
Route Selection
Mode Binding
Priority Ordering
Handover Specification
Next Processing Laye

ルーティング分岐の典型



典型例

入力	Task Routing の判定
これを要約してください	要約タスク
これを比較してください	比較分析タスク
最新の状況を教えてください	外部検索タスク
この資料をもとに作ってください	ファイル根拠生成タスク
このメールに返信してください	メール読解 + 草案生成タスク
予定を入れてください	外部実行タスク
この形式でノートを作ってください	成果物生成タスク

よくある失敗

1. 表面語だけで判断する

「作ってください」を全部同じ生成タスクとして扱ってしまう。

2. 複合依頼を分解しない

検索・比較・提案が混ざっているのに一括処理して粗くなる。

3. 主目的を取り違える

補助要求を主タスクとして処理してしまう。

4. ルートは合っているがモードが違う

たとえば比較なのに説明モードで流してしまう。

5. 引継ぎ仕様が弱い

後段がタスクを再解釈し直す必要があり、ぶれが出る。

設計原則

- 最初にタスク型を見極める
- 曖昧な依頼は成果物基準で解く
- 複合依頼は必要に応じて分解する
- 主目的を優先してルートを選ぶ
- ルートだけでなく処理モードも束ねる
- 後段に十分な引継ぎ情報を渡す

位置づけ

Task Routing は、
自然言語依頼を実行可能な処理系へ変換する入口の分配機構である。

これが弱いと、

- 検索すべきものを直答し
- 作るべきものを説明し

- 比較すべきものを要約し
- 実行すべきものを助言で終える

というずれが起こる。
したがってこの構造は、単なる分類器ではなく、
LLM 実行全体の方向を最初に定める進路決定装置である。

関連ノート

- LLM Control Layer
- Tool Orchestration
- Constraint Monitor
- Termination Control
- Intent Interpretation
- LLM Output Layer02_zettelkasten/00_system/Intent Interpretation
- LLM Output Layer

Termination Control

Termination Control

Termination Control は、LLM の処理を**どの時点で十分とみなし、探索・推論・ツール利用を打ち切って出力段階へ移るか**を決める構造である。
これは単に処理を止める機能ではなく、**十分性・費用対効果・未達の許容範囲を見極めて、最適な停止点を選ぶ統制機構**である。

要点

- 良い応答は、考え続けることではなく、適切な時点で止まることを含む
- 停止条件は一律ではなく、タスク型ごとに異なる
- 十分性とは「完璧」ではなく、「目的達成に必要な水準を満たした状態」である
- 停止判断は、深掘り継続の利得とコストの比較で決まる
- 未達部分がある場合でも、部分達成で止める方がよい場合がある

なぜ必要か

LLM は、与えられた入力から連続的に推論・検索・生成を続けられる。
しかし、実務上は次の問題がある。

- 考えすぎて冗長になる
- 検索しすぎて遅くなる
- 比較しすぎて結論が遅れる
- 細部にこだわって主目的を逃す
- 逆に、早く止まりすぎて不十分になる

このバランスを調整するのが Termination Control である。
つまりこれは、**処理の終点品質を決める構造**である。

中核機能

1. Sufficiency Assessment

現在の結果が、ユーザー要求に対して十分かを評価する。

見る観点:

- 質問に答えているか
- 成果物が完成しているか
- 必要根拠がそろっているか
- 指定形式を満たしているか
- 未解決部分が致命的でないか

ここでの「十分」は、厳密完全性ではなく、**目的充足性**である。

---

2. Marginal Gain Evaluation

これ以上続けた場合の追加利得があるかを見積もる。

追加利得の例:

- さらに正確になる
- 比較が深まる
- 不確実性が減る
- 欠けていた根拠が埋まる

一方でコスト:

- 時間
- 応答長
- ツール呼び出し回数
- 認知負荷
- 冗長化リスク

追加利得が小さければ、止めるべきである。

---

3. Stop Condition Matching

タスク類型に応じた停止条件を適用する。

例:

- 質問応答: 問いに直接答え、最低限の根拠がある
- 要約: 主要論点を落とさず圧縮できた
- 比較: 比較軸・差分・結論が出た
- 生成: 指定フォーマットの成果物が完成した
- ツール実行: 必要な操作が完了し結果確認も取れた
- 調査: 信頼できる根拠が十分集まった

4. Early Stop Decision

必要以上の深掘りを防ぐため、早期停止を判断する。

有効な場面:

- ユーザーが簡潔さを求めている
- 問いが単純
- 高速回答が価値を持つ
- 残り探索が重複的
- 形式生成がすでに完了している

---

5. Partial Completion Exit

完全達成は難しいが、部分達成なら可能な場合の停止を扱う。

例:

- 一部資料にアクセス不可
- 検索結果が限定的
- ツール障害
- 外部変更はできないが草案は作れる

この場合、

- 何ができたか
- 何が未達か
- 何が原因か

を明示して止める。

---

6. Fail-safe Termination

危険・制約違反・無意味な反復が起きる前に停止する。

対象:

- 禁止領域への進入
- 同じ失敗の反復
- 根拠なき推測の拡大

- ツールエラー連鎖
- 形式崩壊

---

7. Handover to Output

停止後、どの状態で Output Layer へ渡すかを整える。

引継ぎ内容:

- 最終結論
- 確定事項
- 未確定事項
- 根拠の強さ
- 省略した探索
- 出力形式要件

停止条件の代表類型

A. Goal Reached

目的が達成されたため止める。

B. Good Enough

完全ではないが、実用上十分なので止める。

C. Diminishing Returns

追加探索の価値が低いため止める。

D. Constraint Bound

制約上、これ以上進めないため止める。

E. Partial Completion

一部達成の形で止める。

F. Safety Stop

安全上、ここで打ち切る。

下位構造

A. Sufficiency Judge

結果が目的に足りているか判定する部分。

B. Gain Estimator

続行による追加価値を見積もる部分。

C. Stop Rule Library

タスクごとの停止条件を保持する部分。

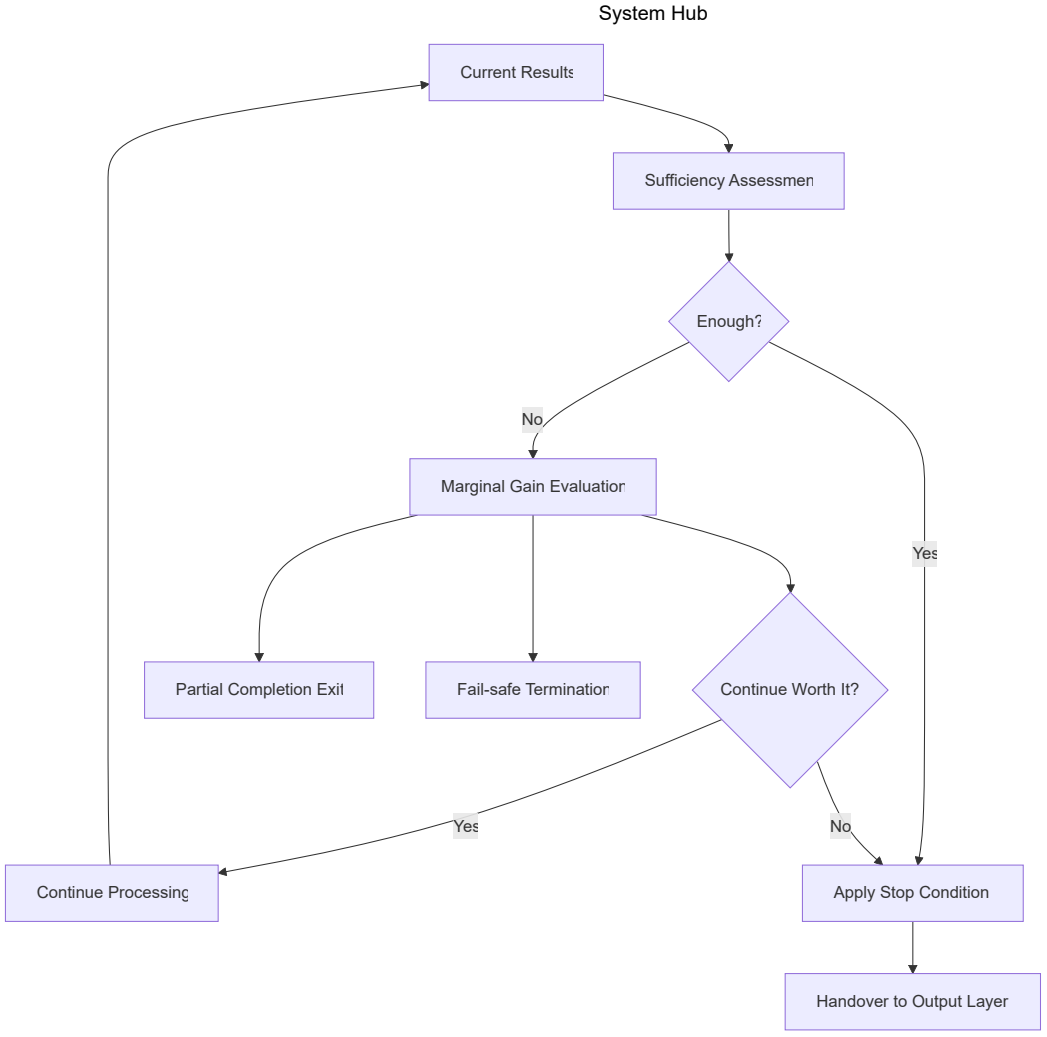
D. Partial Exit Manager

部分達成時の終了処理を担当する部分。

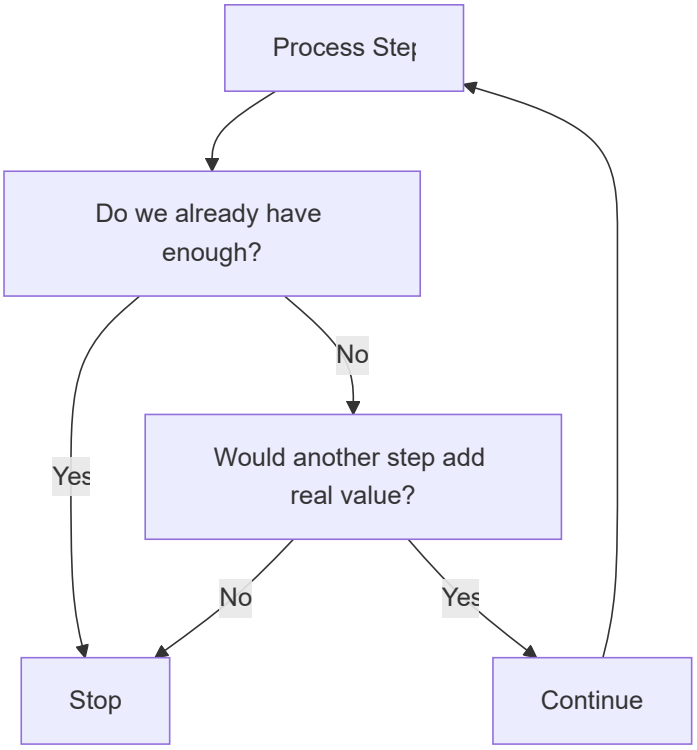
E. Finalizer

出力層へ渡すために結果を確定させる部分。

全体構造


System Hub
Current Results
Sufficiency Assessmen
Enough?
No
Yes
Marginal Gain Evaluatior
Partial Completion Exit
Fail-safe Terminatior
Continue Worth It?
Yes
No
Continue Processing
Apply Stop Condition
Handover to Output Layer

停止判断ループ



タスク別停止例

タスク	十分条件
一般質問	結論 + 最低限の根拠
比較	比較軸 + 差分 + 推奨
要約	主要論点を保持した圧縮
ノート生成	指定形式で完成
検索	信頼できる情報が十分収集済み
メール草案	送信可能な本文完成
スケジュール確認	空き時間または候補が明示済み

よくある失敗

1. 止まるのが遅い

十分な答えがあるのに、さらに調べて冗長になる。

2. 止まるのが早い

結論だけ出して根拠や形式要件が足りない。

3. 完璧主義に陥る

実用上十分なのに、未達ゼロを目指して過剰探索する。

4. 部分達成を失敗とみなす

全部できないと何も返さない。

5. 停止理由が不明確

なぜここで止めたかが内部的にも不明で、一貫性が崩れる。

設計原則

- 完全性より目的達成を基準にする
- 継続の利得とコストを比較する
- タスクごとに停止条件を変える
- 部分達成を正当に扱う
- 無意味な反復は早めに止める
- 停止後は未達部分を明示する
- 出力層へ渡す状態を整理して終える

位置づけ

Termination Control は、
LLM の処理を適切な完成点で閉じるための終端統制機構である。

これが弱いと、

- 冗長で遅い応答になり
- 逆に不十分な応答にもなり
- 実用上の完成度が安定しない

したがってこの構造は、単なる停止ボタンではなく、
十分性判断に基づいて成果物を確定させる完成管理機構である。

関連ノート

- LLM Control Layer
- Task Routing
- Tool Orchestration
- Constraint Monitor
- LLM Output Layer
- Confidence Calibration

Tool Orchestration

Tool Orchestration

Tool Orchestration は、LLM が利用可能な外部ツール群を、**目的に応じて選択し、順序づけ、接続し、結果を統合するための統制構造**である。
単に「ツールを使う」ことではなく、**どのツールを、いつ、何のために、どの順で使い、どの結果を次へ渡すか**を設計する。

要点

- ツール利用は単発呼び出しではなく、処理フローの一部として設計する
- 適切なツール選択は、タスク適合性・信頼性・コスト・制約で決まる
- ツールの出力はそのまま使わず、検証・要約・再接続する
- 複数ツールを連鎖させる場合、前段の出力を後段の入力へ正しく変換する
- ツール失敗時には代替経路・縮退運転・部分達成を考える

このノートの対象

このノートが扱うのは、たとえば次のような状況である。

- Web検索で最新情報を取り、その結果を要約する
- ファイルを読み、その内容に基づいて表や文書を作る
- カレンダーを読んで空き時間を調べ、予定案を作る
- メール内容を読み、返信草案を生成する
- Python で計算し、その結果をレポートへ埋め込む
- 画像生成や文書生成を最終成果物として出す

つまり Tool Orchestration は、**LLM を単独推論器から、外部資源を使える実行システムへ拡張するための中心機構**である。

中核機能

1. Tool Need Detection

まず、そのタスクにツールが本当に必要かを判定する。

判定観点:

- 内部知識だけで足りるか
- 最新情報が必要か
- ユーザー固有データが必要か
- ファイルや外部状態の参照が必要か
- 実計算や変換処理が必要か
- 外部システムを書き換える必要があるか

ツールは便利だが、不要な呼び出しは遅延・複雑化・失敗点増加を招く。
したがって最初に「使うべきか」を見極める必要がある。

---

2. Tool Selection

必要であると判定した後、どのツールを使うかを決める。

代表選択基準:

- 問題との適合性
- 情報の新しさ
- 取得対象の種類
- 書き込み権限の要否
- 精度と再現性
- 入出力形式の相性
- コストと速度

例:

- 最新ニュース → Web
- ユーザーのGoogle Drive資料 → File Search
- スケジュール確認 → Calendar
- 数値集計やファイル生成 → Python
- 画像生成 → Image Tool

---

3. Invocation Planning

選んだツールを、どの順番でどう呼ぶかを設計する。

要素:

- 前処理の有無
- 1回で済むか、反復が必要か

- 並列可能か、逐次依存か
- どの結果を保持するか
- エラー時の分岐
- どの時点で停止するか

ここでは「ツールの列」を作るだけではなく、**一連の実行計画**を組み立てる。

---

4. Input Adaptation

自然言語の要求を、各ツールが理解できる形式へ変換する。

例:

- 調べてください → 検索クエリ
- このPDFの要点 → ファイル参照 + 該当範囲取得
- 明日空いてる時間 → 時間範囲指定付きカレンダー検索
- この表を整形 → データフレーム操作指示

この変換が甘いと、適切なツールを選んでも精度が落ちる。

---

5. Output Capture and Interpretation

ツールの返した結果を受け取り、意味のある情報として再解釈する。

処理:

- 生データの抽出
- 重要箇所の選別
- 必要情報の正規化
- ノイズ除去
- 後続処理への受け渡し

重要なのは、ツール結果をそのまま最終回答に貼るのではなく、**LLM の文脈理解に再接続する**ことである。

---

6. Multi-Tool Coordination

複数ツールをまたぐ処理を調整する。

例:

- Webで情報取得 → Pythonで整理 → 文書生成
- File Searchで資料取得 → 要点抽出 → メール草案生成
- Contactsで相手を引く → Calendarで日程確認 → Event作成

ここでは、

- データ形式の橋渡し
- 手順の依存管理
- 中間結果の保持
- 成功条件の定義

が重要になる。

---

7. Failure Handling

ツール失敗時の扱いを決める。

失敗類型:

- ツール未接続
- 権限不足
- 検索結果不足
- APIエラー
- フォーマット不一致
- 出力が空
- タイムアウト

対応原則:

- 代替ツールを探す
- 内部知識で部分的に補う
- できた部分まで返す
- 不足部分を明示する
- 同じ失敗を無限反復しない

---

8. Result Handover

最終的に、ツール実行結果を次の処理層へ渡す。

渡す先:

- Reasoning Layer
- Decision Layer
- Output Layer
- Artifact Generator
- Memory Candidate Selector

ここでは、生のツール出力ではなく、**次段が使いやすい中間表現**へ整えて渡すのが望ましい。

ツール利用の基本原則

原則1: 必要な時だけ使う

ツールは能力を増やすが、複雑性も増やす。
不要なら使わない方がよい。

原則2: 最も直接的なツールを優先する

遠回りな組み合わせより、問題に最も近い専用ツールを選ぶ。

原則3: 結果を再評価する

ツールの返した内容を盲信しない。
不足・誤差・曖昧さを点検する。

原則4: ツール結果は意味変換して使う

生出力を貼るのではなく、文脈に沿って要約・比較・再構成する。

原則5: 失敗しても全体を止めすぎない

一部失敗しても、可能な範囲で成果を返す。

下位構造

A. Tool Registry View

利用可能なツール群を把握し、その能力範囲を管理する部分。

内容:

- 何が読めるか

- 何が書けるか
- 何が計算できるか
- 何が生成できるか

B. Tool Matcher

タスクとツール候補を対応づける部分。

役割:

- 問題タイプの認識
- 適切なツール候補列挙
- 優先順位づけ

C. Tool Chain Planner

複数ツールの呼び出し順を計画する部分。

役割:

- 順序設計
- 入出力接続
- 分岐設計
- 終了条件設定

D. Adapter

自然言語とツール入出力の橋渡しを行う部分。

役割:

- クエリ生成
- 引数成形
- 出力正規化
- 中間形式変換

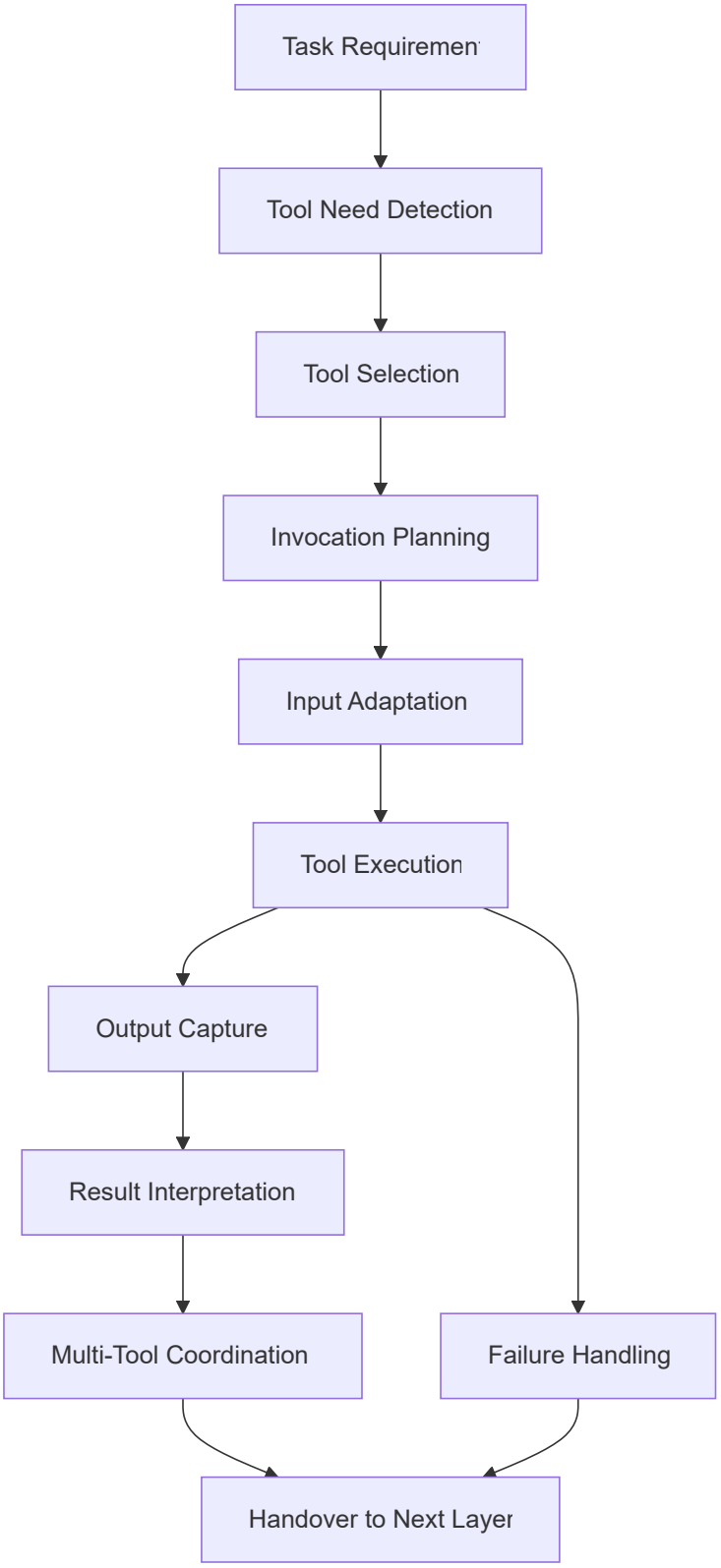
E. Recovery Path

失敗時の代替経路を管理する部分。

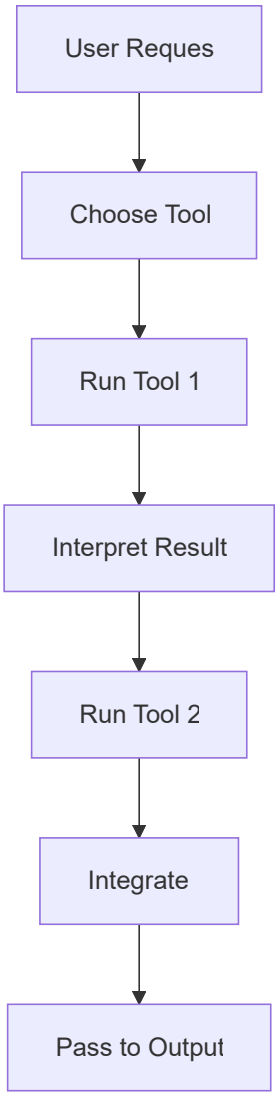
役割:

- リトライ判断
- 代替ツール選択
- 部分応答化
- 制約説明

全体構造


Task Requiremen
Tool Need Detection
Tool Selection
Invocation Planning
Input Adaptation
Tool Execution
Output Capture
Result Interpretation
Multi-Tool Coordination
Failure Handling
Handover to Next Layer

連鎖実行の典型形



典型パターン

1. Search → Summarize

最新情報取得の基本形。

2. Read → Compare → Answer

複数資料比較の基本形。

3. Query → Compute → Report

数値処理や分析の基本形。

4. Read State → Propose Action

カレンダー・メール・在庫など外部状態を読んで提案する形。

5. Write Action → Confirm

メール送信や予定作成など、外部変更を伴う形。

よくある失敗

1. ツールを使うべきなのに使わない

最新情報や外部状態が必要なのに、内部知識だけで答えてしまう。

2. ツールを使わなくてよいのに使う

簡単な説明まで外部検索して冗長になる。

3. ツール結果を読まずに流す

取得はしたが解釈が弱く、有用な答えにならない。

4. 複数ツールの接続が粗い

前段の出力が後段の入力形式に合わず、流れが崩れる。

5. 失敗時の代替がない

1つのツール失敗で全体が止まる。

設計原則

- 適合性の高いツールを優先する
- ツール呼び出し前に目的を明確にする
- 入力変換を丁寧に行う
- 出力は必ず再解釈する
- 複数ツールは依存関係を明示してつなぐ
- 失敗時は部分達成を志向する

- ツールは手段であって、答えそのものではない

位置づけ

Tool Orchestration は、
LLM が外部世界に手を伸ばすための実行編成機構である。

この構造が弱いと、

- 必要情報を取りに行けず
- 取っても活かせず
- 多段処理も安定しない

したがってこれは、ツール利用の補助ではなく、
拡張知能としてのLLMを成立させる運用中枢である。

関連ノート

- LLM Control Layer
- Task Routing
- Constraint Monitor
- Termination Control
- LLM Reasoning Layer
- LLM Output Layer- LLM Reasoning Layer
- LLM Output Layer

Memory Injection Rule

Memory Injection Rule

Memory Injection Rule は
Zettelkasten 内の知識を **LLM推論に注入する規則**である。

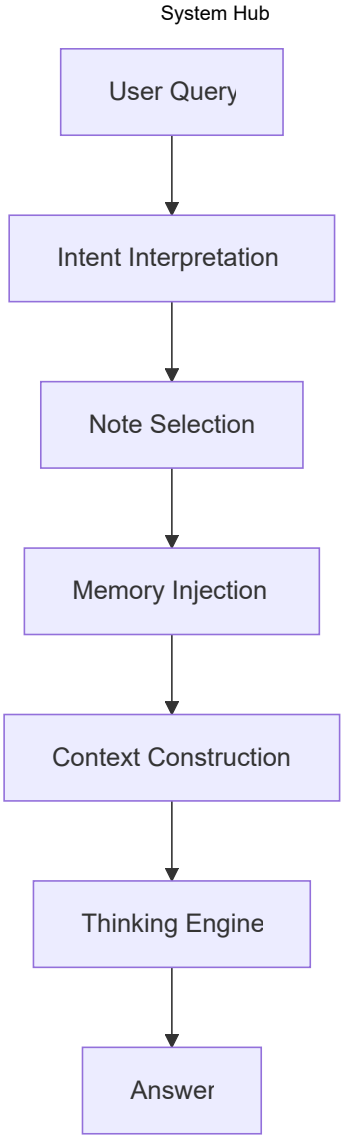
LLMは単体では長期記憶を持たないため
Vaultの知識を **Runtime Memory として一時注入する**。

この仕組みにより

- 長期知識利用
- 推論精度向上
- ハルシネーション抑制
- 自動思考

が可能になる。

Memory Injection Pipeline



Memory Types

LLMに注入されるMemoryは4種類ある。

Memory	内容

Semantic Memory | 概念知識 |
Structural Memory | 構造知識 |
Mechanism Memory | 原理知識 |
Case Memory | 事例 |

Memory Source

Memoryは以下のノートから取得する。

```
Kernel
Concept
Structure
Mechanism
Pattern
Case
Method
```

Memory Priority

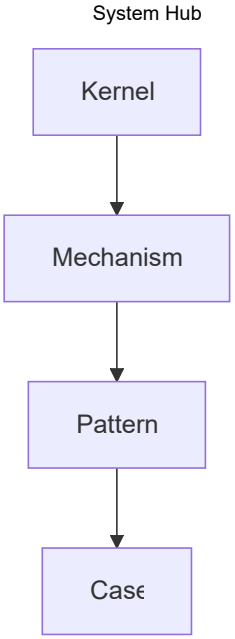
Memoryの優先順位は以下。

```
Kernel
Mechanism
Pattern
Concept
Case
Structure
Method
Domain
```

抽象度が高いものを優先する。

Memory Injection Structure

LLMに渡すMemory構造。



この構造を **推論骨格**として使用する。

Kernel Injection Rule

最初にKernelを注入する。

Kernelは

```
世界の基本原理
```

である。

例

```
限定合理性
注意資源制約
社会的影響
情報非対称
```

Kernelは最大3個。

Mechanism Injection Rule

次にMechanismを注入する。

Mechanismは

```
Kernelが現実でどう作用するか
```

を説明する。

例

```
Signaling Mechanism
Coordination Mechanism
Free Rider Mechanism
```

最大5個。

Pattern Injection Rule

Mechanismの **繰り返し構造**をPatternとして注入する。

例

```
寡占パターン
炎上パターン
規範形成パターン
```

最大5個。

Case Injection Rule

Patternを説明する事例をCaseとして注入する。

例

```
ドイツ革命1918
韓国併合
Twitter炎上
```

最大3個。

Memory Injection Depth

Memoryは最大 **4階層**まで。

```
Kernel
↓
Mechanism
↓
Pattern
↓
Case
```

これ以上深くすると推論が不安定になる。

Memory Compression Rule

Context Window不足時の削減順。

```
Case削除
↓
Pattern要約
↓
Concept要約
```

Kernelは削除しない。

Memory Stability Rule

LLMが推論するときは必ず

```
Kernel
↓
Mechanism
↓
Pattern
↓
Case
```

の順で思考する。

Memory Consistency Rule

複数Memoryが矛盾する場合

```
Kernel優先
```

とする。

Memory Injection Template

LLMに渡すMemoryテンプレート。

```
# Kernel

...

# Mechanism

...

# Pattern

...

# Case

...
```

Related Notes

- LLM Runtime Rule
- System Prompt / Note Selection Rule
- Context Construction Rule
- Intent Interpretation
- 02_zettelkasten/03_thinking_engine/Thinking Engine

- Constraint Monitor

Self Reflection Rule

Self-Reflection Rule

Self-Reflection Rule は
LLMが生成した推論と回答を **自己検証する仕組み**を定義する。

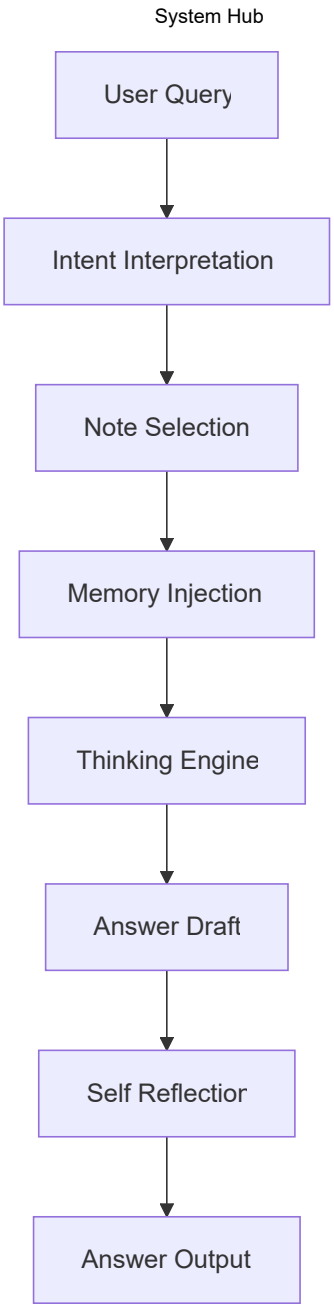
LLMは生成時に誤推論・知識欠落・論理矛盾を起こす可能性があるため、
回答前に **Reflection Phase** を挿入する。

この仕組みにより

- ハルシネーション抑制
- 推論品質向上
- 知識整合性確認
- Vault構造との整合

を実現する。

Self-Reflection Pipeline



Reflection Phase

Self-Reflectionは以下の4段階で行う。

Phase	内容

Consistency Check | 知識整合 |
Mechanism Check | 原理検証 |
Pattern Check | パターン検証 |
Case Check | 事例検証 |

Consistency Check

回答が **Kernelと矛盾していないか**確認する。

例

```
限定合理性
注意資源制約
社会的影響
```

Kernelと矛盾する回答は修正する。

Mechanism Check

回答が **Mechanismに基づいているか**確認する。

チェック項目

```
原因
作用
結果
```

Mechanismが存在しない回答は

```
説明不足
```

と判断する。

Pattern Check

回答が **既知のPatternと一致するか**確認する。

例

```
規範形成パターン
寡占パターン
炎上パターン
```

Patternと一致しない場合は

```
新規パターン
```

として扱う。

Case Check

回答が **実際のCaseで説明可能か**確認する。

例

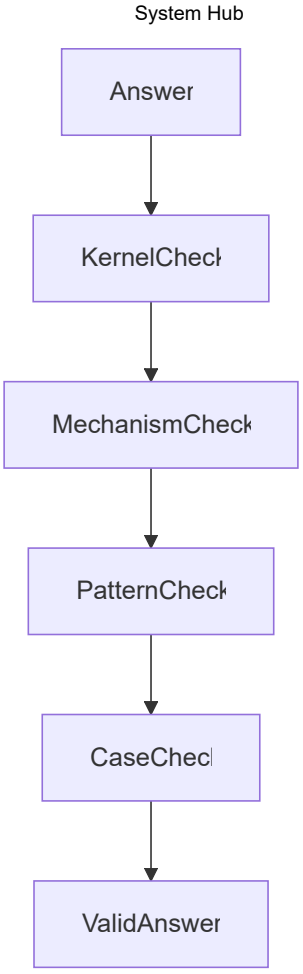
```
ドイツ革命1918
韓国併合
SNS炎上
```

Caseで説明できない場合

```
抽象理論
```

として扱う。

Reflection Decision Tree



Reflection Output Types

Self-Reflectionの結果は3種類。

Type	内容

Valid | 問題なし |
Partial | 一部不十分 |
Invalid | 再推論必要 |

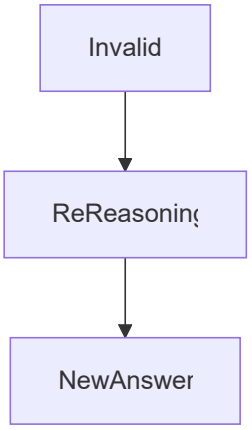
Re-Reasoning Rule

Reflection結果が

```
Invalid
```

の場合

Thinking Engineを **再実行する。**



Reflection Compression Rule

Reflectionは **簡潔に行う。**

最大

```
300 tokens
```

以内。

Reflection Priority

Reflectionは次の順で行う。

```
Kernel
↓
Mechanism
↓
Pattern
↓
Case
```

Reflection Template

LLM内部で使用するテンプレート。

```
Kernel consistency:
Mechanism explanation:
Pattern alignment:
Case verification:

Reflection result:
```

Related Notes

- LLM Runtime Rule
- Memory Injection Rule
- Context Construction Rule
- 02_zettelkasten/03_thinking_engine/Thinking Engine
- Constraint Monitor

Reasoning Strategy Rule

Reasoning Strategy Rule

Reasoning Strategy Rule は
LLMが問題を解くとき **どの推論戦略を使うかを決定する規則**である。

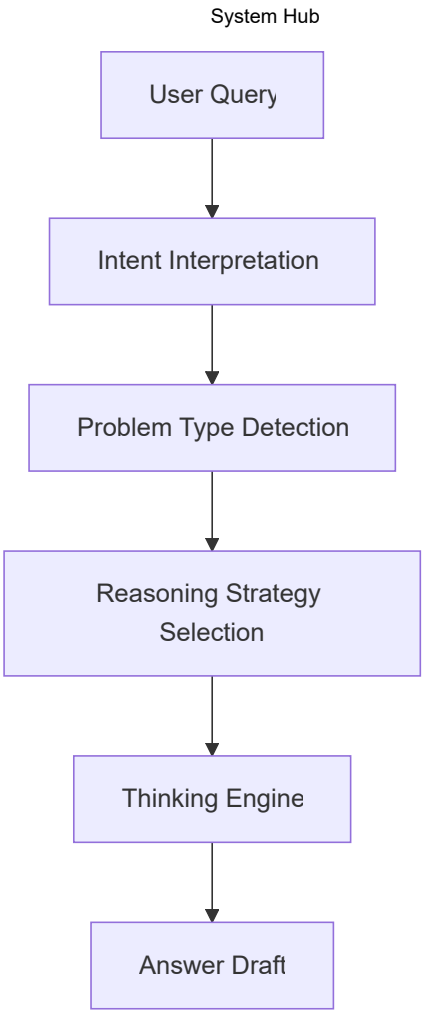
すべての問題は同じ推論方法で解けるわけではないため
問題タイプに応じて **最適なReasoning Strategyを選択する。**

この仕組みにより

- 推論精度向上
- 推論速度向上
- ハルシネーション抑制
- Zettelkasten構造との整合

を実現する。

Reasoning Strategy Pipeline



Problem Types

問題は次のタイプに分類する。

Type	内容

Explanation | 原理説明 |
Causal | 原因分析 |
Comparison | 比較 |
Prediction | 予測 |
Design | 設計 |
Diagnosis | 問題特定 |

Reasoning Strategies

LLMが使用する基本戦略。

Strategy	内容

Causal Reasoning | 因果推論 |
Comparative Reasoning | 比較推論 |
Analogical Reasoning | アナロジー |
Hypothesis Reasoning | 仮説推論 |
Structural Reasoning | 構造推論 |

Strategy Selection Rule

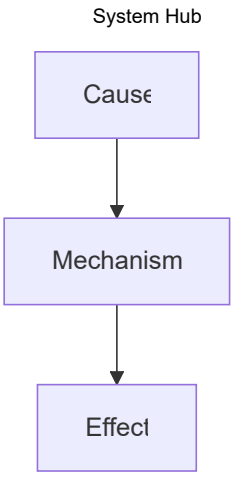
問題タイプごとの戦略。

Problem	Strategy

Explanation | Structural Reasoning |
Causal | Causal Reasoning |
Comparison | Comparative Reasoning |
Prediction | Hypothesis Reasoning |
Design | Analogical Reasoning |
Diagnosis | Causal + Hypothesis |

Causal Reasoning

因果関係を特定する。

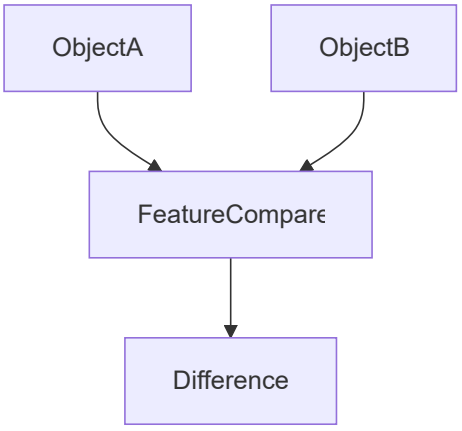


使用ノート

```
Mechanism
Pattern
Case
```

Comparative Reasoning

複数対象を比較する。

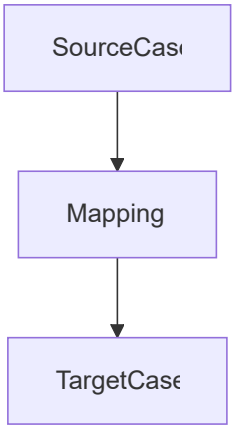


使用ノート

```
Concept
Structure
Pattern
```

Analogical Reasoning

類似事例から推論する。

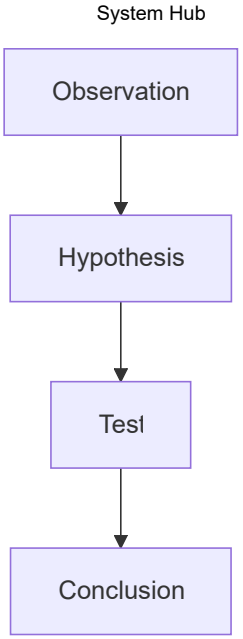


使用ノート

```
Pattern
Case
Mechanism
```

Hypothesis Reasoning

仮説を生成して検証する。

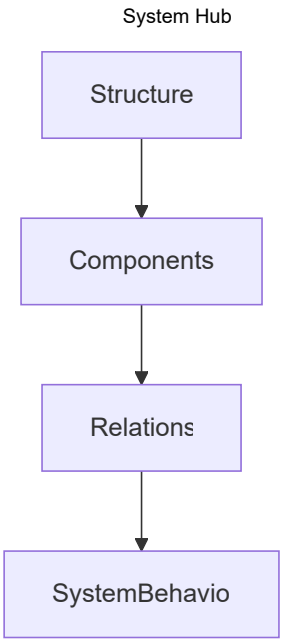


使用ノート

```
Mechanism
Case
Method
```

Structural Reasoning

構造を分析する。



使用ノート

```
Structure
Concept
Pattern
```

Strategy Priority

複数戦略が必要な場合。

```
Causal
↓
Structural
↓
Comparative
↓
Analogical
↓
Hypothesis
```

Strategy Depth

推論深度は最大

```
4 steps
```

とする。

例

```
Kernel
↓
Mechanism
↓
Pattern
↓
Case
```

Strategy Stability Rule

推論は必ず

```
Mechanism説明
```

を含める。

Mechanismがない推論は

```
説明不足
```

と判断する。

Strategy Output Structure

回答の基本構造。

```
結論
↓
構造
↓
メカニズム
```

```
↓
事例
```

Related Notes

- Knowledge Activation Rule
- Memory Injection Rule
- Context Construction Rule
- 02_zettelkasten/03_thinking_engine/Thinking Engine
- Self Reflection Rule

Context Construction Rule

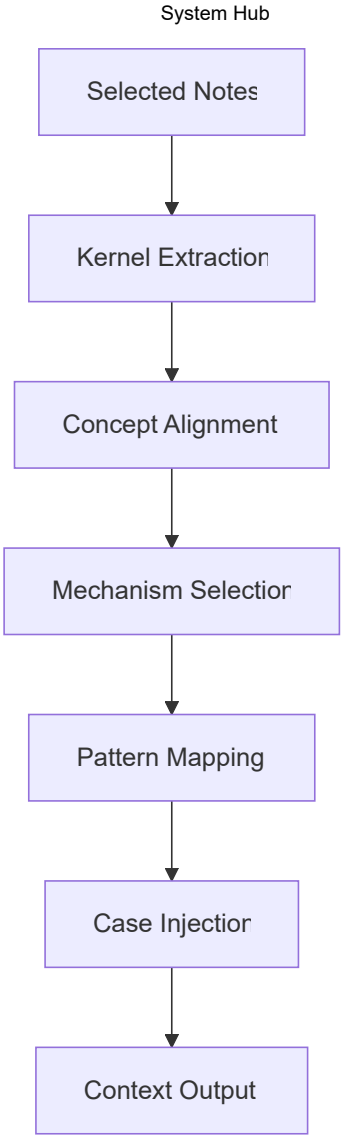
Context Construction Rule

Context Construction Rule は
Vault 内のノートを **LLMに渡すコンテキストとして組み立てる規則**である。

このルールの目的は次の4つ。

- Context Window最適化
- 推論精度向上
- ハルシネーション抑制
- 再現可能な推論

Context Construction Pipeline



Context Layer Structure

LLMに渡すコンテキストは **5層構造**とする。

```
Kernel
Concept
Mechanism
Pattern
Case
```

抽象 → 具体の順に並べる。

Context Layout

LLMに渡すContextは以下の構造で構築する。

```
# Kernel

基本原理

# Concept

主要概念

# Mechanism

作用メカニズム

# Pattern

一般パターン

# Case

具体事例
```

この順序を崩さない。

Kernel Extraction Rule

選択されたノートの中から
最も抽象的な原理を優先して抽出する。

例

```
注意資源制約
限定合理性
社会的影響
```

Kernelは **最大3つまで**。

Concept Alignment Rule

Conceptは

```
Kernelを説明する概念
```

を選ぶ。

例

```
認知
動機
権力
情報
```

Conceptは **最大5個**。

Mechanism Selection Rule

Mechanismは

```
Kernel → Concept
```

を **因果的に接続するもの**を選ぶ。

例

```
Signaling Mechanism
Information Asymmetry
Coordination Mechanism
```

Mechanismは **最大5個**。

Pattern Mapping Rule

Patternは

```
Mechanismの繰り返し構造
```

として選択する。

例

```
規範形成パターン
寡占パターン
炎上パターン
```

Patternは **最大5個**。

Case Injection Rule

Caseは

```
Patternを説明する実例
```

として挿入する。

例

```
ドイツ革命1918
韓国併合
Twitter炎上事件
```

Caseは **最大3個**。

Context Size Rule

Contextは次の範囲に制限する。

```
1500〜4000 tokens
```

超える場合は

1 Kernel優先
2 Mechanism優先
3 Case削減

で削減する。

Ordering Rule

Contextは必ず以下の順序で並べる。

```
Kernel
Concept
Mechanism
Pattern
Case
```

順序を変えると推論性能が低下する。

Context Stability Rule

以下のルールを守る。

- Kernel優先
- Mechanism必須
- Caseは補助

つまり

```
Kernel > Mechanism > Pattern > Case
```

Context Compression Rule

Context Window不足時は

```
Case削除
↓
Pattern要約
↓
Concept要約
```

の順で圧縮する。

Kernelは削除しない。

Context Output Template

LLMに渡す最終形式。

```
# Kernel

...

# Concept

...

# Mechanism

...

# Pattern

...

# Case

...
```

Related Notes

- LLM Runtime Rule
- System Prompt / Note Selection Rule
- Intent Interpretation
- 02_zettelkasten/03_thinking_engine/Thinking Engine
- Constraint Monitor

Knowledge Activation Rule

Knowledge Activation Rule

Knowledge Activation Rule は
Vault内の膨大なノートから **どの知識を起動するか**を決定する規則である。

Zettelkastenでは

> 全知識を読むことはできない

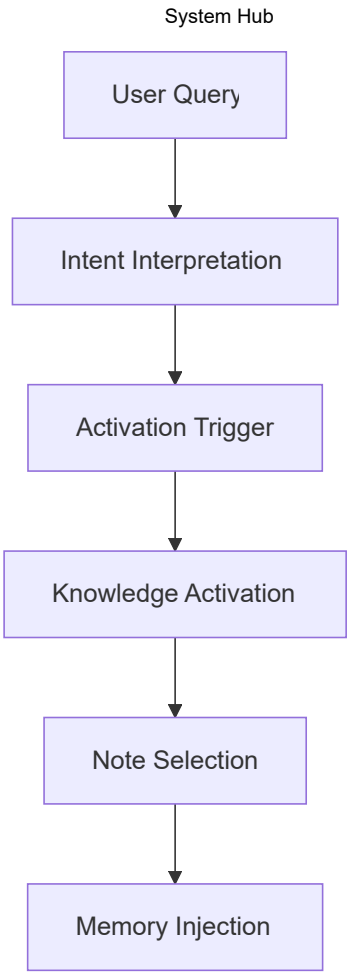
ため、LLMは **必要な知識だけを起動（activate）する。

この仕組みにより

- Context Window節約
- 推論速度向上
- 知識検索精度向上
- ハルシネーション抑制

を実現する。

Knowledge Activation Pipeline



Activation Trigger

知識起動は以下の **3種類のトリガー**によって開始される。

Trigger	内容

Concept Trigger | 概念一致 |
Mechanism Trigger | 原理一致 |
Pattern Trigger | 構造一致 |

Concept Trigger

質問に含まれる **概念語**によってノートを起動する。

例

```
権力
認知
動機
都市
市場
```

この場合

```
Conceptノート
```

が起動する。

Mechanism Trigger

質問が

```
なぜ
どうして
仕組み
メカニズム
```

を含む場合、Mechanismを起動する。

例

```
Signaling Mechanism
Coordination Mechanism
Information Asymmetry
```

Pattern Trigger

質問が

```
繰り返し構造
歴史パターン
社会現象
```

を含む場合、Patternを起動する。

例

```
炎上パターン
寡占パターン
規範形成パターン
```

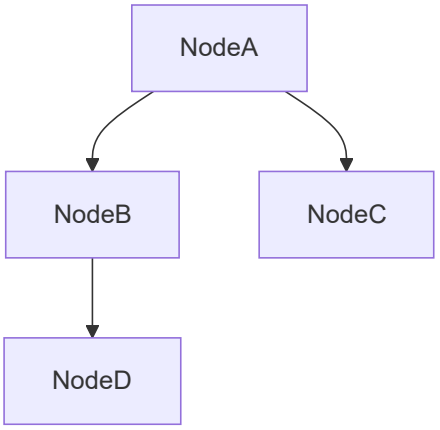
Activation Priority

知識起動の優先順位。

```
Kernel
Mechanism
Pattern
Concept
Case
Structure
Method
Domain
```

Activation Radius

1つのノートが起動した場合
リンクされたノートも探索する。



ただし探索深度は

```
2リンクまで
```

とする。

Activation Scope

LLMが同時に起動するノート数。

```
5〜12ノート
```

Activation Types

知識起動には3種類ある。

Type	内容

Direct Activation | 直接一致 |
Semantic Activation | 意味一致 |
Structural Activation | 構造一致 |

Direct Activation

質問語とノート名が一致。

例

```
韓国併合
限定合理性
寡占構造
```

Semantic Activation

意味的に近い概念。

例

```
市場支配
→ 寡占
```

Structural Activation

構造が一致。

例

```
権力集中
→ 寡占構造
```

Activation Stop Rule

次の条件で探索停止する。

```
12ノート到達
```

または

```
Kernel + Mechanism + Pattern
```

が揃った時。

Activation Output

Knowledge Activationの結果は

```
Activated Notes
```

として出力される。

```
Kernel
Mechanism
```

```
Pattern
Concept
Case
```

Activation Stability Rule

知識起動は以下を守る。

- Kernel優先
- Mechanism必須
- Pattern優先

つまり

```
Kernel > Mechanism > Pattern
```

Activation Template

LLM内部フォーマット。

```
Activated Kernel

Activated Mechanism

Activated Pattern

Activated Concept

Activated Case
```

Related Notes

- Intent Interpretation
- Note Selection
- Memory Injection Rule
- Context Construction Rule

- 02_zettelkasten/03_thinking_engine/Thinking Engine

LLM Reasoning Architecture

LLM Reasoning Architecture

LLM Reasoning Architecture は、LLM が入力を受け取り、文脈を参照し、推論し、出力し、評価し、必要に応じて再推論するまでの全体構造を整理するための上位ノートである。

これは単なる「推論手順」ではなく、**入力処理・文脈構成・思考過程・記憶参照・出力整形・制御・評価**を含む統合アーキテクチャである。
Zettelkasten や業務OSに接続する場合も、この構造を起点に各モジュールを分離して設計すると安定する。

---

1. このアーキテクチャの目的

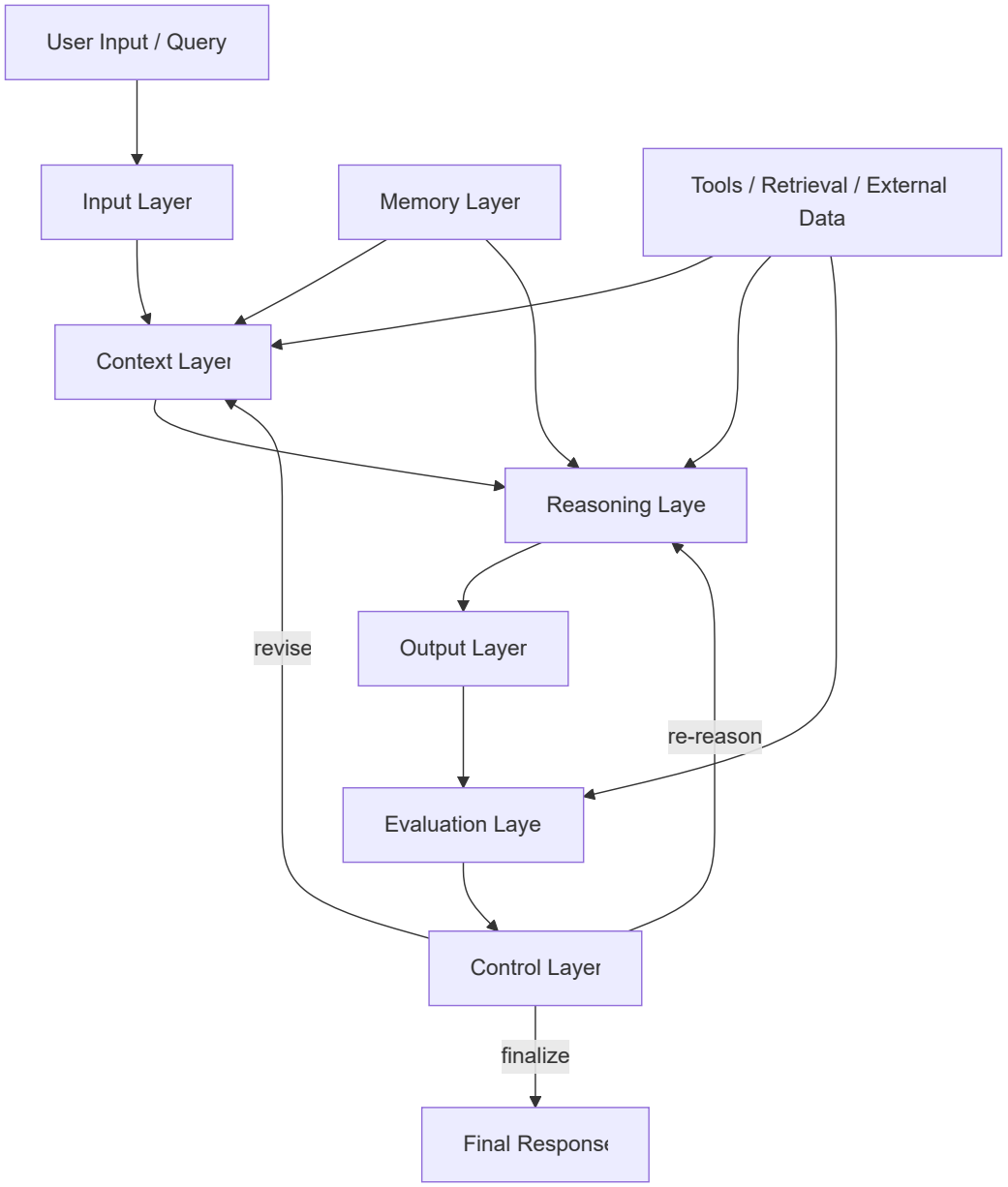
LLM は単体では「次の単語を予測する装置」に近いが、実運用では以下が必要になる。

1. **何を問題として受け取るか**
2. **どの文脈を読むか**
3. **どの推論形式で考えるか**
4. **どの知識を参照するか**
5. **どの粒度で答えるか**
6. **答えが妥当かをどう検査するか**
7. **必要なら再思考するか**

したがって、LLM を単なる生成器ではなく、
「入力→文脈化→推論→出力→評価→制御」から成る思考システムとして扱う必要がある。

---

2. 全体構造



3. 構成要素

3-1. Input Layer

Input Layer は、ユーザー入力をそのまま受け取るのではなく、
問いとして整形可能な単位へ分解する層である。

主な役割

- 指示の抽出
- 質問の型判定
- 対象の特定
- 制約条件の抽出
- 出力形式の抽出
- 暗黙要件の補完

典型的な処理

- 「何をしてほしいか」
- 「対象は何か」
- 「どこまで深くやるか」
- 「コードブロックが必要か」
- 「既存の vault 構造に合わせる必要があるか」

入る下位ノート

- LLM Input Layer
- Task Parsing
- Intent Detection
- Constraint Extraction
- Output Format Recognition

3-2. Context Layer

Context Layer は、問いを理解するために必要な文脈を構成する層である。

LLM は与えられた文字列しか見えないため、
実用上はこの層で**適切な前提・関連ノート・履歴・定義**を集める必要がある。

主な役割

- 会話履歴参照
- 関連ノート選択
- 定義の参照
- 前提条件の確定
- 文脈の圧縮
- ノイズ除去

文脈の種類

- 会話文脈

- タスク文脈
- ドメイン文脈
- ユーザー固有文脈
- システム制約文脈
- 外部データ文脈

入る下位ノート

- LLM Context Layer
- Context Assembly
- Context Compression
- Relevant Context Selection
- Prompt State Management

3-3. Reasoning Layer

Reasoning Layer は、このアーキテクチャの中心である。
ここでは文脈をもとに、問いに対する中間的思考を形成する。

この層は単一ではなく、複数の推論モードを持つ。

主な推論モード

- 演繹推論
- 帰納推論
- 仮説推論
- 因果推論
- 比較推論
- 構造推論
- 診断推論
- 設計推論
- 反証推論
- 分岐推論

役割

- 問題の再定義
- 論点の分解
- 仮説生成
- 選択肢設計
- 因果連鎖の整理

- 抽象化と具体化の往復
- 結論候補の生成

入る下位ノート

- LLM Reasoning Layer
- Causal Reasoning
- Hypothesis Generation
- Comparative Reasoning
- Decision Reasoning
- Solution Design Reasoning
- Abductive Reasoning
- [[Reflective Reasoning]

3-4. Memory Layer

Memory Layer は、現在の推論に必要な知識や過去情報を供給する層である。

LLM 単体の内部知識だけに依存すると、
更新不能・曖昧・再利用困難になるため、外部記憶との接続が重要になる。

主な役割

- 長期知識の保持
- ノート・DB 参照
- 過去会話の再利用
- ユーザー固有設定の保持
- 定義と用語体系の維持
- 推論資源の再利用

記憶の種類

- Semantic Memory
- Episodic Memory
- Procedural Memory
- Working Memory
- Externalized Memory

入る下位ノート

- LLM Memory Layer
- Semantic Memory

- Working Memory
- External Knowledge Interface
- User Preference Memory
- Reasoning State Memory

3-5. Output Layer

Output Layer は、推論結果を人間が使える形へ整形する層である。

ここでは「何を答えるか」だけでなく、
どう並べるか・どの粒度で見せるか・どの記法で出すかが重要になる。

主な役割

- 結論の明示
- 論理展開の整形
- コードブロック化
- markdown 化
- 表や箇条書きへの整形
- ノート形式への変換
- Mermaid 図への変換

出力形式の例

- 通常回答
- Obsidian ノート
- Hub ノート
- Structure ノート
- Mermaid 図
- テーブル
- テンプレート
- 手順書

入る下位ノート

- LLM Output Layer
- Response Structuring
- Markdown Rendering
- Note Generation
- Mermaid Generation
- Format Adaptation

3-6. Evaluation Layer

Evaluation Layer は、出力候補が妥当かどうかを検査する層である。
ここがないと、LLM は「それらしく見える誤答」をそのまま出しやすい。

主な役割

- 一貫性検査
- 制約適合性検査
- 網羅性検査
- 論理飛躍の検査
- 形式不備の検査
- 事実性の検査
- ユーザー要求との整合確認

典型的な観点

- 質問に答えているか
- 指示形式を守っているか
- 前提がずれていないか
- Mermaid が閉じているか
- 必要なノート名が揃っているか
- 構造が重複していないか

入る下位ノート

- LLM Evaluation Layer
- Consistency Check
- Constraint Check
- Coverage Check
- Format Validation
- Reasoning Quality Review

3-7. Control Layer

Control Layer は、全体の流れを管理する層である。
いわば **メタ思考・実行制御・再試行判断** を担う。

主な役割

- 推論深度の調整

- モード切替
- 再推論判断
- 打ち切り判断
- ツール利用判断
- 文脈再収集判断
- 出力粒度の制御

制御対象

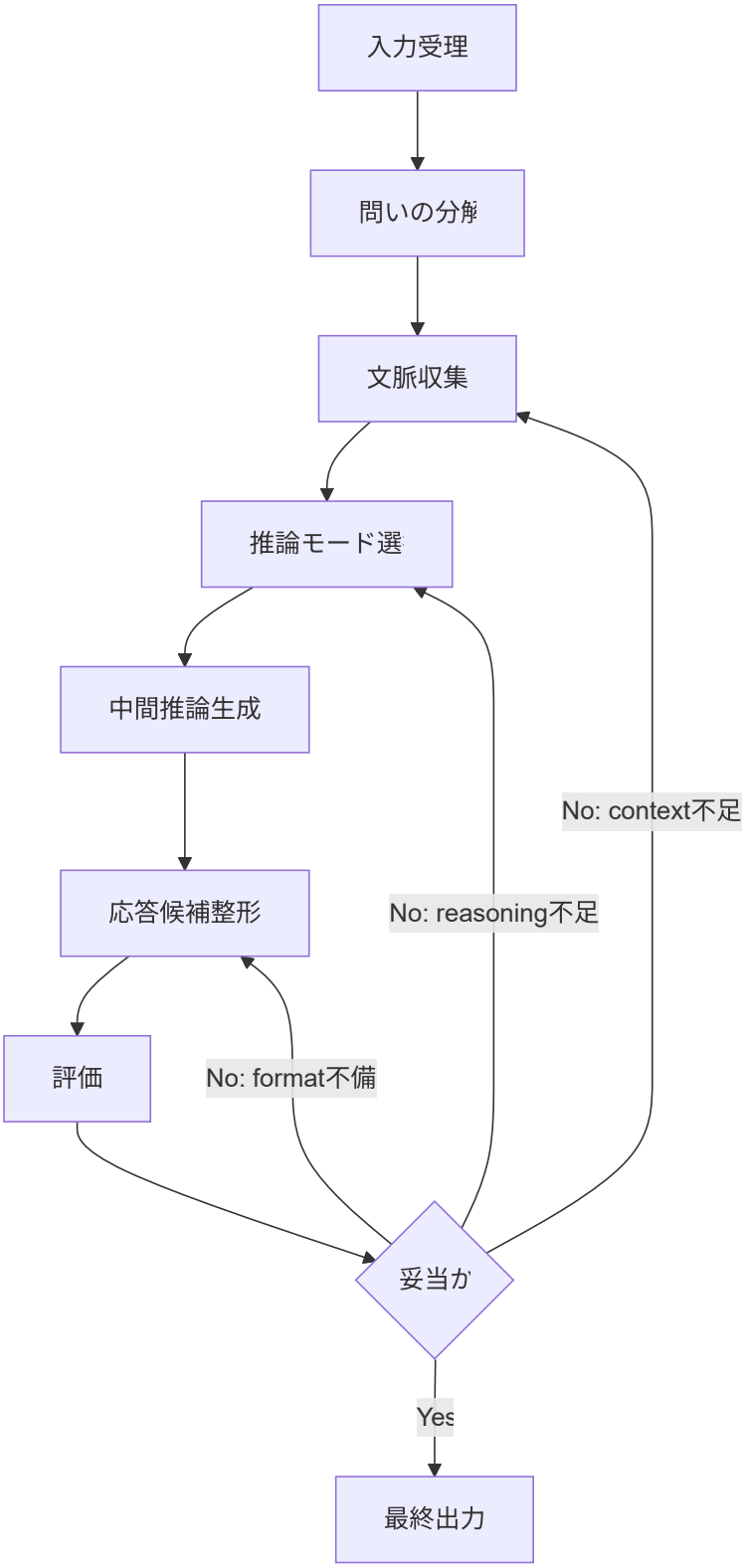
- どの推論モードを使うか
- どこまで詳細に答えるか
- 再検索が必要か
- 既存ノートに合わせるか
- 新規設計か整理・統合か
- 回答を一括で出すか分割するか

入る下位ノート

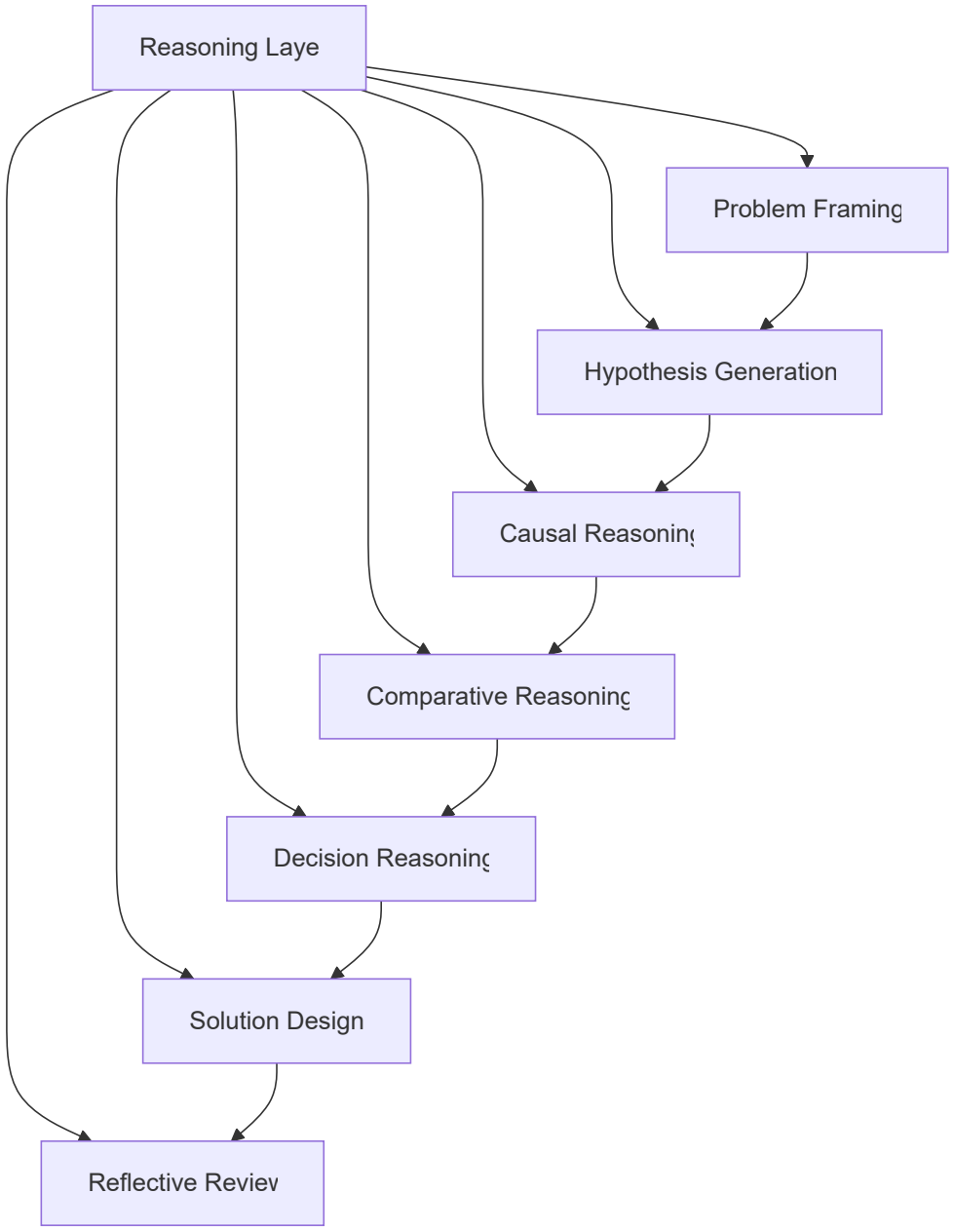
- LLM Control Layer
- Reasoning Orchestration
- Mode Switching
- Depth Control
- Retry Strategy
- Tool Use Policy

---

4. 中核フロー


入力受理
問いの分解
文脈収集
推論モード選
中間推論生成
応答候補整形
評価
妥当か
No: context不足
No: reasoning不足
No: format不備
Yes
最終出力

5. 推論層の内部構造



---

6. 実運用での読み方

このノートは「LLM の仕組み説明」で終わるものではない。
実際には次のように使う。

用途1: vault の上位設計

- 00_system に置く
- 各下位ノートへの入口にする
- system 全体の基準ノートにする

用途2: 自作AI設計の骨格

- Input/Context/Reasoning/Output を分離する
- RAG や Memory をどこに差し込むか決める
- 評価器や再試行器の設計に使う

用途3: プロンプト設計の基準

- いま不足しているのは入力定義か
- 文脈不足か
- 推論不足か
- 出力整形不足か
- 評価不足か
  を切り分ける

用途4: Zettelkasten との接続

- concept としては「推論アーキテクチャ」
- structure としては「入力→文脈→推論→出力→評価→制御構造」
- method としては「LLM運用設計法」
- tool としては「AIアシスタント実装骨格」
  として接続できる

---

7. 関連ノート

上位

- 00_system

下位

- LLM Input Layer
- LLM Context Layer
- LLM Reasoning Layer
- LLM Memory Layer
- LLM Output Layer

- LLM Control Layer
- LLM Evaluation Layer

関連

- Prompt Architecture
- Reasoning Engine
- Context Engineering
- Memory Architecture
- Evaluation Loop
- Tool-Augmented Reasoning
- Human-in-the-Loop Reasoning

---

8. 要約

LLM Reasoning Architecture とは、
LLM を単なる応答生成器ではなく、**入力処理・文脈構成・推論・記憶参照・出力整形・評価・制御**を備えた思考システムとして整理するための全体構造である。

中心は Reasoning Layer だが、実運用で安定性を決めるのはむしろ以下である。

- Input Layer の問いの切り方
- Context Layer の文脈構成
- Memory Layer の知識参照
- Evaluation Layer の検査
- Control Layer の再推論制御

したがって、LLM を強くするとは、モデル本体だけでなく、
その周辺構造を設計することでもある。

LLM Input Layer

3. 構成要素

3-1. Input Layer

Input Layer は、ユーザー入力をそのまま受け取るのではなく、
問いとして整形可能な単位へ分解する層である。

主な役割

- 指示の抽出
- 質問の型判定
- 対象の特定
- 制約条件の抽出
- 出力形式の抽出
- 暗黙要件の補完

典型的な処理

- 何をしてほしいか
- 対象は何か
- どこまで深くやるか
- コードブロックが必要か
- 既存の vault 構造に合わせる必要があるか

入る下位ノート

- LLM Input Layer
- Task Parsing
- Intent Detection
- Constraint Extraction
- Output Format Recognition

3-2. Context Layer

Context Layer は、問いを理解するために必要な文脈を構成する層である。

LLM は与えられた文字列しか見えないため、
実用上はこの層で**適切な前提・関連ノート・履歴・定義**を集める必要がある。

主な役割

- 会話履歴参照
- 関連ノート選択
- 定義の参照
- 前提条件の確定
- 文脈の圧縮
- ノイズ除去

文脈の種類

- 会話文脈
- タスク文脈
- ドメイン文脈
- ユーザー固有文脈
- システム制約文脈
- 外部データ文脈

入る下位ノート

- LLM Context Layer
- Context Assembly
- Context Compression
- Relevant Context Selection
- Prompt State Management

3-3. Reasoning Layer

Reasoning Layer は、このアーキテクチャの中心である。
ここでは文脈をもとに、問いに対する中間的思考を形成する。

この層は単一ではなく、複数の推論モードを持つ。

主な推論モード

- 演繹推論
- 帰納推論
- 仮説推論
- 因果推論
- 比較推論
- 構造推論
- 診断推論
- 設計推論

- 反証推論
- 分岐推論

役割

- 問題の再定義
- 論点の分解
- 仮説生成
- 選択肢設計
- 因果連鎖の整理
- 抽象化と具体化の往復
- 結論候補の生成

入る下位ノート

- LLM Reasoning Layer
- Causal Reasoning
- Hypothesis Generation
- Comparative Reasoning
- Decision Reasoning
- Solution Design Reasoning
- Abductive Reasoning
- Reflective Reasoning

3-4. Memory Layer

Memory Layer は、現在の推論に必要な知識や過去情報を供給する層である。

LLM 単体の内部知識だけに依存すると、
更新不能・曖昧・再利用困難になるため、外部記憶との接続が重要になる。

主な役割

- 長期知識の保持
- ノート・DB 参照
- 過去会話の再利用
- ユーザー固有設定の保持
- 定義と用語体系の維持
- 推論資源の再利用

記憶の種類

- Semantic Memory
- Episodic Memory
- Procedural Memory
- Working Memory
- Externalized Memory

入る下位ノート

- LLM Memory Layer
- Semantic Memory
- Working Memory
- External Knowledge Interface
- User Preference Memory
- Reasoning State Memory

3-5. Output Layer

Output Layer は、推論結果を人間が使える形へ整形する層である。

ここでは「何を答えるか」だけでなく、
どう並べるか・どの粒度で見せるか・どの記法で出すかが重要になる。

主な役割

- 結論の明示
- 論理展開の整形
- コードブロック化
- Markdown 化
- 表や箇条書きへの整形
- ノート形式への変換
- Mermaid 図への変換

出力形式の例

- 通常回答
- Obsidian ノート
- Hub ノート
- Structure ノート
- Mermaid 図
- テーブル
- テンプレート

- 手順書

入る下位ノート

- LLM Output Layer
- Response Structuring
- Markdown Rendering
- Note Generation
- Mermaid Generation
- Format Adaptation

3-6. Evaluation Layer

Evaluation Layer は、出力候補が妥当かどうかを検査する層である。
ここがないと、LLM は「それらしく見える誤答」をそのまま出しやすい。

主な役割

- 一貫性検査
- 制約適合性検査
- 網羅性検査
- 論理飛躍の検査
- 形式不備の検査
- 事実性の検査
- ユーザー要求との整合確認

典型的な観点

- 質問に答えているか
- 指示形式を守っているか
- 前提がずれていないか
- Mermaid が閉じているか
- 必要なノート名が揃っているか
- 構造が重複していないか

入る下位ノート

- LLM Evaluation Layer
- Consistency Check
- Constraint Check
- Coverage Check

- Format Validation
- [[Reasoning Quality Review]

3-7. Control Layer

Control Layer は、全体の流れを管理する層である。
いわば**メタ思考・実行制御・再試行判断**を担う。

主な役割

- 推論深度の調整
- モード切替
- 再推論判断
- 打ち切り判断
- ツール利用判断
- 文脈再収集判断
- 出力粒度の制御

制御対象

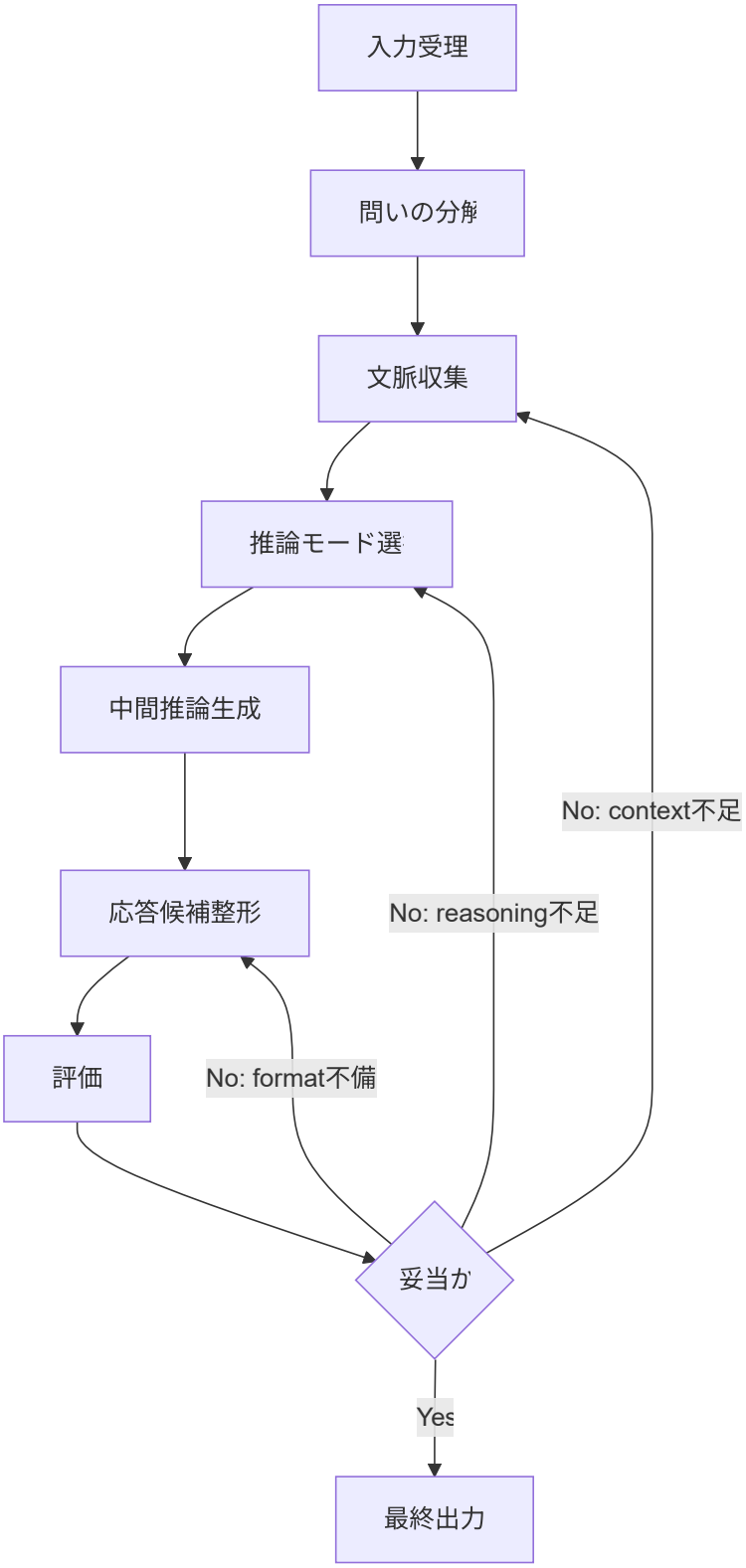
- どの推論モードを使うか
- どこまで詳細に答えるか
- 再検索が必要か
- 既存ノートに合わせるか
- 新規設計か整理・統合か
- 回答を一括で出すか分割するか

入る下位ノート

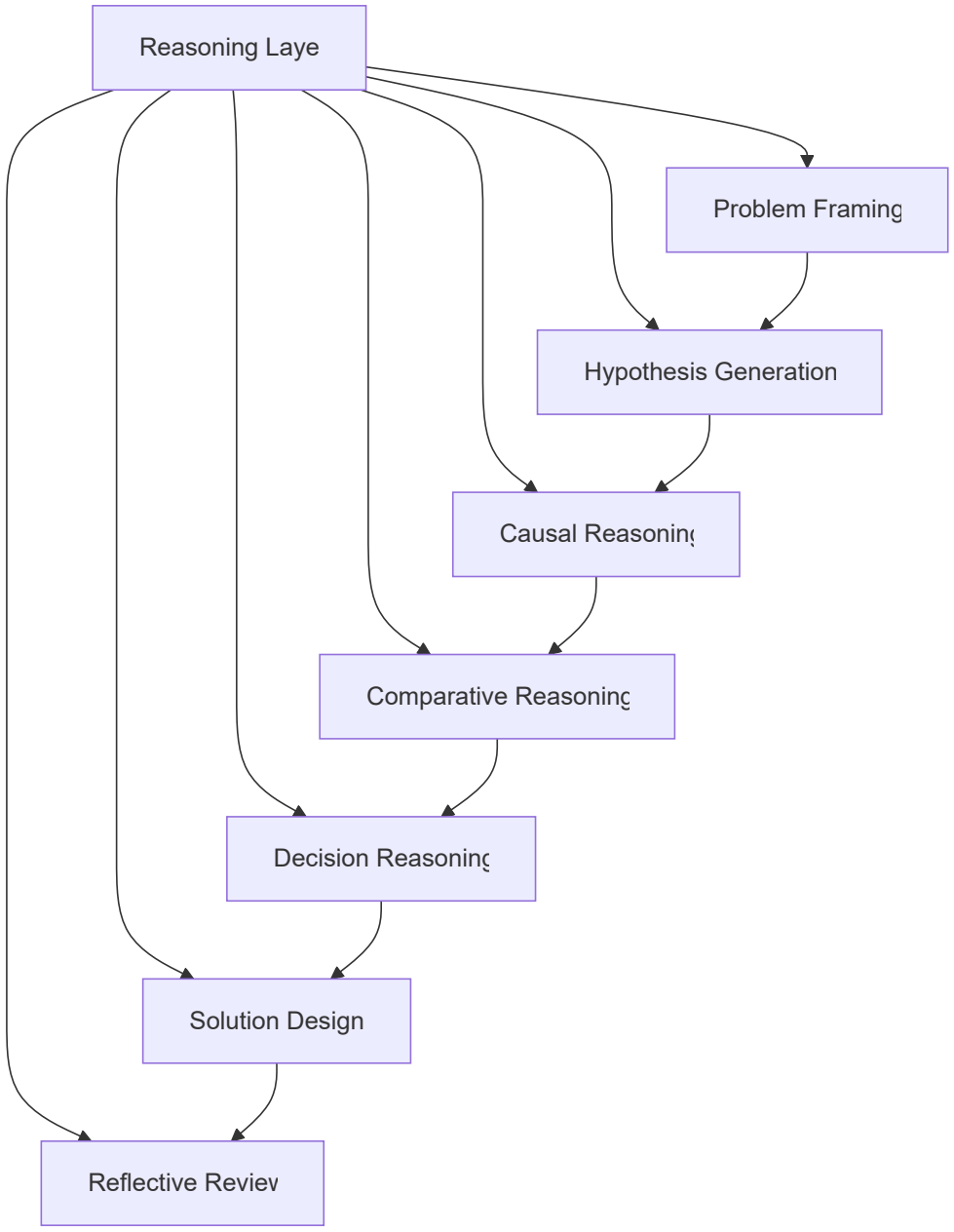
- LLM Control Layer
- Reasoning Orchestration
- Mode Switching
- Depth Control
- Retry Strategy
- Tool Use Policy

---

4. 中核フロー


入力受理
問いの分解
文脈収集
推論モード選
中間推論生成
応答候補整形
評価
妥当か
No: context不足
No: reasoning不足
No: format不備
Yes
最終出力

5. 推論層の内部構造



---

6. 実運用での読み方

このノートは「LLM の仕組み説明」で終わるものではない。
実際には次のように使う。

用途1: vault の上位設計

- 00_system に置く
- 各下位ノートへの入口にする
- system 全体の基準ノートにする

用途2: 自作AI設計の骨格

- Input / Context / Reasoning / Output を分離する
- RAG や Memory をどこに差し込むか決める
- 評価器や再試行器の設計に使う

用途3: プロンプト設計の基準

- いま不足しているのは入力定義か
- 文脈不足か
- 推論不足か
- 出力整形不足か
- 評価不足か
  を切り分ける

用途4: Zettelkasten との接続

- concept としては「推論アーキテクチャ」
- structure としては「入力→文脈→推論→出力→評価→制御構造」
- method としては「LLM運用設計法」
- tool としては「AIアシスタント実装骨格」
  -して接続できる

---

7. 関連ノート

上位

- 00_system

下位

- LLM Input Layer
- LLM Context Layer
- LLM Reasoning Layer
- LLM Memory Layer
- LLM Output Layer

- LLM Control Layer
- LLM Evaluation Layer

関連

- Prompt Architecture
- Reasoning Engine
- Context Engineering
- Memory Architecture
- Evaluation Loop
- Tool-Augmented Reasoning
- Human-in-the-Loop Reasoning

---

8. 要約

LLM Reasoning Architecture とは、
LLM を単なる応答生成器ではなく、**入力処理・文脈構成・推論・記憶参照・出力整形・評価・制御**を備えた思考システムとして整理するための全体構造である。

中心は Reasoning Layer だが、実運用で安定性を決めるのはむしろ以下である。

- Input Layer の問いの切り方
- Context Layer の文脈構成
- Memory Layer の知識参照
- Evaluation Layer の検査
- Control Layer の再推論制御

したがって、LLM を強くするとは、モデル本体だけでなく、
その周辺構造を設計することでもある。

LLM Context Layer

LLM Context Layer

LLM Context Layer は、推論に必要な **情報環境（Context）を構築する層**である。

LLMは入力されたテキストしか見えないため、
適切な推論を行うには **必要な情報をまとめて提示する必要がある**。

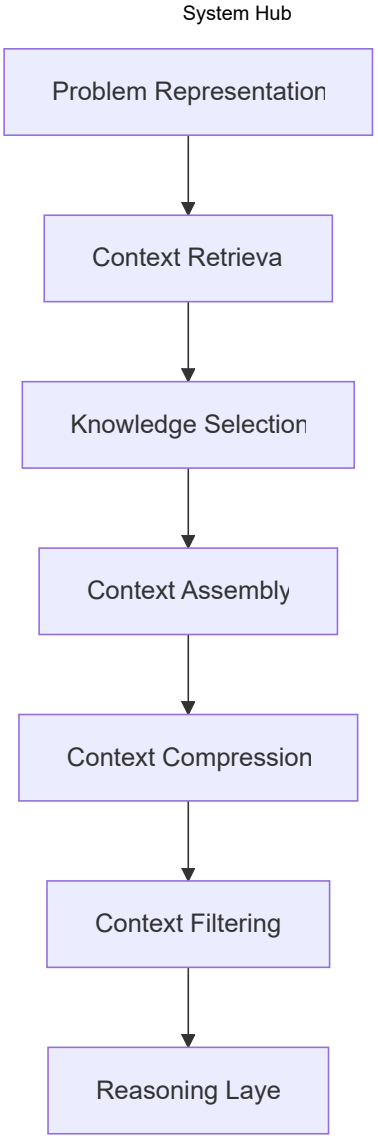
この層では次の処理を行う。

- 関連情報の検索
- 知識の参照
- 文脈の整理
- 情報の圧縮
- ノイズ除去

Input Layer が **問題を作る層**なら、
Context Layer は **思考環境を作る層**である。

---

1 全体構造



---

2 Context の種類

Context は一種類ではない。

主に次の5種類がある。

種類	内容
Conversation Context	会話履歴
Task Context	現在のタスク
Knowledge Context	知識

種類	内容
User Context	ユーザー固有情報
System Context	システム制約

---

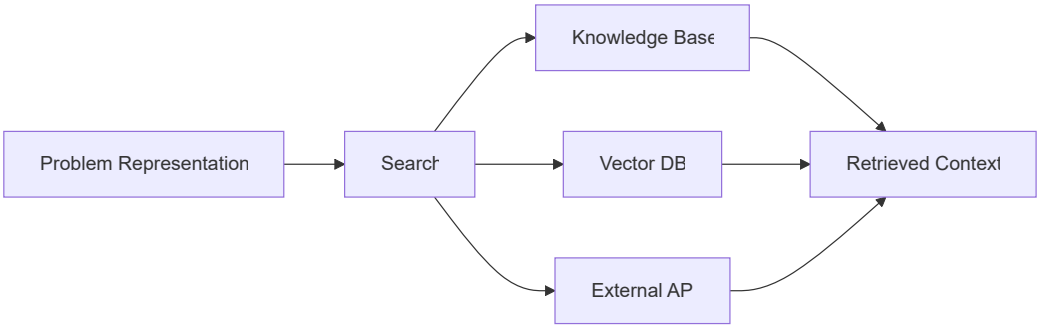
3 Context Retrieval

Context Retrieval は
推論に必要な情報を取得する処理である。

情報源

- 会話履歴
- vaultノート
- 知識ベース
- 外部検索
- API
- DB

構造



---

4 Knowledge Selection

取得された情報のすべてが必要とは限らない。

Knowledge Selection は
関連性の高い情報だけを選択する処理である。

選択基準

- relevance
- novelty
- importance
- reliability
- recency

例

質問

- 人格OSを作る

取得情報

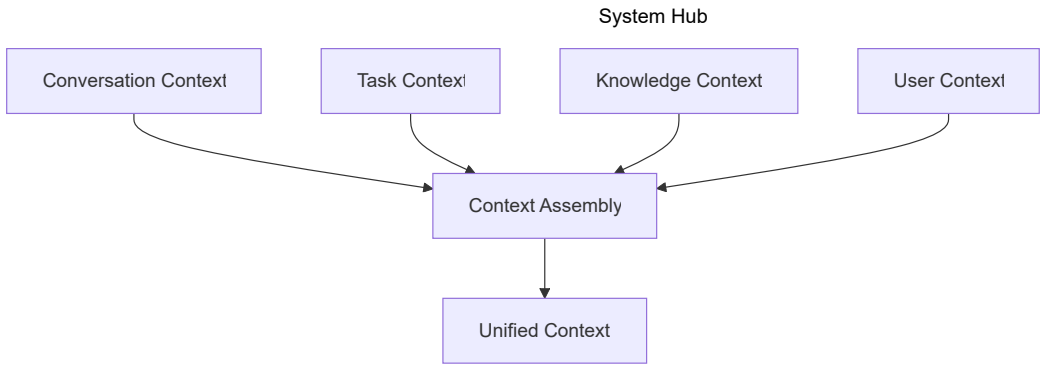
- 心理モデル
- 意思決定モデル
- 学習モデル
- 人格理論

---

5 Context Assembly

Context Assembly は
選択された情報を1つの推論環境へ統合する処理である。

構造



---

6 Context Compression

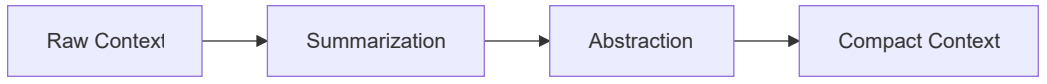
LLMには **コンテキスト長の制限**がある。

そのため Context Compression が必要になる。

圧縮方法

- 要約
- 抽象化
- 情報選択
- 冗長削除
- 構造化

構造



---

7 Context Filtering

Context Filtering は
推論の邪魔になる情報を除去する処理である。

除去対象

- irrelevant information
- outdated knowledge
- conflicting data
- noise

---

8 Context の最終形

Reasoning Layer に渡される Context は次の構造になる。

context:

problem
knowledge
constraints
definitions
examples
relevant_cases

---

9 Context Layer が弱いと起こる問題

問題1

知識不足

LLM hallucination

問題2

ノイズ過多

推論が拡散

問題3

情報不足

推論停止

---

10 Reasoning Layer への入力

Context Layer の最終出力

reasoning_input:

problem_definition
relevant_knowledge
constraints
context_information

これが **Reasoning Layer の入力**になる。

---

11 関連ノート

上位

- LLM Reasoning Architecture

前

- LLM Input Layer

次

- LLM Reasoning Layer

関連

- RAG Architecture
- Context Engineering
- Knowledge Retrieval

LLM Output Layer

LLM Output Layer

LLM Output Layer は、内部で生成された推論・判断・設計結果を、外部に提示可能な形へ変換する層である。
この層は「考えたことをそのまま出す」のではなく、**目的に応じて整形・圧縮・翻訳・安全化し、ユーザーや下位システムに受け渡す**役割を担う。

要点

- 推論結果をそのまま露出させず、**出力可能な表現**へ変換する
- 内容を、**正確性・可読性・安全性・形式適合性**の観点で整える
- 出力先に応じて、文章・表・箇条書き・JSON・コード・命令列などへ再構成する
- ユーザー向け出力と、ツール向け出力と、内部記録向け出力を分ける
- 「何を答えるか」だけでなく、**どの粒度・どの順序・どのトーンで出すか**を決める

この層が必要な理由

LLM の内部では、観察・検索・仮説・比較・評価・選択などが複合的に進む。
しかし、その内部状態はそのままでは冗長で不安定であり、また相手の要求形式にも合わない。

たとえば内部では以下が混在する。

- 未確定の仮説
- 途中計算
- 分岐した比較案
- 参照した根拠
- 安全上そのままは出せない記述
- ユーザーが求めていない補助推論

そのため、最終提示の直前に、

1. 残すべき内容を選び、
2. 順序を整え、
3. 表現形式へ落とし込み、
4. 不要・危険・冗長なものを除く

という変換層が必要になる。
それが LLM Output Layer である。

中核機能

1. Content Selection

内部で得られた候補のうち、何を最終出力に載せるかを決める。

対象:

- 結論
- 根拠
- 但し書き
- 不確実性
- 次の行動
- 補足情報

判断基準:

- ユーザー要求との適合
- タスク達成への寄与
- 認知負荷
- 安全性
- 冗長性

---

2. Structural Composition

選ばれた内容を、伝わる順序に並べる。

典型順序:

1. 結論
2. 根拠
3. 補足
4. 注意点
5. 次アクション

ただし、用途によって順序は変わる。

- 説明タスク: 定義 → 分解 → 具体例 → 注意点

- 提案タスク: 結論 → 比較 → 推奨案 → 実行手順
- デバッグ: 症状 → 原因候補 → 切り分け → 修正案
- 生成物: 完成物 → 使い方 → 変更点

---

3. Format Realization

構造化済み内容を、具体的な出力形式へ変換する。

代表形式:

- 通常文章
- Markdown
- 箇条書き
- 表
- JSON
- YAML
- コード
- Mermaid
- tool call payload
- schema-constrained output

この変換では、**意味の正確さ**と**形式妥当性**の両方が必要となる。

---

4. Style and Tone Adaptation

同じ内容でも、受け手や用途に応じて言い方を変える。

調整対象:

- 丁寧さ
- 専門性
- 簡潔さ
- 断定度
- 抽象度
- 説明密度

例:

- 初学者向け: 用語説明を入れる
- 実務者向け: 先に結論と手順を出す

- API向け: 人間向け説明を削り schema 優先にする

---

5. Safety and Policy Filtering

出力内容が安全・規約・公開範囲の条件を満たすよう調整する。

対象:

- 危険情報の制限
- 個人情報の除去
- 内部思考の露出抑制
- 根拠不十分な断定の緩和
- 不適切表現の抑制

この機能は、単なる削除ではなく、**安全な代替表現への置換**も含む。

---

6. Confidence Framing

不確実な内容を、確実な内容と同じ調子で出さないようにする。

方法:

- 確定的事実
- 高確率推定
- 仮説
- 未検証案
- 要確認項目

を区別し、表現を変える。

例:

- 「〜です」
- 「〜の可能性が高いです」
- 「現時点では仮説です」
- 「確認が必要です」

---

7. Delivery Routing

整形した出力を、どこへ送るかを決める。

出力先:

- ユーザー表示
- ツール呼び出し
- ファイル生成
- メモリ保存候補
- 内部ログ
- 他モジュールへの受け渡し

つまり Output Layer は、**書く層**であると同時に**配る層**でもある。

下位構造

A. Answer Constructor

最終回答本文を作る部分。

役割:

- 問いへの直接回答
- 根拠の簡潔な統合
- 補足と注意点の配置
- 体裁の最終調整

B. Evidence Presenter

根拠・出典・引用・観測結果を提示可能な形にする部分。

役割:

- 根拠の要約
- 出典対応付け
- どの主張が何に支えられているかの明示
- 過剰引用の抑制

C. Format Emitter

特定フォーマットへ厳密に落とし込む部分。

対象:

- JSON 出力
- CSV 断片
- Markdown テンプレート
- Mermaid
- コードファイル
- 表形式

D. Safety Rewriter

危険・過剰・不適切な表現を、安全な形に再記述する部分。

役割:

- 直接手順の抽象化
- 個人情報の伏せ字化
- 有害意図への拒否文生成
- 安全代替案の追加

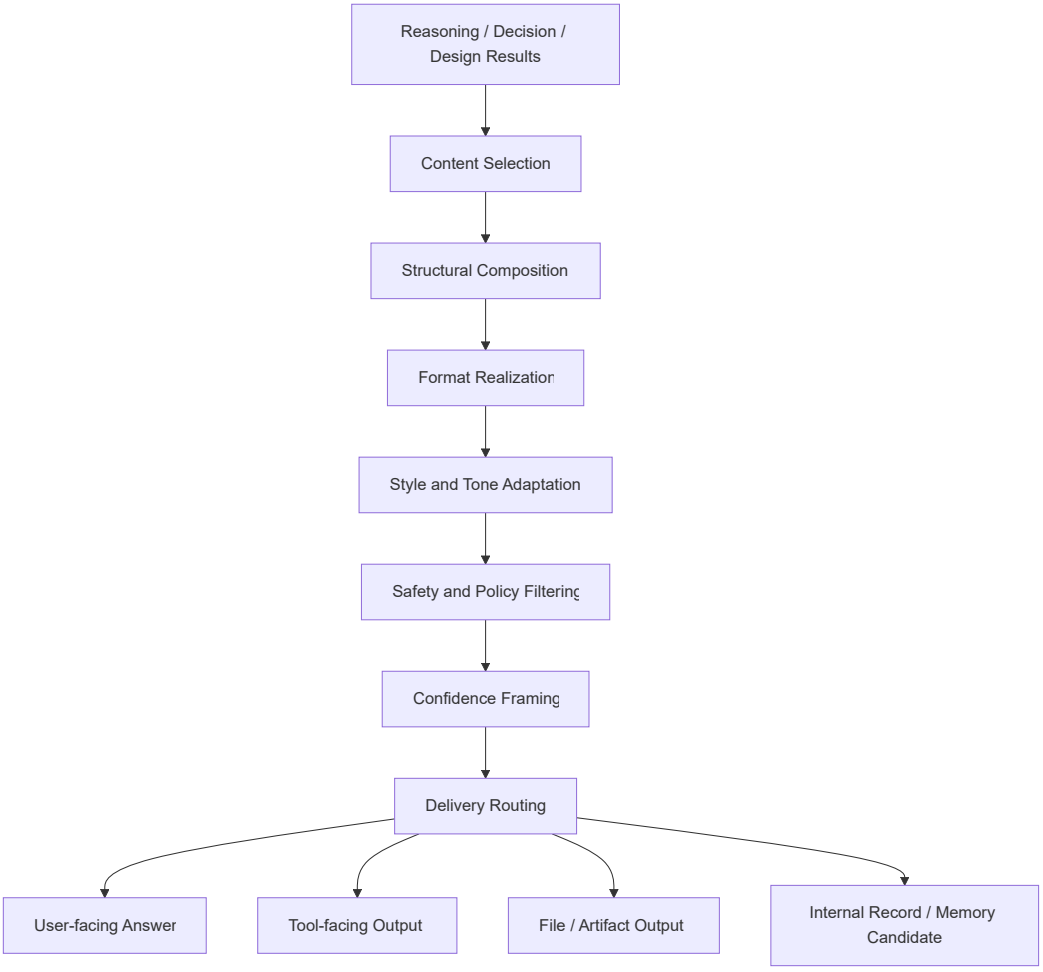
E. Response Compressor

冗長な内部結果を、要求に見合う長さへ圧縮する部分。

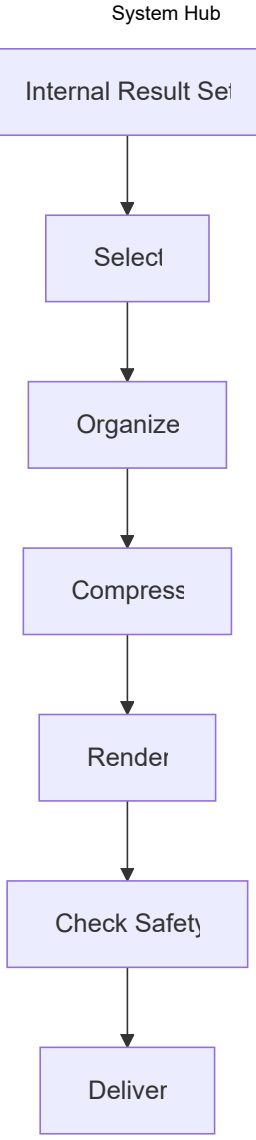
役割:

- 要約
- 重複削除
- 低優先情報の削減
- 長文から短文への縮約

全体構造


Reasoning / Decision / Design Results
Content Selection
Structural Composition
Format Realization
Style and Tone Adaptation
Safety and Policy Filtering
Confidence Framing
Delivery Routing
User-facing Answer
Tool-facing Output
File / Artifact Output
Internal Record / Memory Candidate

出力変換の基本フロー



入出力の対応

入力状態	Output Layer の処理	出力形
推論が複雑で長い	要点抽出と順序再構成	簡潔な説明文
比較案が複数ある	比較軸で整列	比較表 / 推奨文
不確実性が高い	断定度調整	仮説的回答
根拠が多数ある	根拠を束ねる	引用付き要約
strict format が必要	schema 準拠整形	JSON / YAML / code
危険内容を含む	再記述・制限	安全化された応答

他層との関係

LLM Control Layer との関係

Control Layer は、

- 何をするか
- どのモードで動くか
- どのツールを使うか
- どこで止めるか

を決める。

Output Layer は、

- 得られた結果をどう出すか
- どの形式で渡すか
- どの程度圧縮するか

を決める。

つまり、

- Control Layer = 実行統制
- Output Layer = 表出統制

である。

Reasoning Layer との関係

Reasoning Layer が意味内容を生成し、
Output Layer がその意味内容を外部向け表現へ変換する。

Reasoning が弱ければ Output は空疎になり、
Output が弱ければ Reasoning は伝達不能になる。

Memory / Record Layer との関係

最終出力と内部保存内容は一致しないことがある。
Output Layer は、

- ユーザーへ見せる版
- 将来参照用に残す版

を分けうる。

よくある失敗

1. 内部推論の垂れ流し

思考途中の内容をそのまま出して冗長になる。

2. 結論が遅い

前置きが長く、ユーザーが欲しい答えが後ろに埋もれる。

3. 形式不一致

JSON を求められているのに説明文を混ぜるなど、出力仕様を破る。

4. 不確実性の隠蔽

推定や仮説を事実のように出す。

5. 安全化しすぎ

有用性まで失って、空疎な一般論になる。

6. ユーザー粒度との不一致

初学者に専門家向け説明を出す、またはその逆。

設計原則

- まず答える
- 必要な根拠だけ添える
- 形式要件を優先する
- 不確実性を明示する
- 危険情報は安全化して代替する
- 出力先ごとに別物として設計する
- 内部の豊かさと外部の簡潔さを両立する

位置づけ

LLM Output Layer は、
推論結果を人間・ツール・記録へ橋渡しする最終変換層である。

この層が弱いと、

- 良い推論も伝わらず
- 良い判断も使われず
- 良い設計も実装へつながらない

したがってこの層は、単なる「文章整形」ではなく、
意味を成果物へ変換する実用化層として理解すべきである。

関連ノート

- LLM Control Layer
- Reasoning Layer
- Decision Layer
- Solution Design Layer
- Evidence Integration
- Response Formatting
- Safety Filter
- Confidence Calibration

LLM Reasoning Layer

LLM Reasoning Layer

LLM Reasoning Layer は、Context Layer で準備された情報を使い、
実際の推論を行う中核層である。

この層では次の処理が行われる。

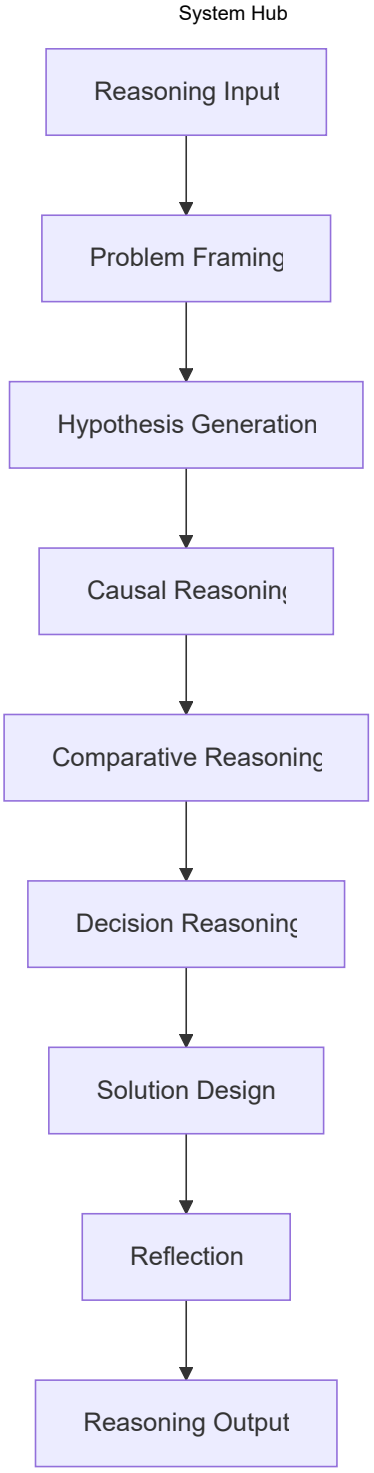
- 問題の再定義
- 仮説生成
- 因果推論
- 比較検討
- 意思決定
- 解決策設計
- 自己検証

Input Layer が **問題を作る層**、
Context Layer が **思考環境を作る層**なら、

Reasoning Layer は **思考そのものを行う層**である。

---

1 全体構造



2 Problem Framing

Problem Framing は
問題の構造を明確化する処理である。

多くの場合、入力された問題は曖昧であるため、
推論前に問題を整理する必要がある。

例

入力

観光地評価OSを作りたい

Problem Framing

goal: 観光地評価システム設計
domain: tourism
object: 観光資源
evaluation_method: structure

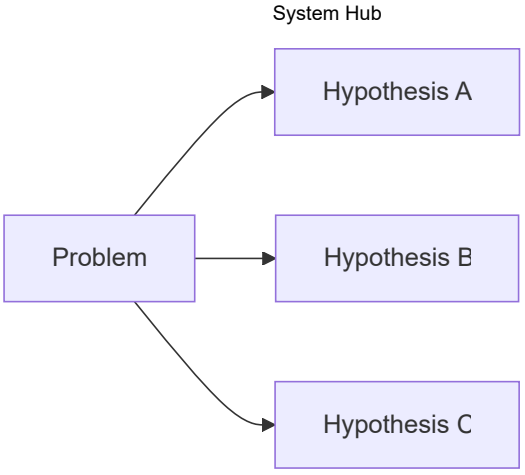
---

3 Hypothesis Generation

Hypothesis Generation は
仮説を生成する処理である。

推論では1つの答えを直接求めるのではなく、
複数の仮説を作ることで思考の探索空間を広げる。

構造



---

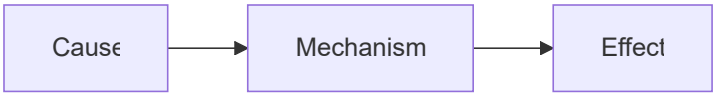
4 Causal Reasoning

Causal Reasoning は
因果関係を推論する処理である。

例

原因 → 結果

構造



---

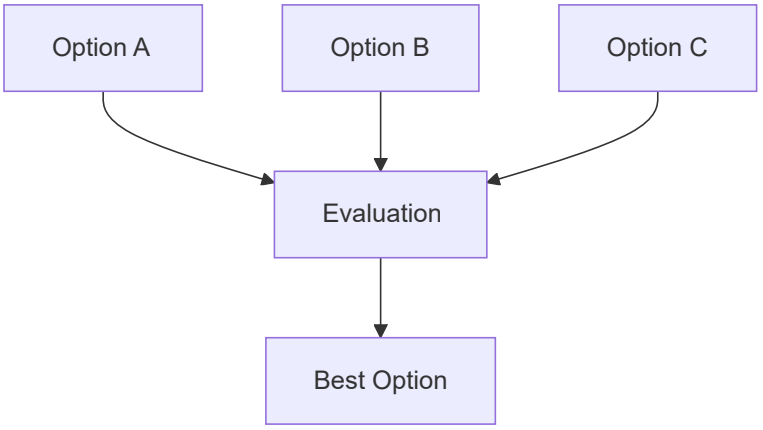
5 Comparative Reasoning

Comparative Reasoning は
複数の選択肢を比較する推論である。

比較軸

- cost
- risk
- efficiency
- scalability
- feasibility

構造



---

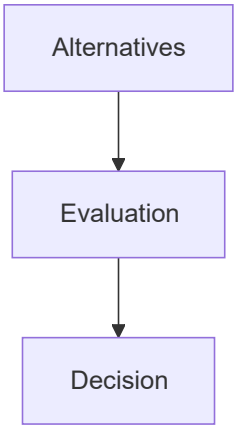
6 Decision Reasoning

Decision Reasoning は
比較結果をもとに意思決定する処理である。

決定基準

- expected value
- risk
- constraints
- goal alignment

構造

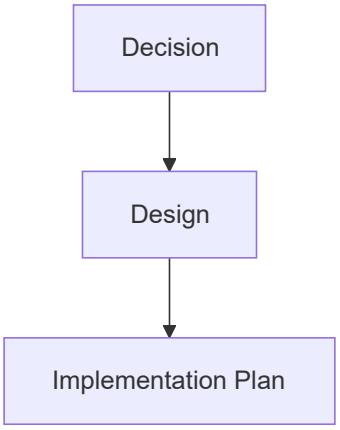


---

7 Solution Design

Solution Design は
実際の解決策を設計する処理である。

構造



---

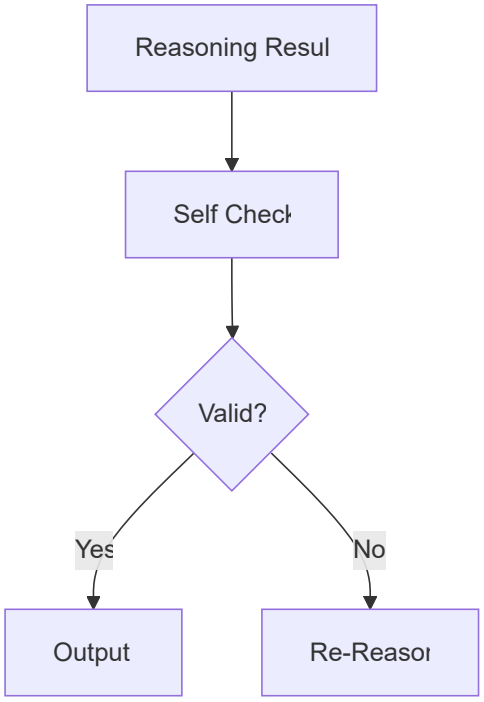
8 Reflection

Reflection は
推論結果を自己検証する処理である。

検証内容

- 論理整合性
- 制約違反
- 不完全推論
- 誤前提

構造



---

9 Reasoning Output

Reasoning Layer の出力は次の構造になる。

reasoning_result:

problem_definition
hypotheses
causal_analysis
comparative_evaluation
decision
solution_design

これが **Output Layer** に渡される。

---

10 Reasoning Layer の特徴

Reasoning Layer は **単一の推論ではなく複数推論の連鎖**である。

problem framing
↓
hypothesis
↓
causal reasoning
↓
comparison
↓
decision
↓
solution design

---

11 関連ノート

上位

- LLM Reasoning Architecture

前

- LLM Context Layer

次

- LLM Output Layer

関連

- Causal Reasoning
- Hypothesis Generation
- Decision Reasoning
- Solution Design

LLM Control Layer

LLM Control Layer

LLM Control Layer は、LLM システム全体の処理進行を統制する層である。
この層は、入力を受けたあとに、

- 何を目標とするか
- どの処理モードを使うか
- どの順序で進めるか
- どのツールや知識源を呼ぶか
- どこで止めるか
- 何を出力段階へ渡すか

を決定する。

言い換えると、Control Layer は **「考える前に進行を設計し、考えている途中も流れを監督する層」** である。

要点

- 入力を見て、タスクの種類を判定する
- 必要なモジュールだけを起動する
- 深さ・順序・ツール使用・反復回数を制御する
- 制約時間・安全性・形式要求を監視する
- 推論を無限に伸ばさず、十分条件で打ち切る
- 最終的に Output Layer へ渡す内容を確定する

この層が必要な理由

LLM は、入力さえあれば自動で何かしらの文章を生成できる。
しかし、実務的に有用なシステムにするには、単純な逐語生成では足りない。

なぜなら実際のタスクは以下のように多様だからである。

- 一問一答
- 比較検討
- 調査
- 要約

- 設計
- コード生成
- ツール利用
- ファイル作成
- 拒否と代替提案
- 段階的な問題解決

これらを同じ流れで処理すると、

- 必要ない深掘りをする
- 必要な検索をしない
- ツールを呼ぶべき場面で呼ばない
- 出力形式を誤る
- 安全制約を見落とす
- 長すぎるか浅すぎる応答になる

そこで、処理全体の交通整理を行う制御層が必要になる。
それが LLM Control Layer である。

中核機能

1. Task Classification

入力がどの種類の仕事かを判定する。

代表分類:

- 質問応答
- 分析
- 比較
- 要約
- 設計
- 生成
- 実行支援
- 検索
- 編集
- スケジューリング
- 拒否判定付き案件

この分類により、後続の推論・検索・出力モードが決まる。

---

2. Goal Framing

ユーザーの表層的な言葉を、その奥にある処理目標へ変換する。

例:

- 「これどう思う？」
  → 評価・論点整理・不確実性提示
- 「作ってください」
  → 成果物生成・テンプレート化・貼り付け可能整形
- 「調べてください」
  → 情報検索・根拠取得・比較整理

つまり Control Layer は、発話を**実行可能な目的記述**へ変換する。

---

3. Mode Selection

どの認知モードで処理するかを決める。

代表モード:

- Direct Answer Mode
- Search-Augmented Mode
- File-Grounded Mode
- Stepwise Reasoning Mode
- Comparative Analysis Mode
- Drafting Mode
- Tool Execution Mode
- Safety Refusal Mode

同じ質問でも、モードが違えば処理は大きく変わる。

---

4. Resource Routing

必要な知識源・ツール・記憶へ処理を振り分ける。

対象:

- 内部知識
- Web 検索
- ファイル検索

- カレンダー
- メール
- 連絡先
- Python
- ドキュメント生成
- 画像生成

この機能により、LLM は単体生成器ではなく、**多資源統合オーケストレータ**として働ける。

---

5. Process Sequencing

処理の順序を決める。

例:

1. 意図理解
2. タスク分類
3. 制約確認
4. 追加情報取得
5. 推論
6. 出力整形

あるいは

1. ファイルを読む
2. 必要箇所を抽出
3. 比較する
4. まとめる

Control Layer は、処理の順番を誤らないようにする。

---

6. Budget Management

時間・トークン・注意資源・応答長などの上限を管理する。

管理対象:

- どこまで深掘りするか
- 何件まで検索するか
- 何段階まで比較するか

- どこで打ち切るか
- 長文にするか簡潔にするか

これは、実用上きわめて重要である。
良いシステムは「どこまでやるか」も設計している。

---

7. Termination Control

十分な根拠や成果が得られた時点で、処理を止める。

停止条件の例:

- ユーザー要求を満たした
- 必要情報がそろった
- 追加探索の利得が小さい
- 安全上これ以上進めない
- 指定形式の成果物が完成した

Control Layer が弱いと、推論が冗長化しやすい。

---

8. Constraint Enforcement

各種制約を、実行全体に対して強制する。

対象:

- 安全制約
- 形式制約
- ツール使用規則
- 最新性確認要件
- 引用要件
- ユーザー指示
- ファイル形式指定
- 言語指定

これは「あとで直す」のではなく、最初から進行全体に効かせる必要がある。

下位構造

A. Intent Interpreter

発話の意味と要求を読み取る部分。

役割:

- 明示要求の抽出
- 暗黙要求の推定
- 成果物要求の検出
- 形式指定の検出

B. Task Router

タスクタイプに応じて、適切な処理ラインへ振り分ける部分。

役割:

- QA か生成かの分岐
- 検索が必要かの判定
- ツールが必要かの判定
- 単発応答か多段処理かの判定

C. Tool Orchestrator

使用可能ツール群の起動順・利用条件を管理する部分。

役割:

- 適切なツール選択
- 入出力接続
- 結果再利用
- 不要なツール呼び出しの回避

D. Reasoning Scheduler

どの程度の推論を、どの順序で走らせるかを決める部分。

役割:

- 仮説先行か検索先行か

- 比較から入るか直答から入るか
- 分割統治するか一括処理するか

E. Constraint Monitor

安全・形式・最新性などの制約を監視する部分。

役割:

- web 必須条件の確認
- 引用必要条件の確認
- 禁止事項の検知
- 出力フォーマット違反の防止

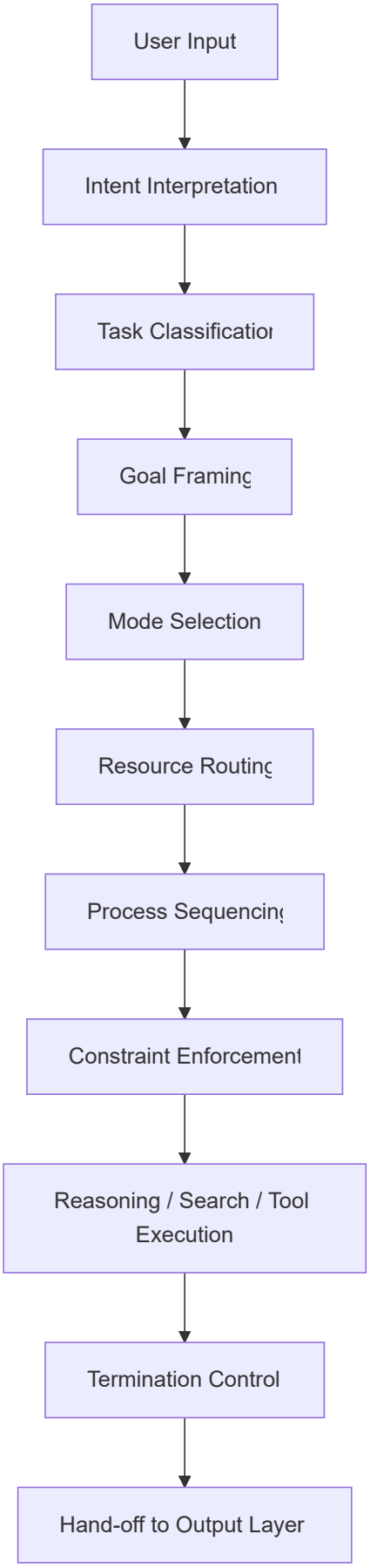
F. Stop Controller

いつ処理を終了し、出力段階へ渡すかを決める部分。

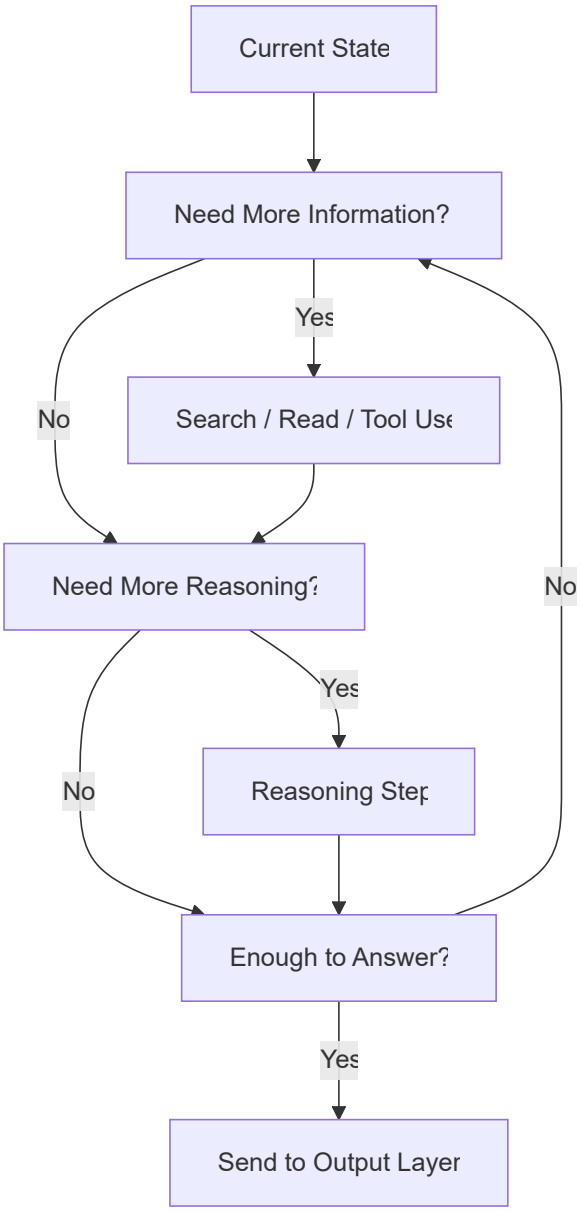
役割:

- 十分条件判定
- 探索打ち切り
- 残課題の明示
- 未達部分の説明

全体構造


User Input
Intent Interpretation
Task Classificatio
Goal Framing
Mode Selection
Resource Routing
Process Sequencing
Constraint Enforcement
Reasoning / Search / Tool Execution
Termination Control
Hand-off to Output Layer

制御ループ



Control Layer の代表判断

状況	Control Layer の判断
最新情報が必要	web 検索を先行させる
添付ファイルが主対象	file 読解を先行させる
単純質問	直答モードにする
複雑比較	比較モード + 構造化出力にする
形式が指定されている	format-first で処理する
安全リスクあり	拒否または安全代替へ切り替える
時間や長さ制約が厳しい	深掘りを制限し要点優先にする
成果物要求	出力テンプレート生成を優先する

典型サブモード

Direct Answer Mode

検索不要で、内部知識と軽い推論だけで返す。

Search-Augmented Mode

外部情報取得を前提に動く。

File-Grounded Mode

アップロードファイルや接続ソースを根拠にする。

Artifact Generation Mode

文書・表・コード・スライド・設計書などの完成物を作る。

Tool Execution Mode

メール送信、予定作成、分析実行など、ツール操作が中心。

Safety Refusal Mode

禁止領域に入るため、拒否 + 代替案提示へ切り替える。

他層との関係

Input Layer との関係

Input Layer が入力を受け取り正規化し、
Control Layer がそれを **処理計画** に変える。

Reasoning Layer との関係

Reasoning Layer は中身を考える。
Control Layer は、**いつ・どれだけ・どの順序で考えさせるか**を決める。

したがって、

- Reasoning は内容生成
- Control は進行統制

である。

Output Layer との関係

Control Layer は処理が終わった時点で、

- 何を渡すか
- どの形式要求を守るべきか
- どの程度確定しているか

を Output Layer に引き継ぐ。

Output Layer はその結果を、最終表現へ変換する。

よくある失敗

1. 直答してはいけないのに直答する

本当は検索が必要なのに、記憶だけで答える。

2. 検索しすぎる

安定知識なのに探索コストを無駄に払う。

3. ツール選択を誤る

ファイル検索すべき場面で web を使うなど。

4. 制約を後付けで処理する

最後に整形すればよいと思い、途中でルール違反が起こる。

5. 打ち切れない

十分な答えがあるのに、処理を延々続けて冗長化する。

6. ユーザーの真要求を誤る

説明が欲しいのに成果物を出す、またはその逆。

設計原則

- まずタスク種別を見極める
- 最新性・安全性・形式制約を先に確認する
- 不要な処理は起動しない
- 必要な外部資源は早めに取りに行く
- 深さより適合を優先する
- 十分条件で止める
- 出力層へ渡す前に状態を整理する

位置づけ

LLM Control Layer は、
LLM 全体の実行計画・進行管理・資源配分を担う統制中枢である。

この層が弱いと、

- 検索の要否を誤り
- 推論の深さを誤り
- ツール利用を誤り
- 出力形式も崩れやすくなる

したがって Control Layer は、単なる補助機能ではなく、
LLM を実用システムにするための運転席として理解すべきである。

関連ノート

- LLM Output Layer
- Input Layer
- Reasoning Layer
- Decision Layer
- Tool Orchestration
- Constraint Monitor
- Task Routing
- Termination Control

LLM System Hub

LLM System Hub

LLM System Hub は、LLM Reasoning Architecture における **system 層の中核ハブ**である。
このハブは、各層・各モジュール・各運転モードを一覧可能にし、**どの構造が何を担当し、どの順で接続され、どこで分岐し、どこで成果物へ至るか**を俯瞰するための入口となる。

system 層は、個別の推論内容そのものではなく、**推論を成立させる全体構造・統制機構・出力接続・運転モード群**を扱う。
したがってこのハブは、ノート群の単なる索引ではなく、**LLM 全体を運転席から見た構造地図**として機能する。

このハブの目的

- system 層に属するノート群の位置づけを整理する
- Control / Mode / Output / Tool の接続関係を明示する
- どのノートから読めばよいかの導線を与える
- 重複しやすい概念の境界を切る
- hub から各ノートへ自然に辿れる構造を作る

---

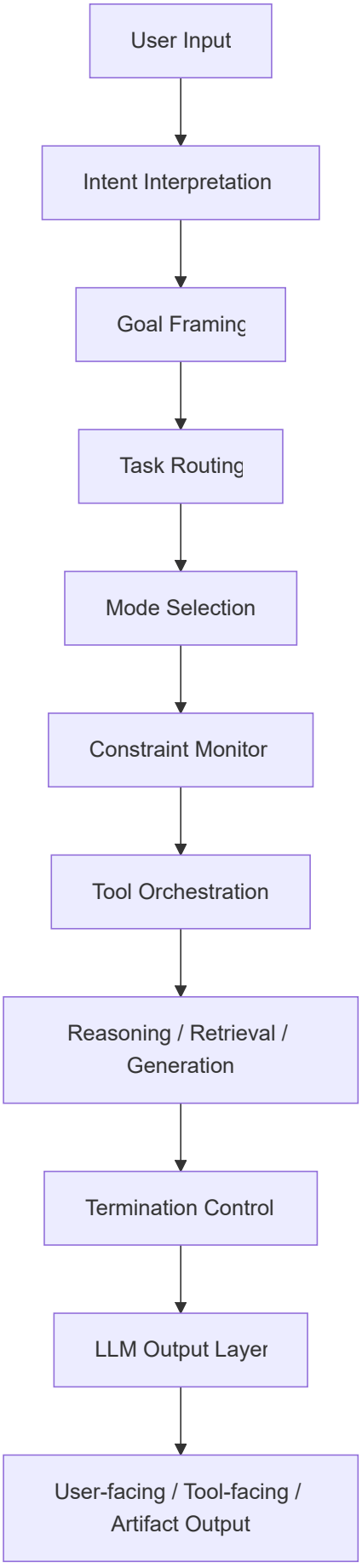
system 層の考え方

system 層は、主に次の問いに答える。

- LLM は入力をどう解釈するのか
- どのタスクとして扱うのか
- どのモードで運転するのか
- いつ検索し、いつファイルを読むのか
- いつ止め、何を出力するのか
- 外部ツールをどう編成するのか
- 制約や安全性をどこで監視するのか

つまり system 層は、**「何を知っているか」ではなく「どう動くか」** を扱う層である。

全体地図


User Input
Intent Interpretation
Goal Framing
Task Routing
Mode Selection
Constraint Monitor
Tool Orchestration
Reasoning / Retrieval / Generation
Termination Control
LLM Output Layer
User-facing / Tool-facing / Artifact Output

---

system 層の基本構造

1. Input Understanding

入力を読み、意図と目標を解釈する領域。

主ノート:

- Intent Interpretation
- Goal Framing

役割:

- 発話の背後にある要求を読む
- 主目的と副目的を分ける
- 成功条件を定義する
- 出力期待を先取りする

---

2. Control and Routing

依頼を処理ラインへ振り分ける領域。

主ノート:

- LLM Control Layer
- Task Routing
- Mode Selection

役割:

- タスク型を判定する
- 適切な処理ラインを選ぶ
- 運転モードを決める
- 深さ・速度・探索要否を調整する

---

3. Constraint and Safety Governance

処理全体に制約をかける領域。

主ノート:

- Constraint Monitor

役割:

- 安全制約の監視
- 形式制約の監視
- 最新性要件の監視
- 引用・権限・言語・範囲の監視
- 違反時の進路修正

---

4. External Action and Retrieval

外部ツールや外部資源を使う領域。

主ノート:

- Tool Orchestration
- Search-Augmented Mode
- File-Grounded Mode

役割:

- ツール要否の判定
- 外部検索の起動
- 文書根拠の取得
- 複数ツールの連鎖
- 外部結果の統合

---

5. Reasoning Modes

具体的な思考運転形態を切り替える領域。

主ノート:

- Direct Answer Mode
- Search-Augmented Mode

- File-Grounded Mode
- Comparative Reasoning Mode
- Stepwise Reasoning Mode
- Artifact Generation Mode

役割:

- 直答
- 検索拡張
- 文書根拠読解
- 比較整理
- 多段推論
- 成果物生成

---

6. Completion and Delivery

処理を終え、出力へつなぐ領域。

主ノート:

- Termination Control
- LLM Output Layer

役割:

- どこで止めるか決める
- 部分達成を扱う
- 出力形式に整える
- ユーザー向け・ツール向け・成果物向けへ分配する

---

system 層の主要ノート一覧

中核制御

- LLM Control Layer
- Task Routing
- Goal Framing
- Intent Interpretation
- Mode Selection

制約監視

- Constraint Monitor

外部実行・接続

- Tool Orchestration

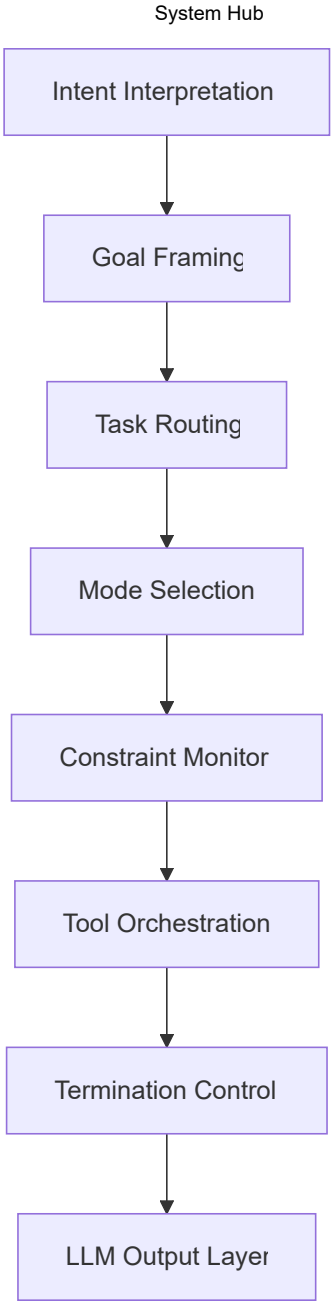
運転モード

- Direct Answer Mode
- Search-Augmented Mode
- File-Grounded Mode
- Comparative Reasoning Mode
- Stepwise Reasoning Mode
- Artifact Generation Mode

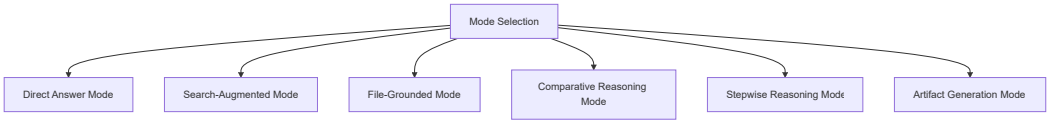
終了と出力

- Termination Control
- LLM Output Layer

依存関係マップ


System Hub
Intent Interpretation
Goal Framing
Task Routing
Mode Selection
Constraint Monitor
Tool Orchestration
Termination Control
LLM Output Layer

モード群マップ


Mode Selection
Direct Answer Mode
Search-Augmented Mode
File-Grounded Mode
Comparative Reasoning Mode
Stepwise Reasoning Mode
Artifact Generation Mode

system 層の読み順

入口から理解したい場合

1. Intent Interpretation
2. Goal Framing
3. Task Routing
4. Mode Selection
5. LLM Control Layer

この順で読むと、入力がどのように処理方針へ変換されるかが分かる。

運転モードを整理したい場合

1. Mode Selection
2. Direct Answer Mode
3. Search-Augmented Mode
4. File-Grounded Mode
5. Comparative Reasoning Mode
6. Stepwise Reasoning Mode
7. Artifact Generation Mode

この順で読むと、状況ごとの運転形態の違いが見える。

実務運用から理解したい場合

1. Constraint Monitor
2. Tool Orchestration
3. Termination Control
4. LLM Output Layer

この順で読むと、実際の運用・停止・成果物化の流れが見える。

---

system 層で区別すべき概念

Intent Interpretation と Goal Framing の違い

- Intent Interpretation は「何を求めているか」を読む

- Goal Framing は「それを何として達成するか」を定義する

Task Routing と Mode Selection の違い

- Task Routing は「どの処理ラインへ送るか」
- Mode Selection は「そのライン上でどう運転するか」

Constraint Monitor と Termination Control の違い

- Constraint Monitor は「越えてはいけない境界を監視する」
- Termination Control は「どこで十分とみなして止めるかを決める」

Tool Orchestration と Search-Augmented Mode の違い

- Tool Orchestration はツール利用全般の編成
- Search-Augmented Mode は検索を主軸とした運転形態

Artifact Generation Mode と LLM Output Layer の違い

- Artifact Generation Mode は成果物そのものを作るモード
- LLM Output Layer は最終出力全般を整形・配布する層

---

system 層の設計原則

- 入力理解と出力整形を分ける
- ルーティングとモード選択を分ける
- 制約監視を後付けにしない
- 外部接続は統制下で使う
- 停止条件を必ず持つ
- 回答と成果物を区別する
- hub から全体の流れが一望できるようにする

---

system 層の位置づけ

system 層は、
LLM が何を知っているかではなく、どう運転されるかを記述する統制層である。

この層が整っていると、

- 入力解釈が安定し
- タスク処理がぶれず
- モード切替が自然になり
- 制約違反が減り
- 出力品質が揃いやすくなる

したがってこの層は、個別ノート群の上位にある
LLM 全体の運転原理と接続構造を束ねる設計層として扱うべきである。

関連ノート

- LLM Control Layer
- Intent Interpretation
- Goal Framing
- Task Routing
- Mode Selection
- Constraint Monitor
- Tool Orchestration
- Direct Answer Mode
- Search-Augmented Mode
- File-Grounded Mode
- Comparative Reasoning Mode
- Stepwise Reasoning Mode
- Artifact Generation Mode
- Termination Control
- LLM Output Layer